# BayOPy user guide

One experiment interface, your own objectives and data

1. Synthetic Functions Optimizations
2. Real Data Formatting
   2.1 Real Life Objective Optimization
3. Early Stopping
4. Different Settings Examples

Eight independent main comparisons submit all 19 algorithms across four synthetic objectives and four distinct real datasets. Every method receives five trials, each capped at 50 evaluations. Early stopping and selected configuration changes are demonstrated separately.

All executable examples import only BayOPy and use Python built-ins. Each experiment block contains its own inputs, configuration and environment, and can be copied and run independently.

20 executed code blocks | 15 captured figures | 760 / 760 main algorithm trials completed

Assembled 2026-09-10; figures and summaries refreshed from saved result JSONs

Explanations, code and figures follow the current notebook. Terminal outputs are omitted. Refreshing outputs did not rerun optimization. The complete LaTeX project compiles this PDF without executing experiments. Finite-budget results and targeted checks do not establish universal convergence or a crash-free guarantee.

# Contents

**1. Synthetic Functions Optimizations** **3**

How to read the full configurations . . . 3

1.1 Stochastic: all algorithms . . . 5

1.2 Contextual: all algorithms . . . 14

1.3 Distributional: all algorithms . . . 23

1.4 Adversarial: all algorithms . . . 32

**2. Real Data Formatting** **42**

Stochastic CSV . . . 42

Contextual CSV . . . 43

Distributional CSV . . . 44

Adversarial CSV . . . 45

2.1 Real Life Objective Optimization . . . 46

**3. Early Stopping** **83**

Accuracy and interpretation of the measured results . . . 85

**4. Different Settings Examples** **88**

Defaults and a list of algorithms . . . 88

Acquisition settings: UCB beta and EI offset . . . 89

GP kernels inside a contextual real-data experiment . . . 90

Sampling and batches . . . 92

Save, resume, and explicit display . . . 93

**Appendix A. Algorithm adaptations across environments** **95**

A.1. Stochastic algorithms . . . 95

A.2. Adversarial and robust algorithms . . . 96

A.3. Distributional algorithms . . . 97

A.4. Contextual algorithms . . . 98

A.5. Shared precedence, warnings, and limits . . . 98

# How to use this guide

**Saved-output refresh.** Experiment summaries and figures below were rebuilt from the current saved result JSON files, without running optimization again. Each refreshed output identifies its source file and preserves recorded warnings and stopping reasons. Execution counters are cleared for those cells to distinguish this refresh from running the code. Editing an experiment configuration does not change its saved measurements until you execute it again. The distributional example uses the completed OPSD wind-power run with 5 trials and 50 evaluations.

This guide has four main sections: **1. Synthetic Functions Optimizations**, **2. Real Data Formatting** (including **2.1 Real Life Objective Optimization**), **3. Early Stopping**, and **4. Different Settings Examples**.

Install the updated checkout with `python -m pip install .`, or install its supplied wheel. Installing BayOPy installs its runtime dependencies. Every executable example imports only `bayopy` and uses Python built-ins. No other package imports or separate dependency-install commands are required. These changes must be released before a package-index installation provides this exact API. Open this notebook with `python -m bayopy notebook docs/bayopy_last_ver_user_guide.ipynb`.

Each experiment block runs independently: copy the entire block, including its import and configuration definitions. Earlier blocks are not prerequisites. The shared `code/` folder contains these editable Python blocks. Data files live in the project's external `data` directory; locate this guide within that project, or edit the explicit data path.

The execution manifest records the Python version, platform and backend versions reported by `bayopy.runtime_info()`. The exact dependency snapshot is retained at `execution_requirements.txt` and included with the editable Overleaf source. BayOPy installs its runtime dependencies automatically; these recorded versions identify the environment that produced the figures.

`Experiment` is the single runner. Its `algorithm` argument accepts one algorithm, a list/tuple of algorithms, or a label-to-algorithm dictionary. A dictionary makes labels explicit when comparing several configurations of the same method. Multiple algorithms receive independent environments and stopping rules; each has its own 50-evaluation budget. Individual failures are reported and the other methods continue. With the default `trials=1`, single-algorithm exceptions remain visible to the caller. With `trials>1`, a failed single-algorithm trial is recorded and the remaining seeds continue.

**There is no profile setting.** Omitting optional configuration fields uses the algorithm's dataclass defaults; required scientific choices must still be supplied. No preset chooses your objective, environment, threshold, or budget. The expanded configurations below show algorithm hyperparameters, including options inactive for the selected method variant. Environment observations and support belong to the environment, not to algorithm configurations. Example compute budgets are deliberately modest and are not convergence claims.

# 1. Synthetic Functions Optimizations

Define an objective as an ordinary Python function returning one finite scalar. BayOPy maximizes reward by default, so negate a loss to minimize it. There is no predefined synthetic benchmark inside the library. You can replace any function body below with your own calculation or simulator.

A stochastic function receives the chosen action. A contextual function also receives a context revealed before selection. A distributional function receives the context sampled after action selection; the learner sees a reference distribution beforehand. An adversarial function receives the evaluated coordinate after the environment has applied its attack.

These four definitions are repeated in the independent experiments below. Only the distributional function includes an added sinusoidal component. Observation noise is configured separately in each environment.

**Editable Python / block 1**

```
import bayopy

def stochastic_objective(values):
    target = (0.2, 0.65, 0.4)
    return -sum((x - t) ** 2 for x, t in zip(values, target))


def contextual_objective(values, context):
    c = float(context[0])
    target = (0.2 + 0.5 * c, 0.75 - 0.4 * c, 0.4)
    return -sum((x - t) ** 2 for x, t in zip(values, target))


def reveal_context(*, round_index, generator):
    return [float(round_index % 2)]


def distributional_objective(values, context):
    c = float(context[0])
    target = (0.25 + 0.35 * c, 0.6 - 0.2 * c, 0.3)
    distance = sum((x - t) ** 2 for x, t in zip(values, target))
    wave = 0.08 * bayopy.sin(6.0 * 3.141592653589793 * values[0])
    return float(-distance + wave)


def adversarial_objective(values):
    # This function receives the evaluated (already perturbed) coordinate.
    target = (0.3, 0.7, 0.5)
    return -sum((x - t) ** 2 for x, t in zip(values, target))


def apply_attack(action, *, round_index, generator):
    direction = 1.0 if round_index % 2 else -1.0
    changed = min(1.0, max(0.0, float(action[0]) + 0.05 * direction))
    return [changed - float(action[0]), 0.0, 0.0]
```

## How to read the full configurations

### Five independent trials and vertical uncertainty bars

Every optimization block uses `experiment.run(trials=5, max_evaluations=50)`. Trial 1 uses each algorithm's configured seed; trials 2-5 add 1-4. A base seed of 42 therefore becomes 42, 43, 44, 45, 46. Explicit nested acquisition seeds and the environment's own base seed advance by the same trial offset. Each trial starts with fresh algorithm state, embeddings, environment history and stopping rules. Configured previous data remain available to every trial. The total budget is up to 250 evaluations per algorithm, 50 per trial.

`trials=1` (the default) preserves the existing single-run result type. Repeated runs return `ExperimentTrialsResult`: `result.results[label]` contains the individual outcomes (or `None` for failed methods), `result.statistics[label]` contains means, sample standard deviations and contributing counts, and `experiment.trial_experiments` exposes the independent trial coordinators. The original algorithm objects serve as configuration templates.

`bayopy.plot(experiment)` draws mean reward and mean per-trial incumbent curves. It computes each trial's best-so-far curve before averaging. Vertical bars show mean $\pm$ 1.96 sample standard deviations (`uncertainty="std95"`), describing between-trial variability. They are not a 95% confidence interval for the mean, and five trials do not establish 95% coverage. Use `uncertainty="std"` for one sample SD or `uncertainty=None` for means only. No shaded bands are used. `errorbar_cycles=5` controls the target repetition frequency; BayOPy scales spacing with the evaluation range and number of methods, and gives each method a different offset. All bars use actual integer evaluation positions within the data. If there are more methods than evaluations, some methods cannot have distinct bar positions. No SD bar is drawn when only one trial contributes. Changing the seed does not force randomness: deterministic finite-table policies can follow identical trajectories across trials, giving zero sample SD and invisible zero-height bars.

Early-stopped or failed histories are never extended. Later means can therefore use fewer trials; inspect `result.statistics[label].count` before comparing tails. Failed trials and their partial measurements remain in the audit data. Five-trial success does not validate every algorithm's modeling assumptions.

Checkpoints live under `trials/trial-001/` through `trials/trial-005/`; each multi-algorithm trial has its own `algorithms/` subdirectories. The root JSON stores aggregate statistics and all per-trial records. To resume, reconstruct the same base configurations and output directory with `resume=True`, then call `run(trials=5, max_evaluations=50)`. Do not increment seeds yourself. Completed evaluations are retained; increasing a run-call ceiling extends each trial, while completed trials at the requested ceiling perform no new evaluations. Changing the saved trial count, base seeds or immutable algorithm/environment settings is rejected. Mutable numerical fit budgets follow the checkpoint compatibility policy. Stateful external objectives still need their own durable side-effect handling.

## Environment information versus algorithm settings

The environment supplies observations during optimization; algorithms do not receive the environment object in their configuration. The papers specify an information protocol, not a Python constructor design. Known domains and kernels are legitimate initial information, whereas future observations and the hidden sampling distribution must not leak into decisions.

| Method | Before choosing an action | After choosing an action | Algorithm settings |
|---|---|---|---|
| CGP_UCB | Current context from environment | Reward | Action/context kernels, confidence, numerical solver |
| DRBO | Public context support, current reference distribution and ambiguity margin | Realized context and reward | Kernel, confidence, divergence and solver |
| RoBOS | Public context support and current reference distribution | Realized context and reward | Aspiration, kernel and fragility solver |
| AdveRS | Feasible action domain | Perturbed action and reward | Aspiration, metric and solver; actual attack budget is unknown |
| StableOPT | Declared robustness model | Evaluated action and reward | Chosen perturbation radius and metric |

Contexts, support and reference probabilities are environment inputs. They are not fields to set through `AlgorithmConfig()`. Environment-provided values take precedence over the algorithm's internal fallback values. Missing current context uses an internal zero context of the required dimension. A distributional environment reveals its realized context only after the action: CGP_UCB therefore chooses using the zero default, then trains on the actual observed context. The warning explains this mismatch; the future realized context is not leaked into decisions. A distributional algorithm outside its native environment uses a point mass at the current environment context, or at zero if no context was supplied. Missing reference probabilities use a uniform distribution over known support, and a missing ambiguity margin uses zero. The warning records

the specific adaptation used. The true sampling distribution remains private. A changed reference distribution is allowed; changing the finite support during a run is rejected. `context_kernel` is a model choice, not an observed context. Likewise, `distribution_grid` and `distribution_floor` are numerical search controls, not the environment's reference or sampling probabilities.

The AdveRS examples use `FullDomainPerturbations`: the actual attack stays in the environment. Its L2 metric is a modeling choice; the paper's kernel-metric guarantees do not automatically transfer to it. StableOPT's robustness radius is a user-selected decision criterion, not an observation of the next attack. These distinctions follow the original CGP-UCB paper, DRBO Algorithm 1, RoBOS Algorithm 1, and AdveRS formulation.

All 19 methods are submitted once per trial in each environment: 14 stochastic, one contextual (CGP_UCB), two distributional (DRBO and RoBOS), and two robust or adversarial methods (StableOPT and AdveRS). Cross-environment execution is automatic when the available internal defaults make it technically possible. Extra context, distribution or attack metadata are ignored by algorithms that do not need them. Missing attack feedback means the nominal action is also the evaluated action: no perturbation is invented. Warnings identify the non-native pairing, the missing information, and each fallback used. A genuine inability to select or evaluate an action still raises an error; non-standard modeling assumptions alone do not stop an experiment.

Required embedding dimensions, decomposition groups, thresholds, aspirations, and robustness settings are visible in algorithm code. Context support belongs to the environment configuration. Optional kernel names apply only where a method exposes a kernel field; some methods fix a source-specific kernel. An isotropic kernel uses one lengthscale across all its inputs. Setting `ard_num_dims` requests separate lengthscales. `active_dims=None` gives the GP every relevant action/context coordinate.

For `AcquisitionConfig` UCB, beta is inside the square root: mean + sqrt(beta) times standard deviation. CGP_UCB and DRBO use the same beta convention. MCTS_VS instead uses its own `acquisition_beta` directly as the standard- deviation multiplier. EI uses `xi` as its improvement offset. Standard acquisition objects are not interchangeable with algorithm-specific robust decision rules. Sequential examples use `batch_size=1, q_batch=None`. Exactly one batching mode may be active. A method that only supports one action per round rejects a larger batch.

Initial designs, candidate counts, GP optimizer steps, NUTS settings, solver tolerances, and presentation flags are exposed. These numerical settings affect speed and approximation quality. SAASBO runs genuine NUTS and prints its diagnostics; a short chain with warnings is not a validated posterior. ALEBO's current `metric_inference="laplace"` option records fixed-scale local metric perturbations; it is not Hessian-derived Laplace inference, and acquisition decisions use the fitted GP rather than integrating those recorded samples. The numerical audit below states this limitation. Changing an environment changes an experiment's assumptions; a cross-family curve does not inherit the original paper's guarantees.

Main experiments use only `max_evaluations=50`; they have no patience, target, or time stopping rules. Initialization counts toward this budget. A failure or exhausted finite source may still prevent completion; diagnostics identify it. Every figure ends at evaluation 50 and never extends partial histories.

## 1.1 Stochastic: all algorithms

```
This block defines its own objective, providers, settings, environment,
and results directory. Every method is reconstructed with explicit
```

**Editable Python / block 2**

```python
import bayopy

seed = bayopy.set_seed(110, threads=1)
run_directory = bayopy.new_run_directory("results/last_guide/synthetic_stochastic")
bounds = bayopy.Bounds(lower=[0.0] * 3, upper=[1.0] * 3)

def stochastic_objective(values):
    target = (0.2, 0.65, 0.4)
    return -sum((x - t) ** 2 for x, t in zip(values, target))
```

```
# Fresh, fully explicit configurations for stochastic observations.
# These bounded computation settings are editable examples.
# No setting overrides are hidden behind a profile.
dimension = bounds.dimension
reduced_dimension = min(2, dimension)
constructor_options = {
    'maximize': True,
    'dtype': 'float64',
    'device': 'cpu',
    'plot': False,
    'verbose': False,
    'verbose_setting': False,
    'verbose_iter_by_iter': False,
}

# Isotropic kernels model every input; no active coordinates are given.
action_kernel = bayopy.KernelConfig(
    name='matern52',
    ard_num_dims=None,
    active_dims=None,
    use_outputscale=True,
    lengthscale_prior=None,
    outputscale_prior=None,
    kwargs={},
)
context_kernel = bayopy.KernelConfig(
    name='rbf',
    ard_num_dims=None,
    active_dims=None,
    use_outputscale=True,
    lengthscale_prior=None,
    outputscale_prior=None,
    kwargs={},
)
ei = bayopy.AcquisitionConfig(
    name='EI',
    beta=3.0,
    xi=0.0,
    mc_samples=64,
    seed=seed,
)
ucb = bayopy.AcquisitionConfig(
    name='UCB',
    beta=3.0,
    xi=0.0,
    mc_samples=64,
    seed=seed,
)
contextual_solver = bayopy.AcquisitionOptimizerSpec(
    raw_samples=32,
    restarts=2,
    maxiter=50,
    fit_maxiter=50,
    solver_tolerance=1e-06,
)
distributional_solver = bayopy.ActionOptimizerSpec(
    raw_samples=24,
    restarts=2,
    maxiter=50,
    fit_maxiter=50,
)
robust_solver = bayopy.MultiStartSolver(
    method='multistart',
)

rembo_config = bayopy.REMBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
```

```
    Y_lower=None,
    Y_upper=None,
    kernel_type='low_dimensional',
    custom_kernel=None,
    acquisition='EI',
    initial_points=8,
    initial_points_sampling_method='sobol',
    batch_size=1,
    q_batch=None,
    lengthscale_lower=0.01,
    lengthscale_upper=50.0,
    initial_lengthscale=None,
    t_sigma=0.002,
    refit_interval=20,
    exploitation_count_limit=5,
    upper_bound_multiplier=0.9,
    seed=seed + 0,
)

hesbo_config = bayopy.HeSBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
    acquisition=ei,
    initial_points=8,
    initial_points_sampling_method='sobol',
    hash_family='paper',
    batch_size=1,
    q_batch=None,
    seed=seed + 1,
)

alebo_config = bayopy.ALEBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
    initial_points=8,
    initial_points_sampling_method='sobol',
    acquisition=ei,
    metric_inference='laplace',
    metric_posterior_samples=8,
    feasibility_tolerance=1e-09,
    acquisition_solver='slsqp',
    raw_feasible_samples=64,
    num_restarts=2,
    batch_size=1,
    q_batch=None,
    seed=seed + 2,
)

add_gp_ucb_config = bayopy.AddGPUCBConfig(
    groups=tuple((i,) for i in range(dimension)),
    maximum_group_size=1,
    learn_decomposition=False,
    decomposition_candidates=None,
    refresh_cycle=25,
    beta=3.0,
    initial_points=8,
    initial_points_sampling_method='sobol',
    component_optimizer=None,
    batch_size=1,
    q_batch=None,
    seed=seed + 3,
)

turbo_config = bayopy.TuRBOConfig(
    batch_size=1,
    q_batch=None,
    initial_points=8,
    initial_points_sampling_method='sobol',
```

```
    length_init=0.8,
    length_min=0.0078125,
    length_max=1.6,
    success_tolerance=3,
    failure_tolerance=None,
    candidate_count=128,
    perturbation_cap=20.0,
    acquisition=ei,
    success_relative_tolerance=0.001,
    noisy_center='posterior_mean',
    max_cholesky_size=2000,
    gp_training_steps=50,
    seed=seed + 4,
)

dropout_config = bayopy.DropoutConfig(
    active_dimension=reduced_dimension,
    fill_strategy='mix',
    random_fill_probability=0.1,
    beta=3.0,
    acquisition_optimizer=None,
    initial_points=8,
    initial_points_sampling_method='sobol',
    batch_size=1,
    q_batch=None,
    seed=seed + 5,
)

mcts_vs_config = bayopy.MCTSVSConfig(
    Nv=2,
    Ns=2,
    archive_size=20,
    initial_points=8,
    initial_points_sampling_method='sobol',
    bad_count_limit=5,
    split_threshold=3,
    exploration_constant=0.1,
    inner_optimizer='bo',
    candidate_pool_size=64,
    kernel_lengthscale=0.2,
    acquisition_beta=3.0,
    gp_maxiter=50,
    batch_size=1,
    q_batch=None,
    seed=seed + 6,
)

saasbo_config = bayopy.SAASBOConfig(
    shrinkage_scale=0.1,
    warmup_steps=64,
    posterior_samples=64,
    thinning=4,
    num_chains=1,
    initial_points=8,
    initial_points_sampling_method='sobol',
    acquisition='EI',
    max_tree_depth=4,
    target_accept_prob=0.8,
    num_restarts=2,
    raw_samples=32,
    mc_samples=32,
    ucb_beta=3.0,
    max_r_hat=1.1,
    min_effective_sample_size=10.0,
    allow_divergences=False,
    batch_size=1,
    q_batch=None,
    seed=seed + 7,
```

```
)

ag_vs_config = bayopy.AG_VSConfig(
    effective_dimension=reduced_dimension,
    sequence_length=2,
    split_groups=2,
    initial_points=8,
    initial_points_sampling_method='sobol',
    acquisition=ucb,
    stopping_threshold=None,
    stopping_delta=None,
    stagnation_patience=None,
    gp_fit_method='mle',
    gp_maxiter=50,
    acquisition_restarts=2,
    acquisition_raw_samples=64,
    batch_size=1,
    q_batch=None,
    seed=seed + 8,
)

la_mcts_config = bayopy.LAMCTSConfig(
    initial_points=8,
    initial_points_sampling_method='sobol',
    exploration_constant=1.0,
    split_threshold=12,
    min_child_size=3,
    max_depth=20,
    classifier='svm_rbf',
    local_sampler='turbo',
    candidate_pool_size=64,
    max_resampling_attempts=4,
    kernel_lengthscale=0.2,
    acquisition_beta=3.0,
    gp_maxiter=50,
    batch_size=1,
    q_batch=None,
    seed=seed + 9,
)

pg_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=8,
    initial_points_sampling_method='sobol',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 10,
)

eg_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=8,
    initial_points_sampling_method='sobol',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
```

```
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 11,
)

gs_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=8,
    initial_points_sampling_method='sobol',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 12,
)

satisficing_ts_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=8,
    initial_points_sampling_method='sobol',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 13,
)

cgp_ucb_config = bayopy.CGPUCBConfig(
    action_bounds=bounds,
    action_set=None,
    action_kernel=action_kernel,
    context_kernel=context_kernel,
    kernel_combination='product',
    beta=3.0,
    delta=0.1,
    finite_domain_cardinality=None,
    acquisition_optimizer=contextual_solver,
    initial_points=8,
    batch_size=1,
    q_batch=None,
    seed=seed + 14,
)

drbo_config = bayopy.DRBOConfig(
    context_kernel=context_kernel,
    action_kernel=action_kernel,
    beta=3.0,
```

```
    ambiguity_metric='mmd',
    inner_solver='slsqp',
    action_optimizer=distributional_solver,
    distribution_grid=None,
    enumeration_resolution=20,
    initial_points=8,
    feasibility_tolerance=1e-08,
    optimizer_tolerance=1e-06,
    batch_size=1,
    q_batch=None,
    seed=seed + 15,
)

robos_config = bayopy.RoBOSConfig(
    aspiration=-0.1,
    context_kernel=context_kernel,
    action_kernel=action_kernel,
    joint_kernel_combination='product',
    joint_kernel=None,
    beta=3.0,
    simplex_solver='slsqp',
    distribution_floor=0.0,
    distribution_grid=None,
    enumeration_resolution=20,
    action_raw_samples=24,
    action_restarts=2,
    action_maxiter=50,
    initial_points=8,
    fit_maxiter=50,
    feasibility_tolerance=1e-08,
    optimizer_tolerance=1e-06,
    batch_size=1,
    q_batch=None,
    seed=seed + 16,
)

stableopt_config = bayopy.StableOPTConfig(
    perturbation_radius=0.08,
    distance=bayopy.L2Distance(),
    beta=3.0,
    inner_solver=robust_solver,
    outer_solver=robust_solver,
    inner_raw_samples=16,
    outer_raw_samples=32,
    inner_restarts=2,
    outer_restarts=2,
    solver_tolerance=1e-06,
    initial_points=8,
    fit_maxiter=50,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 17,
)

advers_config = bayopy.AdveRSConfig(
    variant='RS1',
    aspiration=-0.1,
    p=1.0,
    beta=3.0,
    perturbation_domain=bayopy.FullDomainPerturbations(norm='l2', distance_floor=1e-8),
    candidate_solver=robust_solver,
    inner_solver=robust_solver,
    decision_model='ucb',
    outer_raw_samples=32,
    inner_raw_samples=16,
    radius_bisection_steps=8,
    solver_tolerance=1e-06,
```

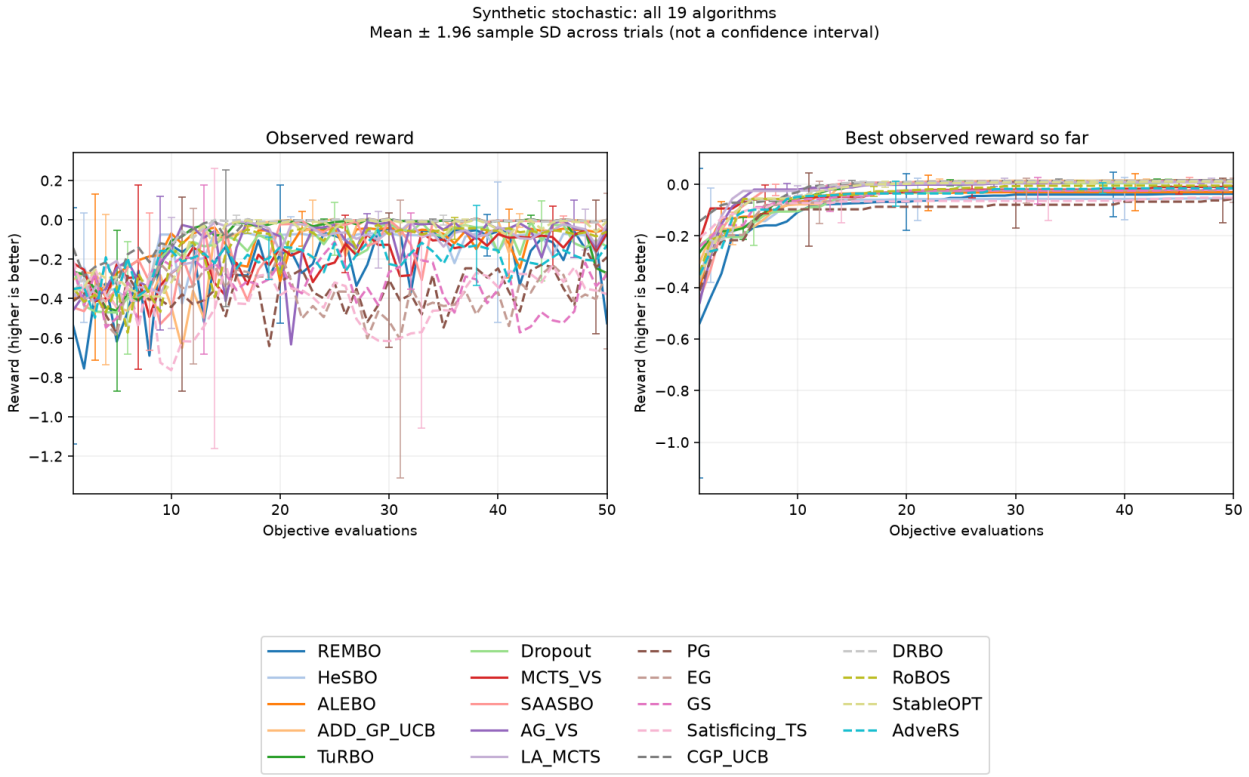
```
    initial_points=8,
    fit_maxiter=50,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 18,
)

# SAASBO uses actual NUTS. Short chains can report poor ESS/divergences.
# Warnings retain finite draws; they do not certify convergence.
# For inference increase warmup/draws and use at least two chains.
algorithms = {
    'REMBO': bayopy.REMBO(
        bounds=bounds, config=rembo_config,
        **constructor_options,
    ),
    'HeSBO': bayopy.HeSBO(
        bounds=bounds, config=hesbo_config,
        **constructor_options,
    ),
    'ALEBO': bayopy.ALEBO(
        bounds=bounds, config=alebo_config,
        **constructor_options,
    ),
    'ADD_GP_UCB': bayopy.ADD_GP_UCB(
        bounds=bounds, config=add_gp_ucb_config,
        **constructor_options,
    ),
    'TuRBO': bayopy.TuRBO(
        bounds=bounds, config=turbo_config,
        **constructor_options,
    ),
    'Dropout': bayopy.Dropout(
        bounds=bounds, config=dropout_config,
        **constructor_options,
    ),
    'MCTS_VS': bayopy.MCTS_VS(
        bounds=bounds, config=mcts_vs_config,
        **constructor_options,
    ),
    'SAASBO': bayopy.SAASBO(
        bounds=bounds, config=saasbo_config,
        **constructor_options,
        diagnostic_policy='warn',
        jit_compile=True,
        mcmc_num_threads=1,
    ),
    'AG_VS': bayopy.AG_VS(
        bounds=bounds, config=ag_vs_config,
        **constructor_options,
    ),
    'LA_MCTS': bayopy.LA_MCTS(
        bounds=bounds, config=la_mcts_config,
        **constructor_options,
    ),
    'PG': bayopy.PG(
        bounds=bounds, config=pg_config,
        **constructor_options,
    ),
    'EG': bayopy.EG(
        bounds=bounds, config=eg_config,
        **constructor_options,
    ),
    'GS': bayopy.GS(
        bounds=bounds, config=gs_config,
        **constructor_options,
    ),
    'Satisficing_TS': bayopy.Satisficing_TS(
```

```
        bounds=bounds, config=satisficing_ts_config,
        **constructor_options,
    ),
    'CGP_UCB': bayopy.CGP_UCB(
        bounds=bounds, config=cgp_ucb_config,
        **constructor_options,
    ),
    'DRBO': bayopy.DRBO(
        bounds=bounds, config=drbo_config,
        **constructor_options,
    ),
    'RoBOS': bayopy.RoBOS(
        bounds=bounds, config=robos_config,
        **constructor_options,
    ),
    'StableOPT': bayopy.StableOPT(
        bounds=bounds, config=stableopt_config,
        **constructor_options,
    ),
    'AdveRS': bayopy.AdveRS(
        bounds=bounds, config=advers_config,
        **constructor_options,
    ),
}
print('Configured algorithms:', len(algorithms), list(algorithms))


environment = bayopy.StochasticEnvironment(
    bounds=bounds, function=stochastic_objective,
    dataset=None, previous_data=None, dtype="float64", device="cpu",
    noise_std=0.01, noise_fn=None, add_noise_to_data=False,
    allow_repeated_actions=False, data_row_selection="first", seed=seed,
)

assert set(algorithms) == set(bayopy.algorithm_names())
print("Algorithms submitted:", len(algorithms))
print("Names:", list(algorithms))
experiment = bayopy.Experiment(
    environment=environment, algorithm=algorithms,
    output_directory=run_directory, resume=False, save_every=10,
    stopping_rule=None,
)
result = experiment.run(
    trials=5, max_evaluations=50, plot=False, show=False, progress=True,
)
print(experiment.summary())
print(experiment.diagnostics())
figure = bayopy.plot(
    experiment, kind="comparison", title="Synthetic stochastic: all 19 algorithms",
    titles=("Observed reward", "Best observed reward so far"),
    xlabel="Objective evaluations", ylabel="Reward (higher is better)",
    figsize=(12.0, 7.5), xlim=(1, 50), grid=True,
    legend=True, legend_columns=4, legend_fontsize=12.0,
    save_path=bayopy.join_path(run_directory, "all_algorithms.png"),
    dpi=180, show=True,
)
print("Saved results:", run_directory)
```




Captured figure 1 / synthetic stochastic

## 1.2 Contextual: all algorithms

This block defines its own objective, providers, settings, environment, and results directory. Every method is reconstructed with explicit

### Editable Python / block 3

```
import bayopy

seed = bayopy.set_seed(111, threads=1)
run_directory = bayopy.new_run_directory("results/last_guide/synthetic_contextual")
bounds = bayopy.Bounds(lower=[0.0] * 3, upper=[1.0] * 3)

def contextual_objective(values, context):
    c = float(context[0])
    target = (0.2 + 0.5 * c, 0.75 - 0.4 * c, 0.4)
    return -sum((x - t) ** 2 for x, t in zip(values, target))


def reveal_context(*, round_index, generator):
    return [float(round_index % 2)]

# Fresh, fully explicit configurations for contextual observations.
# These bounded computation settings are editable examples.
# No setting overrides are hidden behind a profile.
dimension = bounds.dimension
reduced_dimension = min(2, dimension)
constructor_options = {
    'maximize': True,
    'dtype': 'float64',
    'device': 'cpu',
    'plot': False,
    'verbose': False,
    'verbose_setting': False,
```

```
    'verbose_iter_by_iter': False,
}

# Isotropic kernels model every input; no active coordinates are given.
action_kernel = bayopy.KernelConfig(
    name='rbf',
    ard_num_dims=None,
    active_dims=None,
    use_outputscale=True,
    lengthscale_prior=None,
    outputscale_prior=None,
    kwargs={},
)
context_kernel = bayopy.KernelConfig(
    name='rbf',
    ard_num_dims=None,
    active_dims=None,
    use_outputscale=True,
    lengthscale_prior=None,
    outputscale_prior=None,
    kwargs={},
)
ei = bayopy.AcquisitionConfig(
    name='EI',
    beta=2.5,
    xi=0.0,
    mc_samples=64,
    seed=seed,
)
ucb = bayopy.AcquisitionConfig(
    name='UCB',
    beta=2.5,
    xi=0.0,
    mc_samples=64,
    seed=seed,
)
contextual_solver = bayopy.AcquisitionOptimizerSpec(
    raw_samples=32,
    restarts=2,
    maxiter=50,
    fit_maxiter=50,
    solver_tolerance=1e-06,
)
distributional_solver = bayopy.ActionOptimizerSpec(
    raw_samples=24,
    restarts=2,
    maxiter=50,
    fit_maxiter=50,
)
robust_solver = bayopy.MultiStartSolver(
    method='multistart',
)

rembo_config = bayopy.REMBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
    Y_lower=None,
    Y_upper=None,
    kernel_type='low_dimensional',
    custom_kernel=None,
    acquisition='EI',
    initial_points=9,
    initial_points_sampling_method='uniform',
    batch_size=1,
    q_batch=None,
    lengthscale_lower=0.01,
    lengthscale_upper=50.0,
    initial_lengthscale=None,
```

```
    t_sigma=0.002,
    refit_interval=20,
    exploitation_count_limit=5,
    upper_bound_multiplier=0.9,
    seed=seed + 0,
)

hesbo_config = bayopy.HeSBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
    acquisition=ei,
    initial_points=9,
    initial_points_sampling_method='uniform',
    hash_family='paper',
    batch_size=1,
    q_batch=None,
    seed=seed + 1,
)

alebo_config = bayopy.ALEBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
    initial_points=9,
    initial_points_sampling_method='uniform',
    acquisition=ei,
    metric_inference='laplace',
    metric_posterior_samples=8,
    feasibility_tolerance=1e-09,
    acquisition_solver='slsqp',
    raw_feasible_samples=64,
    num_restarts=2,
    batch_size=1,
    q_batch=None,
    seed=seed + 2,
)

add_gp_ucb_config = bayopy.AddGPUCBConfig(
    groups=tuple((i,) for i in range(dimension)),
    maximum_group_size=1,
    learn_decomposition=False,
    decomposition_candidates=None,
    refresh_cycle=25,
    beta=2.5,
    initial_points=9,
    initial_points_sampling_method='uniform',
    component_optimizer=None,
    batch_size=1,
    q_batch=None,
    seed=seed + 3,
)

turbo_config = bayopy.TuRBOConfig(
    batch_size=1,
    q_batch=None,
    initial_points=9,
    initial_points_sampling_method='uniform',
    length_init=0.8,
    length_min=0.0078125,
    length_max=1.6,
    success_tolerance=3,
    failure_tolerance=None,
    candidate_count=128,
    perturbation_cap=20.0,
    acquisition=ei,
    success_relative_tolerance=0.001,
    noisy_center='posterior_mean',
    max_cholesky_size=2000,
    gp_training_steps=50,
```

```
    seed=seed + 4,
)

dropout_config = bayopy.DropoutConfig(
    active_dimension=reduced_dimension,
    fill_strategy='mix',
    random_fill_probability=0.1,
    beta=2.5,
    acquisition_optimizer=None,
    initial_points=9,
    initial_points_sampling_method='uniform',
    batch_size=1,
    q_batch=None,
    seed=seed + 5,
)

mcts_vs_config = bayopy.MCTSVSConfig(
    Nv=2,
    Ns=2,
    archive_size=20,
    initial_points=9,
    initial_points_sampling_method='uniform',
    bad_count_limit=5,
    split_threshold=3,
    exploration_constant=0.1,
    inner_optimizer='bo',
    candidate_pool_size=64,
    kernel_lengthscale=0.2,
    acquisition_beta=2.5,
    gp_maxiter=50,
    batch_size=1,
    q_batch=None,
    seed=seed + 6,
)

saasbo_config = bayopy.SAASBOConfig(
    shrinkage_scale=0.1,
    warmup_steps=64,
    posterior_samples=64,
    thinning=4,
    num_chains=1,
    initial_points=9,
    initial_points_sampling_method='uniform',
    acquisition='EI',
    max_tree_depth=4,
    target_accept_prob=0.8,
    num_restarts=2,
    raw_samples=32,
    mc_samples=32,
    ucb_beta=2.5,
    max_r_hat=1.1,
    min_effective_sample_size=10.0,
    allow_divergences=False,
    batch_size=1,
    q_batch=None,
    seed=seed + 7,
)

ag_vs_config = bayopy.AG_VSConfig(
    effective_dimension=reduced_dimension,
    sequence_length=2,
    split_groups=2,
    initial_points=9,
    initial_points_sampling_method='uniform',
    acquisition=ucb,
    stopping_threshold=None,
    stopping_delta=None,
    stagnation_patience=None,
```

```
    gp_fit_method='mle',
    gp_maxiter=50,
    acquisition_restarts=2,
    acquisition_raw_samples=64,
    batch_size=1,
    q_batch=None,
    seed=seed + 8,
)

la_mcts_config = bayopy.LAMCTSConfig(
    initial_points=9,
    initial_points_sampling_method='uniform',
    exploration_constant=1.0,
    split_threshold=12,
    min_child_size=3,
    max_depth=20,
    classifier='svm_rbf',
    local_sampler='turbo',
    candidate_pool_size=64,
    max_resampling_attempts=4,
    kernel_lengthscale=0.2,
    acquisition_beta=2.5,
    gp_maxiter=50,
    batch_size=1,
    q_batch=None,
    seed=seed + 9,
)

pg_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=9,
    initial_points_sampling_method='uniform',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 10,
)

eg_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=9,
    initial_points_sampling_method='uniform',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 11,
)

gs_config = bayopy.GoodActionConfig(
    threshold=-0.1,
```

```
    candidate_domain=None,
    initial_points=9,
    initial_points_sampling_method='uniform',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 12,
)

satisficing_ts_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=9,
    initial_points_sampling_method='uniform',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 13,
)

cgp_ucb_config = bayopy.CGPUCBConfig(
    action_bounds=bounds,
    action_set=None,
    action_kernel=action_kernel,
    context_kernel=context_kernel,
    kernel_combination='product',
    beta=2.5,
    delta=0.1,
    finite_domain_cardinality=None,
    acquisition_optimizer=contextual_solver,
    initial_points=9,
    batch_size=1,
    q_batch=None,
    seed=seed + 14,
)

drbo_config = bayopy.DRBOConfig(
    context_kernel=context_kernel,
    action_kernel=action_kernel,
    beta=2.5,
    ambiguity_metric='mmd',
    inner_solver='slsqp',
    action_optimizer=distributional_solver,
    distribution_grid=None,
    enumeration_resolution=20,
    initial_points=9,
    feasibility_tolerance=1e-08,
    optimizer_tolerance=1e-06,
    batch_size=1,
    q_batch=None,
    seed=seed + 15,
)
```

```
robos_config = bayopy.RoBOSConfig(
    aspiration=-0.1,
    context_kernel=context_kernel,
    action_kernel=action_kernel,
    joint_kernel_combination='product',
    joint_kernel=None,
    beta=2.5,
    simplex_solver='slsqp',
    distribution_floor=0.0,
    distribution_grid=None,
    enumeration_resolution=20,
    action_raw_samples=24,
    action_restarts=2,
    action_maxiter=50,
    initial_points=9,
    fit_maxiter=50,
    feasibility_tolerance=1e-08,
    optimizer_tolerance=1e-06,
    batch_size=1,
    q_batch=None,
    seed=seed + 16,
)

stableopt_config = bayopy.StableOPTConfig(
    perturbation_radius=0.08,
    distance=bayopy.L2Distance(),
    beta=2.5,
    inner_solver=robust_solver,
    outer_solver=robust_solver,
    inner_raw_samples=16,
    outer_raw_samples=32,
    inner_restarts=2,
    outer_restarts=2,
    solver_tolerance=1e-06,
    initial_points=9,
    fit_maxiter=50,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 17,
)

advers_config = bayopy.AdveRSConfig(
    variant='RS1',
    aspiration=-0.1,
    p=1.0,
    beta=2.5,
    perturbation_domain=bayopy.FullDomainPerturbations(norm='l2', distance_floor=1e-8),
    candidate_solver=robust_solver,
    inner_solver=robust_solver,
    decision_model='ucb',
    outer_raw_samples=32,
    inner_raw_samples=16,
    radius_bisection_steps=8,
    solver_tolerance=1e-06,
    initial_points=9,
    fit_maxiter=50,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 18,
)

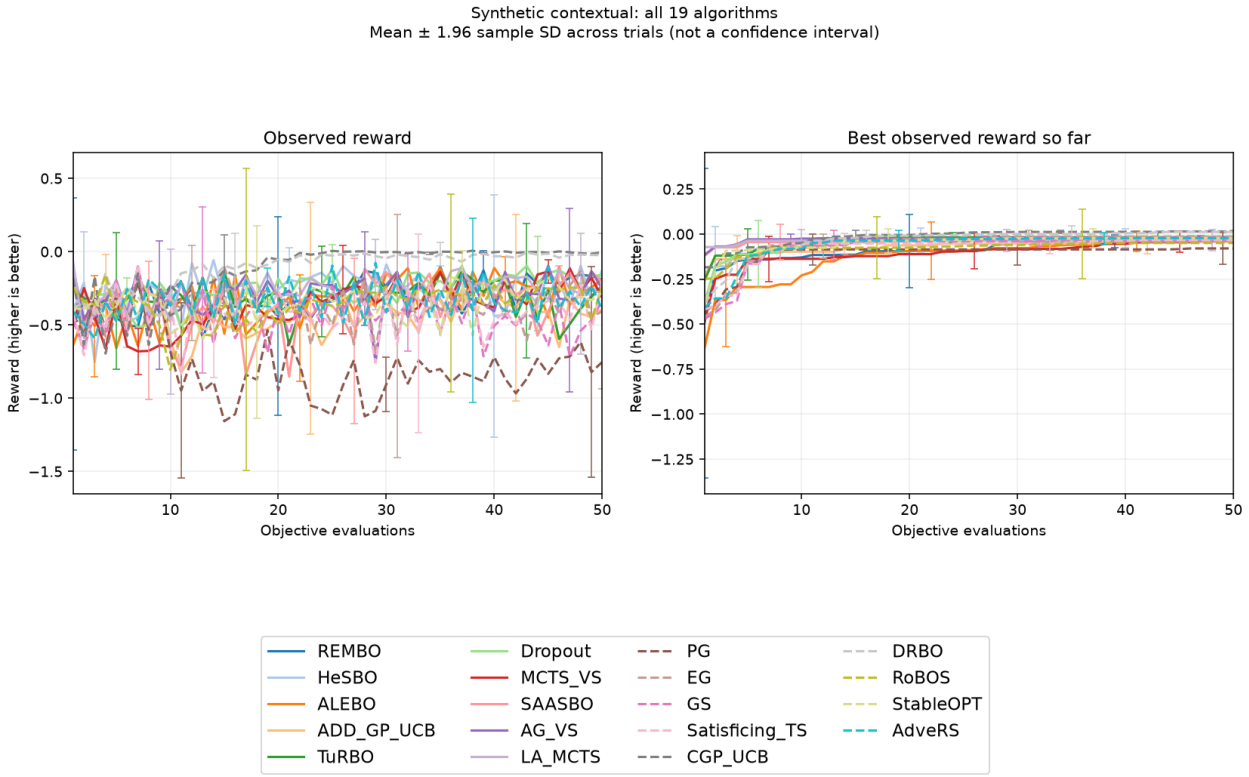
# SAASBO uses actual NUTS. Short chains can report poor ESS/divergences.
# Warnings retain finite draws; they do not certify convergence.
# For inference increase warmup/draws and use at least two chains.
algorithms = {
```

```
    'REMBO': bayopy.REMBO(
        bounds=bounds, config=rembo_config,
        **constructor_options,
    ),
    'HeSBO': bayopy.HeSBO(
        bounds=bounds, config=hesbo_config,
        **constructor_options,
    ),
    'ALEBO': bayopy.ALEBO(
        bounds=bounds, config=alebo_config,
        **constructor_options,
    ),
    'ADD_GP_UCB': bayopy.ADD_GP_UCB(
        bounds=bounds, config=add_gp_ucb_config,
        **constructor_options,
    ),
    'TuRBO': bayopy.TuRBO(
        bounds=bounds, config=turbo_config,
        **constructor_options,
    ),
    'Dropout': bayopy.Dropout(
        bounds=bounds, config=dropout_config,
        **constructor_options,
    ),
    'MCTS_VS': bayopy.MCTS_VS(
        bounds=bounds, config=mcts_vs_config,
        **constructor_options,
    ),
    'SAASBO': bayopy.SAASBO(
        bounds=bounds, config=saasbo_config,
        **constructor_options,
        diagnostic_policy='warn',
        jit_compile=True,
        mcmc_num_threads=1,
    ),
    'AG_VS': bayopy.AG_VS(
        bounds=bounds, config=ag_vs_config,
        **constructor_options,
    ),
    'LA_MCTS': bayopy.LA_MCTS(
        bounds=bounds, config=la_mcts_config,
        **constructor_options,
    ),
    'PG': bayopy.PG(
        bounds=bounds, config=pg_config,
        **constructor_options,
    ),
    'EG': bayopy.EG(
        bounds=bounds, config=eg_config,
        **constructor_options,
    ),
    'GS': bayopy.GS(
        bounds=bounds, config=gs_config,
        **constructor_options,
    ),
    'Satisficing_TS': bayopy.Satisficing_TS(
        bounds=bounds, config=satisficing_ts_config,
        **constructor_options,
    ),
    'CGP_UCB': bayopy.CGP_UCB(
        bounds=bounds, config=cgp_ucb_config,
        **constructor_options,
    ),
    'DRBO': bayopy.DRBO(
        bounds=bounds, config=drbo_config,
        **constructor_options,
    ),
    'RoBOS': bayopy.RoBOS(
```

```
        bounds=bounds, config=robos_config,
        **constructor_options,
    ),
    'StableOPT': bayopy.StableOPT(
        bounds=bounds, config=stableopt_config,
        **constructor_options,
    ),
    'AdveRS': bayopy.AdveRS(
        bounds=bounds, config=advers_config,
        **constructor_options,
    ),
}
print('Configured algorithms:', len(algorithms), list(algorithms))


environment = bayopy.ContextualEnvironment(
    bounds=bounds, function=contextual_objective,
    context_provider=reveal_context,
    dataset=None, previous_data=None, dtype="float64", device="cpu",
    noise_std=0.01, noise_fn=None, add_noise_to_data=False,
    allow_repeated_actions=False, data_row_selection="first", seed=seed,
)

assert set(algorithms) == set(bayopy.algorithm_names())
print("Algorithms submitted:", len(algorithms))
print("Names:", list(algorithms))
experiment = bayopy.Experiment(
    environment=environment, algorithm=algorithms,
    output_directory=run_directory, resume=False, save_every=10,
    stopping_rule=None,
)
result = experiment.run(
    trials=5, max_evaluations=50, plot=False, show=False, progress=True,
)
print(experiment.summary())
print(experiment.diagnostics())
figure = bayopy.plot(
    experiment, kind="comparison", title="Synthetic contextual: all 19 algorithms",
    titles=("Observed reward", "Best observed reward so far"),
    xlabel="Objective evaluations", ylabel="Reward (higher is better)",
    figsize=(12.0, 7.5), xlim=(1, 50), grid=True,
    legend=True, legend_columns=4, legend_fontsize=12.0,
    save_path=bayopy.join_path(run_directory, "all_algorithms.png"),
    dpi=180, show=True,
)
print("Saved results:", run_directory)
```




Captured figure 2 / synthetic contextual

## 1.3 Distributional: all algorithms

This block defines its own objective, providers, settings, environment, and results directory. Every method is reconstructed with explicit

### Editable Python / block 4

```
import bayopy

seed = bayopy.set_seed(112, threads=1)
run_directory = bayopy.new_run_directory("results/last_guide/synthetic_distributional")
bounds = bayopy.Bounds(lower=[0.0] * 3, upper=[1.0] * 3)
contexts = bayopy.array([[0.0], [1.0]], dtype="float64")

def distributional_objective(values, context):
    c = float(context[0])
    target = (0.25 + 0.35 * c, 0.6 - 0.2 * c, 0.3)
    distance = sum((x - t) ** 2 for x, t in zip(values, target))
    wave = 0.08 * bayopy.sin(6.0 * 3.141592653589793 * values[0])
    return float(-distance + wave)

# Fresh, fully explicit configurations for distributional observations.
# These bounded computation settings are editable examples.
# No setting overrides are hidden behind a profile.
dimension = bounds.dimension
reduced_dimension = min(2, dimension)
constructor_options = {
    'maximize': True,
    'dtype': 'float64',
    'device': 'cpu',
    'plot': False,
    'verbose': False,
    'verbose_setting': False,
    'verbose_iter_by_iter': False,
```

```
}

# Isotropic kernels model every input; no active coordinates are given.
action_kernel = bayopy.KernelConfig(
    name='matern32',
    ard_num_dims=None,
    active_dims=None,
    use_outputscale=True,
    lengthscale_prior=None,
    outputscale_prior=None,
    kwargs={},
)
context_kernel = bayopy.KernelConfig(
    name='rbf',
    ard_num_dims=None,
    active_dims=None,
    use_outputscale=True,
    lengthscale_prior=None,
    outputscale_prior=None,
    kwargs={},
)
ei = bayopy.AcquisitionConfig(
    name='EI',
    beta=3.5,
    xi=0.0,
    mc_samples=64,
    seed=seed,
)
ucb = bayopy.AcquisitionConfig(
    name='UCB',
    beta=3.5,
    xi=0.0,
    mc_samples=64,
    seed=seed,
)
contextual_solver = bayopy.AcquisitionOptimizerSpec(
    raw_samples=32,
    restarts=2,
    maxiter=50,
    fit_maxiter=50,
    solver_tolerance=1e-06,
)
distributional_solver = bayopy.ActionOptimizerSpec(
    raw_samples=24,
    restarts=2,
    maxiter=50,
    fit_maxiter=50,
)
robust_solver = bayopy.MultiStartSolver(
    method='multistart',
)

rembo_config = bayopy.REMBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
    Y_lower=None,
    Y_upper=None,
    kernel_type='low_dimensional',
    custom_kernel=None,
    acquisition='EI',
    initial_points=10,
    initial_points_sampling_method='sobol',
    batch_size=1,
    q_batch=None,
    lengthscale_lower=0.01,
    lengthscale_upper=50.0,
    initial_lengthscale=None,
    t_sigma=0.002,
```

```
    refit_interval=20,
    exploitation_count_limit=5,
    upper_bound_multiplier=0.9,
    seed=seed + 0,
)

hesbo_config = bayopy.HeSBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
    acquisition=ei,
    initial_points=10,
    initial_points_sampling_method='sobol',
    hash_family='paper',
    batch_size=1,
    q_batch=None,
    seed=seed + 1,
)

alebo_config = bayopy.ALEBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
    initial_points=10,
    initial_points_sampling_method='sobol',
    acquisition=ei,
    metric_inference='laplace',
    metric_posterior_samples=8,
    feasibility_tolerance=1e-09,
    acquisition_solver='slsqp',
    raw_feasible_samples=64,
    num_restarts=2,
    batch_size=1,
    q_batch=None,
    seed=seed + 2,
)

add_gp_ucb_config = bayopy.AddGPUCBConfig(
    groups=tuple((i,) for i in range(dimension)),
    maximum_group_size=1,
    learn_decomposition=False,
    decomposition_candidates=None,
    refresh_cycle=25,
    beta=3.5,
    initial_points=10,
    initial_points_sampling_method='sobol',
    component_optimizer=None,
    batch_size=1,
    q_batch=None,
    seed=seed + 3,
)

turbo_config = bayopy.TuRBOConfig(
    batch_size=1,
    q_batch=None,
    initial_points=10,
    initial_points_sampling_method='sobol',
    length_init=0.8,
    length_min=0.0078125,
    length_max=1.6,
    success_tolerance=3,
    failure_tolerance=None,
    candidate_count=128,
    perturbation_cap=20.0,
    acquisition=ei,
    success_relative_tolerance=0.001,
    noisy_center='posterior_mean',
    max_cholesky_size=2000,
    gp_training_steps=50,
    seed=seed + 4,
```

```
)

dropout_config = bayopy.DropoutConfig(
    active_dimension=reduced_dimension,
    fill_strategy='mix',
    random_fill_probability=0.1,
    beta=3.5,
    acquisition_optimizer=None,
    initial_points=10,
    initial_points_sampling_method='sobol',
    batch_size=1,
    q_batch=None,
    seed=seed + 5,
)

mcts_vs_config = bayopy.MCTSVSConfig(
    Nv=2,
    Ns=2,
    archive_size=20,
    initial_points=10,
    initial_points_sampling_method='sobol',
    bad_count_limit=5,
    split_threshold=3,
    exploration_constant=0.1,
    inner_optimizer='bo',
    candidate_pool_size=64,
    kernel_lengthscale=0.2,
    acquisition_beta=3.5,
    gp_maxiter=50,
    batch_size=1,
    q_batch=None,
    seed=seed + 6,
)

saasbo_config = bayopy.SAASBOConfig(
    shrinkage_scale=0.1,
    warmup_steps=64,
    posterior_samples=64,
    thinning=4,
    num_chains=1,
    initial_points=10,
    initial_points_sampling_method='sobol',
    acquisition='EI',
    max_tree_depth=4,
    target_accept_prob=0.8,
    num_restarts=2,
    raw_samples=32,
    mc_samples=32,
    ucb_beta=3.5,
    max_r_hat=1.1,
    min_effective_sample_size=10.0,
    allow_divergences=False,
    batch_size=1,
    q_batch=None,
    seed=seed + 7,
)

ag_vs_config = bayopy.AG_VSConfig(
    effective_dimension=reduced_dimension,
    sequence_length=2,
    split_groups=2,
    initial_points=10,
    initial_points_sampling_method='sobol',
    acquisition=ucb,
    stopping_threshold=None,
    stopping_delta=None,
    stagnation_patience=None,
    gp_fit_method='mle',
```

```
    gp_maxiter=50,
    acquisition_restarts=2,
    acquisition_raw_samples=64,
    batch_size=1,
    q_batch=None,
    seed=seed + 8,
)

la_mcts_config = bayopy.LAMCTSConfig(
    initial_points=10,
    initial_points_sampling_method='sobol',
    exploration_constant=1.0,
    split_threshold=12,
    min_child_size=3,
    max_depth=20,
    classifier='svm_rbf',
    local_sampler='turbo',
    candidate_pool_size=64,
    max_resampling_attempts=4,
    kernel_lengthscale=0.2,
    acquisition_beta=3.5,
    gp_maxiter=50,
    batch_size=1,
    q_batch=None,
    seed=seed + 9,
)

pg_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=10,
    initial_points_sampling_method='sobol',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 10,
)

eg_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=10,
    initial_points_sampling_method='sobol',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 11,
)

gs_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
```

```
    initial_points=10,
    initial_points_sampling_method='sobol',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 12,
)

satisficing_ts_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=10,
    initial_points_sampling_method='sobol',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 13,
)

cgp_ucb_config = bayopy.CGPUCBConfig(
    action_bounds=bounds,
    action_set=None,
    action_kernel=action_kernel,
    context_kernel=context_kernel,
    kernel_combination='product',
    beta=3.5,
    delta=0.1,
    finite_domain_cardinality=None,
    acquisition_optimizer=contextual_solver,
    initial_points=10,
    batch_size=1,
    q_batch=None,
    seed=seed + 14,
)

drbo_config = bayopy.DRBOConfig(
    context_kernel=context_kernel,
    action_kernel=action_kernel,
    beta=3.5,
    ambiguity_metric='mmd',
    inner_solver='slsqp',
    action_optimizer=distributional_solver,
    distribution_grid=None,
    enumeration_resolution=20,
    initial_points=10,
    feasibility_tolerance=1e-08,
    optimizer_tolerance=1e-06,
    batch_size=1,
    q_batch=None,
    seed=seed + 15,
)
```

```
robos_config = bayopy.RoBOSConfig(
    aspiration=-0.1,
    context_kernel=context_kernel,
    action_kernel=action_kernel,
    joint_kernel_combination='product',
    joint_kernel=None,
    beta=3.5,
    simplex_solver='slsqp',
    distribution_floor=0.0,
    distribution_grid=None,
    enumeration_resolution=20,
    action_raw_samples=24,
    action_restarts=2,
    action_maxiter=50,
    initial_points=10,
    fit_maxiter=50,
    feasibility_tolerance=1e-08,
    optimizer_tolerance=1e-06,
    batch_size=1,
    q_batch=None,
    seed=seed + 16,
)

stableopt_config = bayopy.StableOPTConfig(
    perturbation_radius=0.08,
    distance=bayopy.L2Distance(),
    beta=3.5,
    inner_solver=robust_solver,
    outer_solver=robust_solver,
    inner_raw_samples=16,
    outer_raw_samples=32,
    inner_restarts=2,
    outer_restarts=2,
    solver_tolerance=1e-06,
    initial_points=10,
    fit_maxiter=50,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 17,
)

advers_config = bayopy.AdveRSConfig(
    variant='RS1',
    aspiration=-0.1,
    p=1.0,
    beta=3.5,
    perturbation_domain=bayopy.FullDomainPerturbations(norm='l2', distance_floor=1e-8),
    candidate_solver=robust_solver,
    inner_solver=robust_solver,
    decision_model='ucb',
    outer_raw_samples=32,
    inner_raw_samples=16,
    radius_bisection_steps=8,
    solver_tolerance=1e-06,
    initial_points=10,
    fit_maxiter=50,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 18,
)

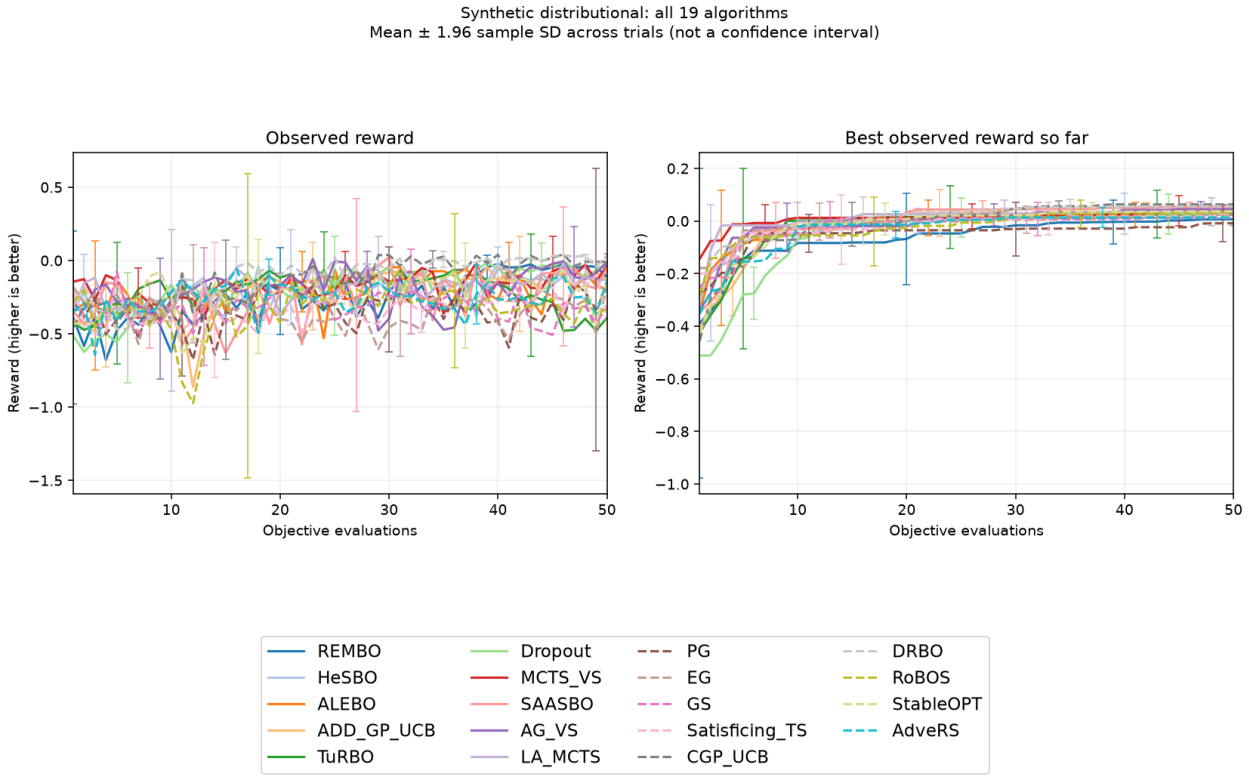
# SAASBO uses actual NUTS. Short chains can report poor ESS/divergences.
# Warnings retain finite draws; they do not certify convergence.
# For inference increase warmup/draws and use at least two chains.
algorithms = {
    'REMBO': bayopy.REMBO(
```

```
        bounds=bounds, config=rembo_config,
        **constructor_options,
    ),
    'HeSBO': bayopy.HeSBO(
        bounds=bounds, config=hesbo_config,
        **constructor_options,
    ),
    'ALEBO': bayopy.ALEBO(
        bounds=bounds, config=alebo_config,
        **constructor_options,
    ),
    'ADD_GP_UCB': bayopy.ADD_GP_UCB(
        bounds=bounds, config=add_gp_ucb_config,
        **constructor_options,
    ),
    'TuRBO': bayopy.TuRBO(
        bounds=bounds, config=turbo_config,
        **constructor_options,
    ),
    'Dropout': bayopy.Dropout(
        bounds=bounds, config=dropout_config,
        **constructor_options,
    ),
    'MCTS_VS': bayopy.MCTS_VS(
        bounds=bounds, config=mcts_vs_config,
        **constructor_options,
    ),
    'SAASBO': bayopy.SAASBO(
        bounds=bounds, config=saasbo_config,
        **constructor_options,
        diagnostic_policy='warn',
        jit_compile=True,
        mcmc_num_threads=1,
    ),
    'AG_VS': bayopy.AG_VS(
        bounds=bounds, config=ag_vs_config,
        **constructor_options,
    ),
    'LA_MCTS': bayopy.LA_MCTS(
        bounds=bounds, config=la_mcts_config,
        **constructor_options,
    ),
    'PG': bayopy.PG(
        bounds=bounds, config=pg_config,
        **constructor_options,
    ),
    'EG': bayopy.EG(
        bounds=bounds, config=eg_config,
        **constructor_options,
    ),
    'GS': bayopy.GS(
        bounds=bounds, config=gs_config,
        **constructor_options,
    ),
    'Satisficing_TS': bayopy.Satisficing_TS(
        bounds=bounds, config=satisficing_ts_config,
        **constructor_options,
    ),
    'CGP_UCB': bayopy.CGP_UCB(
        bounds=bounds, config=cgp_ucb_config,
        **constructor_options,
    ),
    'DRBO': bayopy.DRBO(
        bounds=bounds, config=drbo_config,
        **constructor_options,
    ),
    'RoBOS': bayopy.RoBOS(
        bounds=bounds, config=robos_config,
```

```
        **constructor_options,
    ),
    'StableOPT': bayopy.StableOPT(
        bounds=bounds, config=stableopt_config,
        **constructor_options,
    ),
    'AdveRS': bayopy.AdveRS(
        bounds=bounds, config=advers_config,
        **constructor_options,
    ),
}
print('Configured algorithms:', len(algorithms), list(algorithms))


environment = bayopy.DistributionalEnvironment(
    bounds=bounds, function=distributional_objective,
    context_support=contexts, reference_distribution=[0.5, 0.5],
    sampling_distribution=[0.65, 0.35], margin=0.25,
    dataset=None, previous_data=None, dtype="float64", device="cpu",
    noise_std=0.01, noise_fn=None, add_noise_to_data=False,
    allow_repeated_actions=False, data_row_selection="first", seed=seed,
)

assert set(algorithms) == set(bayopy.algorithm_names())
print("Algorithms submitted:", len(algorithms))
print("Names:", list(algorithms))
experiment = bayopy.Experiment(
    environment=environment, algorithm=algorithms,
    output_directory=run_directory, resume=False, save_every=10,
    stopping_rule=None,
)
result = experiment.run(
    trials=5, max_evaluations=50, plot=False, show=False, progress=True,
)
print(experiment.summary())
print(experiment.diagnostics())
figure = bayopy.plot(
    experiment, kind="comparison", title="Synthetic distributional: all 19 algorithms",
    titles=("Observed reward", "Best observed reward so far"),
    xlabel="Objective evaluations", ylabel="Reward (higher is better)",
    figsize=(12.0, 7.5), xlim=(1, 50), grid=True,
    legend=True, legend_columns=4, legend_fontsize=12.0,
    save_path=bayopy.join_path(run_directory, "all_algorithms.png"),
    dpi=180, show=True,
)
print("Saved results:", run_directory)
```




Captured figure 3 / synthetic distributional

## 1.4 Adversarial: all algorithms

This block defines its own objective, providers, settings, environment, and results directory. Every method is reconstructed with explicit

### Editable Python / block 5

```
import bayopy

seed = bayopy.set_seed(113, threads=1)
run_directory = bayopy.new_run_directory("results/last_guide/synthetic_adversarial")
bounds = bayopy.Bounds(lower=[0.0] * 3, upper=[1.0] * 3)

def adversarial_objective(values):
    # This function receives the evaluated (already perturbed) coordinate.
    target = (0.3, 0.7, 0.5)
    return -sum((x - t) ** 2 for x, t in zip(values, target))


def apply_attack(action, *, round_index, generator):
    direction = 1.0 if round_index % 2 else -1.0
    changed = min(1.0, max(0.0, float(action[0]) + 0.05 * direction))
    return [changed - float(action[0]), 0.0, 0.0]

# Fresh, fully explicit configurations for adversarial observations.
# These bounded computation settings are editable examples.
# No setting overrides are hidden behind a profile.
dimension = bounds.dimension
reduced_dimension = min(2, dimension)
constructor_options = {
    'maximize': True,
    'dtype': 'float64',
    'device': 'cpu',
    'plot': False,
```

```
    'verbose': False,
    'verbose_setting': False,
    'verbose_iter_by_iter': False,
}

# Isotropic kernels model every input; no active coordinates are given.
action_kernel = bayopy.KernelConfig(
    name='matern52',
    ard_num_dims=None,
    active_dims=None,
    use_outputscale=True,
    lengthscale_prior=None,
    outputscale_prior=None,
    kwargs={},
)
context_kernel = bayopy.KernelConfig(
    name='rbf',
    ard_num_dims=None,
    active_dims=None,
    use_outputscale=True,
    lengthscale_prior=None,
    outputscale_prior=None,
    kwargs={},
)
ei = bayopy.AcquisitionConfig(
    name='EI',
    beta=4.0,
    xi=0.0,
    mc_samples=64,
    seed=seed,
)
ucb = bayopy.AcquisitionConfig(
    name='UCB',
    beta=4.0,
    xi=0.0,
    mc_samples=64,
    seed=seed,
)
contextual_solver = bayopy.AcquisitionOptimizerSpec(
    raw_samples=32,
    restarts=2,
    maxiter=50,
    fit_maxiter=50,
    solver_tolerance=1e-06,
)
distributional_solver = bayopy.ActionOptimizerSpec(
    raw_samples=24,
    restarts=2,
    maxiter=50,
    fit_maxiter=50,
)
robust_solver = bayopy.MultiStartSolver(
    method='multistart',
)

rembo_config = bayopy.REMBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
    Y_lower=None,
    Y_upper=None,
    kernel_type='low_dimensional',
    custom_kernel=None,
    acquisition='EI',
    initial_points=11,
    initial_points_sampling_method='uniform',
    batch_size=1,
    q_batch=None,
    lengthscale_lower=0.01,
```

```
    lengthscale_upper=50.0,
    initial_lengthscale=None,
    t_sigma=0.002,
    refit_interval=20,
    exploitation_count_limit=5,
    upper_bound_multiplier=0.9,
    seed=seed + 0,
)

hesbo_config = bayopy.HeSBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
    acquisition=ei,
    initial_points=11,
    initial_points_sampling_method='uniform',
    hash_family='paper',
    batch_size=1,
    q_batch=None,
    seed=seed + 1,
)

alebo_config = bayopy.ALEBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
    initial_points=11,
    initial_points_sampling_method='uniform',
    acquisition=ei,
    metric_inference='laplace',
    metric_posterior_samples=8,
    feasibility_tolerance=1e-09,
    acquisition_solver='slsqp',
    raw_feasible_samples=64,
    num_restarts=2,
    batch_size=1,
    q_batch=None,
    seed=seed + 2,
)

add_gp_ucb_config = bayopy.AddGPUCBConfig(
    groups=tuple((i,) for i in range(dimension)),
    maximum_group_size=1,
    learn_decomposition=False,
    decomposition_candidates=None,
    refresh_cycle=25,
    beta=4.0,
    initial_points=11,
    initial_points_sampling_method='uniform',
    component_optimizer=None,
    batch_size=1,
    q_batch=None,
    seed=seed + 3,
)

turbo_config = bayopy.TuRBOConfig(
    batch_size=1,
    q_batch=None,
    initial_points=11,
    initial_points_sampling_method='uniform',
    length_init=0.8,
    length_min=0.0078125,
    length_max=1.6,
    success_tolerance=3,
    failure_tolerance=None,
    candidate_count=128,
    perturbation_cap=20.0,
    acquisition=ei,
    success_relative_tolerance=0.001,
    noisy_center='posterior_mean',
```

```
    max_cholesky_size=2000,
    gp_training_steps=50,
    seed=seed + 4,
)

dropout_config = bayopy.DropoutConfig(
    active_dimension=reduced_dimension,
    fill_strategy='mix',
    random_fill_probability=0.1,
    beta=4.0,
    acquisition_optimizer=None,
    initial_points=11,
    initial_points_sampling_method='uniform',
    batch_size=1,
    q_batch=None,
    seed=seed + 5,
)

mcts_vs_config = bayopy.MCTSVSConfig(
    Nv=2,
    Ns=2,
    archive_size=20,
    initial_points=11,
    initial_points_sampling_method='uniform',
    bad_count_limit=5,
    split_threshold=3,
    exploration_constant=0.1,
    inner_optimizer='bo',
    candidate_pool_size=64,
    kernel_lengthscale=0.2,
    acquisition_beta=4.0,
    gp_maxiter=50,
    batch_size=1,
    q_batch=None,
    seed=seed + 6,
)

saasbo_config = bayopy.SAASBOConfig(
    shrinkage_scale=0.1,
    warmup_steps=64,
    posterior_samples=64,
    thinning=4,
    num_chains=1,
    initial_points=11,
    initial_points_sampling_method='uniform',
    acquisition='EI',
    max_tree_depth=4,
    target_accept_prob=0.8,
    num_restarts=2,
    raw_samples=32,
    mc_samples=32,
    ucb_beta=4.0,
    max_r_hat=1.1,
    min_effective_sample_size=10.0,
    allow_divergences=False,
    batch_size=1,
    q_batch=None,
    seed=seed + 7,
)

ag_vs_config = bayopy.AG_VSConfig(
    effective_dimension=reduced_dimension,
    sequence_length=2,
    split_groups=2,
    initial_points=11,
    initial_points_sampling_method='uniform',
    acquisition=ucb,
    stopping_threshold=None,
```

```
    stopping_delta=None,
    stagnation_patience=None,
    gp_fit_method='mle',
    gp_maxiter=50,
    acquisition_restarts=2,
    acquisition_raw_samples=64,
    batch_size=1,
    q_batch=None,
    seed=seed + 8,
)

la_mcts_config = bayopy.LAMCTSConfig(
    initial_points=11,
    initial_points_sampling_method='uniform',
    exploration_constant=1.0,
    split_threshold=12,
    min_child_size=3,
    max_depth=20,
    classifier='svm_rbf',
    local_sampler='turbo',
    candidate_pool_size=64,
    max_resampling_attempts=4,
    kernel_lengthscale=0.2,
    acquisition_beta=4.0,
    gp_maxiter=50,
    batch_size=1,
    q_batch=None,
    seed=seed + 9,
)

pg_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=11,
    initial_points_sampling_method='uniform',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 10,
)

eg_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=11,
    initial_points_sampling_method='uniform',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 11,
)
```

```
gs_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=11,
    initial_points_sampling_method='uniform',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 12,
)

satisficing_ts_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=11,
    initial_points_sampling_method='uniform',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 13,
)

cgp_ucb_config = bayopy.CGPUCBConfig(
    action_bounds=bounds,
    action_set=None,
    action_kernel=action_kernel,
    context_kernel=context_kernel,
    kernel_combination='product',
    beta=4.0,
    delta=0.1,
    finite_domain_cardinality=None,
    acquisition_optimizer=contextual_solver,
    initial_points=11,
    batch_size=1,
    q_batch=None,
    seed=seed + 14,
)

drbo_config = bayopy.DRBOConfig(
    context_kernel=context_kernel,
    action_kernel=action_kernel,
    beta=4.0,
    ambiguity_metric='mmd',
    inner_solver='slsqp',
    action_optimizer=distributional_solver,
    distribution_grid=None,
    enumeration_resolution=20,
    initial_points=11,
    feasibility_tolerance=1e-08,
    optimizer_tolerance=1e-06,
    batch_size=1,
    q_batch=None,
```

```
    seed=seed + 15,
)

robos_config = bayopy.RoBOSConfig(
    aspiration=-0.1,
    context_kernel=context_kernel,
    action_kernel=action_kernel,
    joint_kernel_combination='product',
    joint_kernel=None,
    beta=4.0,
    simplex_solver='slsqp',
    distribution_floor=0.0,
    distribution_grid=None,
    enumeration_resolution=20,
    action_raw_samples=24,
    action_restarts=2,
    action_maxiter=50,
    initial_points=11,
    fit_maxiter=50,
    feasibility_tolerance=1e-08,
    optimizer_tolerance=1e-06,
    batch_size=1,
    q_batch=None,
    seed=seed + 16,
)

stableopt_config = bayopy.StableOPTConfig(
    perturbation_radius=0.08,
    distance=bayopy.L2Distance(),
    beta=4.0,
    inner_solver=robust_solver,
    outer_solver=robust_solver,
    inner_raw_samples=16,
    outer_raw_samples=32,
    inner_restarts=2,
    outer_restarts=2,
    solver_tolerance=1e-06,
    initial_points=11,
    fit_maxiter=50,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 17,
)

advers_config = bayopy.AdveRSConfig(
    variant='RS1',
    aspiration=-0.1,
    p=1.0,
    beta=4.0,
    perturbation_domain=bayopy.FullDomainPerturbations(norm='l2', distance_floor=1e-8),
    candidate_solver=robust_solver,
    inner_solver=robust_solver,
    decision_model='ucb',
    outer_raw_samples=32,
    inner_raw_samples=16,
    radius_bisection_steps=8,
    solver_tolerance=1e-06,
    initial_points=11,
    fit_maxiter=50,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 18,
)

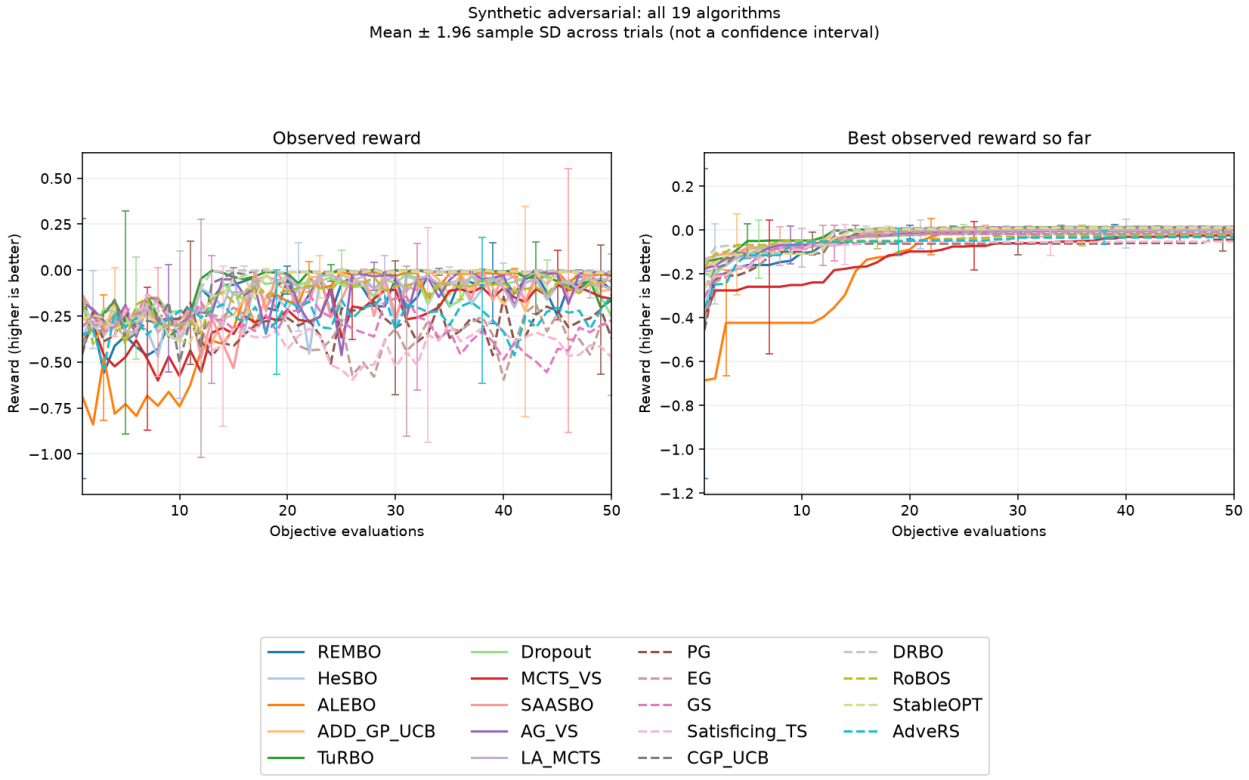
# SAASBO uses actual NUTS. Short chains can report poor ESS/divergences.
# Warnings retain finite draws; they do not certify convergence.
```

```
# For inference increase warmup/draws and use at least two chains.
algorithms = {
    'REMBO': bayopy.REMBO(
        bounds=bounds, config=rembo_config,
        **constructor_options,
    ),
    'HeSBO': bayopy.HeSBO(
        bounds=bounds, config=hesbo_config,
        **constructor_options,
    ),
    'ALEBO': bayopy.ALEBO(
        bounds=bounds, config=alebo_config,
        **constructor_options,
    ),
    'ADD_GP_UCB': bayopy.ADD_GP_UCB(
        bounds=bounds, config=add_gp_ucb_config,
        **constructor_options,
    ),
    'TuRBO': bayopy.TuRBO(
        bounds=bounds, config=turbo_config,
        **constructor_options,
    ),
    'Dropout': bayopy.Dropout(
        bounds=bounds, config=dropout_config,
        **constructor_options,
    ),
    'MCTS_VS': bayopy.MCTS_VS(
        bounds=bounds, config=mcts_vs_config,
        **constructor_options,
    ),
    'SAASBO': bayopy.SAASBO(
        bounds=bounds, config=saasbo_config,
        **constructor_options,
        diagnostic_policy='warn',
        jit_compile=True,
        mcmc_num_threads=1,
    ),
    'AG_VS': bayopy.AG_VS(
        bounds=bounds, config=ag_vs_config,
        **constructor_options,
    ),
    'LA_MCTS': bayopy.LA_MCTS(
        bounds=bounds, config=la_mcts_config,
        **constructor_options,
    ),
    'PG': bayopy.PG(
        bounds=bounds, config=pg_config,
        **constructor_options,
    ),
    'EG': bayopy.EG(
        bounds=bounds, config=eg_config,
        **constructor_options,
    ),
    'GS': bayopy.GS(
        bounds=bounds, config=gs_config,
        **constructor_options,
    ),
    'Satisficing_TS': bayopy.Satisficing_TS(
        bounds=bounds, config=satisficing_ts_config,
        **constructor_options,
    ),
    'CGP_UCB': bayopy.CGP_UCB(
        bounds=bounds, config=cgp_ucb_config,
        **constructor_options,
    ),
    'DRBO': bayopy.DRBO(
        bounds=bounds, config=drbo_config,
        **constructor_options,
```

```
    ),
    'RoBOS': bayopy.RoBOS(
        bounds=bounds, config=robos_config,
        **constructor_options,
    ),
    'StableOPT': bayopy.StableOPT(
        bounds=bounds, config=stableopt_config,
        **constructor_options,
    ),
    'AdveRS': bayopy.AdveRS(
        bounds=bounds, config=advers_config,
        **constructor_options,
    ),
}
print('Configured algorithms:', len(algorithms), list(algorithms))


environment = bayopy.AdversarialEnvironment(
    bounds=bounds, function=adversarial_objective,
    attack=apply_attack,
    dataset=None, previous_data=None, dtype="float64", device="cpu",
    noise_std=0.01, noise_fn=None, add_noise_to_data=False,
    allow_repeated_actions=False, data_row_selection="first", seed=seed,
)

assert set(algorithms) == set(bayopy.algorithm_names())
print("Algorithms submitted:", len(algorithms))
print("Names:", list(algorithms))
experiment = bayopy.Experiment(
    environment=environment, algorithm=algorithms,
    output_directory=run_directory, resume=False, save_every=10,
    stopping_rule=None,
)
result = experiment.run(
    trials=5, max_evaluations=50, plot=False, show=False, progress=True,
)
print(experiment.summary())
print(experiment.diagnostics())
figure = bayopy.plot(
    experiment, kind="comparison", title="Synthetic adversarial: all 19 algorithms",
    titles=("Observed reward", "Best observed reward so far"),
    xlabel="Objective evaluations", ylabel="Reward (higher is better)",
    figsize=(12.0, 7.5), xlim=(1, 50), grid=True,
    legend=True, legend_columns=4, legend_fontsize=12.0,
    save_path=bayopy.join_path(run_directory, "all_algorithms.png"),
    dpi=180, show=True,
)
print("Saved results:", run_directory)
```




Captured figure 4 / synthetic adversarial

# 2. Real Data Formatting

The strict native loader is `bayopy.load_dataset(path, environment="...")`. It reads a finite table of **explicit rewards**, not a model-training dataset whose last column is an arbitrary label. A row is an observation at the named action and, where relevant, context. The library never invents missing rewards or interpolates an unmeasured row. Your objective can instead train a model or run a simulator inside a user-defined function if new evaluations are possible.

Reserved columns use zero-based contiguous numbering. `action_0` through `action_{d-1}` identify learner-controlled coordinates. `reward` is required and must be the final column. Numeric values must be finite. Context vectors describe exogenous information and need not have the same dimension as actions; there is no automatic one-context-column-per-action-column rule.

| Environment | Required column order / roles | Learner sees before acting |
|---|---|---|
| Stochastic | action_0,...,action_(d-1),reward | Available action coordinates |
| Contextual | action_0,...,context_0,...,reward | Current context and actions available there |
| Distributional | action_0,...,context_0,...,reference_ probability, optional sampling_ probability and margin, reward | Support, reference probabilities, ambiguity margin |
| Adversarial | action_0,...,evaluated_action_0,..., reward | Nominal action coordinates; evaluated coordinate arrives in feedback |

Context is not a reward and must not contain the observed target as a shortcut. Reward columns remain hidden from the learner until the environment evaluates its selected action. This is offline finite search, not a causal evaluation of treatments or a policy-value estimator for incomplete logged bandit data. Arbitrary logged actions without counterfactual coverage cannot be treated as a complete distributional reward grid.

The four code blocks below write actual sample rows using only BayOPy, reload them, and print the exact CSV format. Replace the rows with your own measurements. They demonstrate formatting; the four large experiments use the full external files and reconstruct their own environments.

## Stochastic CSV

Only action columns and the final reward are required. The example records three scaled patient descriptors and a negative recorded response. Searching rows selects measured records, not a medical intervention.

**Editable Python / block 6**

```
import bayopy

# Four actual rows from the external example table.
columns = [
    'action_0',
    'action_1',
    'action_2',
    'reward',
]
rows = [
    [
        0.5826446280991735,
        0.5492957746478874,
        0.5622169960335567,
        -151.0,
    ],
    [
        0.14876033057851243,
        0.352112676056338,
        0.22243672996595168,
        -75.0,
```

```
    ],
    [
        0.5165289256198347,
        0.43661971830985913,
        0.49657762645231474,
        -141.0,
    ],
    [
        0.30165289256198347,
        0.30985915492957744,
        0.5729228825160587,
        -206.0,
    ],
]

directory = bayopy.new_run_directory("results/last_guide/format_stochastic")
path = bayopy.save_csv(
    bayopy.join_path(directory, "example.csv"),
    rows, columns=columns,
)
loaded = bayopy.load_dataset(path, environment="stochastic")
print(loaded.preview(rows=4))
print("Action dimension:", loaded.bounds.dimension)
print("Rows:", len(loaded.dataset.observations))
```

## Contextual CSV

Context dimensions are independent of action dimensions. Wine cultivar is encoded with three one-hot context columns so numeric distances do not imply an ordering of categories. All action/reward rows at a revealed cultivar are eligible; other cultivars' rewards remain unavailable.

### Editable Python / block 7

```
import bayopy

# Four actual rows from the external example table.
columns = [
    'action_0',
    'action_1',
    'action_2',
    'context_0',
    'context_1',
    'context_2',
    'reward',
]
rows = [
    [
        0.8421052631578949,
        0.191699604743083,
        0.572192513368984,
        1.0,
        0.0,
        0.0,
        1065.0,
    ],
    [
        0.5710526315789473,
        0.2055335968379447,
        0.4171122994652407,
        1.0,
        0.0,
        0.0,
        1050.0,
    ],
    [
```

```
        0.5605263157894738,
        0.3201581027667984,
        0.7005347593582887,
        1.0,
        0.0,
        0.0,
        1185.0,
    ],
    [
        0.8789473684210525,
        0.2391304347826087,
        0.6096256684491979,
        1.0,
        0.0,
        0.0,
        1480.0,
    ],
]

directory = bayopy.new_run_directory("results/last_guide/format_contextual")
path = bayopy.save_csv(
    bayopy.join_path(directory, "example.csv"),
    rows, columns=columns,
)
loaded = bayopy.load_dataset(path, environment="contextual")
print(loaded.preview(rows=4))
print("Action dimension:", loaded.bounds.dimension)
print("Rows:", len(loaded.dataset.observations))
```

## Distributional CSV

Every distinct action must have a measured row at every context in the support. `reference_probability` must agree across all rows of the same context and sum to one over support, not over CSV rows. `sampling_probability` has the same normalization and is environment-private. When omitted it equals the reference. `margin` is one consistent non-negative scalar; omitted margin defaults to zero. A multirow CSV repeats this metadata. The wind example uses 11 wind-generation levels. Its rewards are calculated from the published revenue function; they are not measured payments. Include all contexts when writing a smaller example so probabilities still sum to one. The real experiment overrides initial CSV probabilities with chronological 48-hour reference and private realization providers.

### Editable Python / block 8

```
import bayopy

# Two actions with all 11 contexts: retain complete probability support.
columns = ['action_0', 'context_0', 'reference_probability', 'sampling_probability', 'margin', 'reward']
rows = [
    [0.0, 0.0, 0.0, 0.0, 0.1, 0.0],
    [0.0, 0.1, 0.0, 0.0, 0.1, 0.010000000000000002],
    [0.0, 0.2, 0.0, 0.0, 0.1, 0.020000000000000004],
    [0.0, 0.3, 0.6875, 0.0, 0.1, 0.03],
    [0.0, 0.4, 0.1875, 0.0, 0.1, 0.04000000000000001],
    [0.0, 0.5, 0.08333333333333333, 1.0, 0.1, 0.05],
    [0.0, 0.6, 0.041666666666666664, 0.0, 0.1, 0.06],
    [0.0, 0.7, 0.0, 0.0, 0.1, 0.06999999999999999],
    [0.0, 0.8, 0.0, 0.0, 0.1, 0.08000000000000002],
    [0.0, 0.9, 0.0, 0.0, 0.1, 0.09000000000000001],
    [0.0, 1.0, 0.0, 0.0, 0.1, 0.1],
    [0.01, 0.0, 0.0, 0.0, 0.1, -0.05],
    [0.01, 0.1, 0.0, 0.0, 0.1, 0.019000000000000003],
    [0.01, 0.2, 0.0, 0.0, 0.1, 0.029000000000000005],
    [0.01, 0.3, 0.6875, 0.0, 0.1, 0.039],
    [0.01, 0.4, 0.1875, 0.0, 0.1, 0.04900000000000001],
    [0.01, 0.5, 0.08333333333333333, 1.0, 0.1, 0.059000000000000004],
    [0.01, 0.6, 0.041666666666666664, 0.0, 0.1, 0.06899999999999999],
```

```
    [0.01, 0.7, 0.0, 0.0, 0.1, 0.07899999999999999],
    [0.01, 0.8, 0.0, 0.0, 0.1, 0.08900000000000001],
    [0.01, 0.9, 0.0, 0.0, 0.1, 0.099],
    [0.01, 1.0, 0.0, 0.0, 0.1, 0.109],
]

directory = bayopy.new_run_directory("results/last_guide/format_distributional_wind")
path = bayopy.save_csv(bayopy.join_path(directory, "example.csv"), rows, columns=columns)
loaded = bayopy.load_dataset(path, environment="distributional")
print(loaded.preview(rows=4))
print("Action dimension:", loaded.bounds.dimension)
print("Rows:", len(loaded.dataset.observations))
```

## Adversarial CSV

Both nominal and evaluated action vectors have dimension d. The final reward belongs to the evaluated coordinate. This finite-table contract requires a deterministic one-to-one permutation over the measured action set, so every attack lookup has a real measured outcome and consumed rows remain consistent. Arbitrary off-grid attacks are rejected in finite mode; use a function-backed environment to evaluate new coordinates. The Iris example constructs swaps between measured records. It is an illustrative adversarial view of observational data, not a historical attack dataset.

### Editable Python / block 9

```
import bayopy

# Four actual rows from the external example table.
columns = [
    'action_0',
    'action_1',
    'action_2',
    'evaluated_action_0',
    'evaluated_action_1',
    'evaluated_action_2',
    'reward',
]
rows = [
    [
        0.0,
        0.423728813559322,
        0.375,
        0.0833333333333334,
        0.5084745762711864,
        0.375,
        6.0,
    ],
    [
        0.0833333333333334,
        0.5084745762711864,
        0.375,
        0.0,
        0.423728813559322,
        0.375,
        5.0,
    ],
    [
        0.0833333333333334,
        0.5932203389830508,
        0.5833333333333334,
        0.0833333333333334,
        0.6779661016949152,
        0.5833333333333334,
        6.0,
    ],
```

```
    [
        0.0833333333333334,
        0.6779661016949152,
        0.5833333333333334,
        0.0833333333333334,
        0.5932203389830508,
        0.5833333333333334,
        6.2,
    ],
]

directory = bayopy.new_run_directory("results/last_guide/format_adversarial")
path = bayopy.save_csv(
    bayopy.join_path(directory, "example.csv"),
    rows, columns=columns,
)
loaded = bayopy.load_dataset(path, environment="adversarial")
print(loaded.preview(rows=4))
print("Action dimension:", loaded.bounds.dimension)
print("Rows:", len(loaded.dataset.observations))
```

## 2.1 Real Life Objective Optimization

Four **different** real datasets provide four large independent comparisons. Each submits all 19 algorithms in five trials, in stochastic, contextual, distributional, then adversarial order: **380 submitted algorithm trials**. Non-native combinations use the documented runtime defaults and emit informative warnings. Genuine failures remain visible in summaries and legends. The source's finite reward table is copied privately for each method. No main experiment enables patience, target, or time stopping; each requests 50 evaluations.

REMBO, HeSBO, and ALEBO use `finite_candidate_mode="project"` by default. This explicit finite-data extension compresses normalized candidate rows: REMBO uses least-squares coordinates, HeSBO uses signed bucket averages (zero for empty buckets), and ALEBO uses its B projection. Acquisition scores select an actual available row and the environment returns its stored reward. The wind table stores explicitly modeled revenue; its physical context comes from measurements. No reward interpolation is performed by the library. Projection can merge distinct rows, so the original embedding guarantees do not apply. The warning `data.projected_embedding` explains this adaptation. Set the configuration field to `"exact"` to restore strict embedding checks.

The original environment classes remain in use. Context still restricts available contextual rows, the distribution still selects the wind-generation context, and the adversarial environment still applies its declared attack. These stochastic policies model pooled scalar feedback without conditioning on those additional inputs; execution does not imply contextual or robust optimality. A distribution alone is not enough: evaluation still requires a reward table or callable objective.

| Environment | Distinct external dataset | Reward and constructed interpretation |
|---|---|---|
| Stochastic | Diabetes | Negative recorded one-year progression; search among measured descriptor rows |
| Contextual | Wine | Recorded proline concentration, conditional on one-hot cultivar |
| Distributional | OPSD German wind generation (2013) | Energy commitment revenue under recorded wind realizations; modeled revenue, not measured market payments |
| Adversarial | Iris | Measured sepal length at a permuted observed coordinate |

These are explicitly constructed finite optimization views of four real observational tables. They were not collected as four native online bandit trials. The results do not establish clinical benefits, causal effects, distributional robustness, or adversarial guarantees for the original studies. We optimize the stated scalar rewards, including the explicit wind revenue model, not classification accuracy. For your own application, select a reward whose maximization is meaningful.

Provenance, source URLs, exact transformations, and SHA-256 hashes are in `data/native_manifest.json`. The original files and prepared CSVs remain under the project-level `data/` folder; none are packaged inside BayOPy. Dataset descriptions are documented by scikit-learn's dataset reference and the wind-source and DRBO references in the distributional subsection.

In the adversarial Iris experiment, StableOPT's selected radius 2 covers the entire [0,1]^3 action box and the constructed swaps. AdveRS instead considers the full feasible domain without receiving the environment's actual attack budget. A small-radius assumption would be false for these data. Best observed reward traces measure sampled outcomes, not the robust objectives optimized by DRBO, RoBOS, StableOPT, or AdveRS.

## Stochastic: diabetes / all 19 algorithms

Run this entire block independently. Its CSV path, full algorithm configurations, and environment are declared again here.

**Editable Python / block 10**

```
import bayopy

seed = bayopy.set_seed(210, threads=1)
data_path = bayopy.find_path("data/stochastic/diabetes_bandit.csv")
loaded = bayopy.load_dataset(data_path, environment="stochastic")
print(loaded.preview(rows=3))
bounds = loaded.bounds
run_directory = bayopy.new_run_directory("results/last_guide/real_stochastic")

# Fresh, fully explicit configurations for stochastic observations.
# These bounded computation settings are editable examples.
# No setting overrides are hidden behind a profile.
dimension = bounds.dimension
reduced_dimension = min(2, dimension)
constructor_options = {
    'maximize': True,
    'dtype': 'float64',
    'device': 'cpu',
    'plot': False,
    'verbose': False,
    'verbose_setting': False,
    'verbose_iter_by_iter': False,
}

# Isotropic kernels model every input; no active coordinates are given.
action_kernel = bayopy.KernelConfig(
    name='matern52',
    ard_num_dims=None,
    active_dims=None,
    use_outputscale=True,
    lengthscale_prior=None,
    outputscale_prior=None,
    kwargs={},
)
context_kernel = bayopy.KernelConfig(
    name='rbf',
    ard_num_dims=None,
    active_dims=None,
    use_outputscale=True,
    lengthscale_prior=None,
    outputscale_prior=None,
    kwargs={},
)
ei = bayopy.AcquisitionConfig(
    name='EI',
    beta=3.0,
    xi=0.0,
    mc_samples=64,
    seed=seed,
```

```
)
ucb = bayopy.AcquisitionConfig(
    name='UCB',
    beta=3.0,
    xi=0.0,
    mc_samples=64,
    seed=seed,
)
contextual_solver = bayopy.AcquisitionOptimizerSpec(
    raw_samples=32,
    restarts=2,
    maxiter=50,
    fit_maxiter=50,
    solver_tolerance=1e-06,
)
distributional_solver = bayopy.ActionOptimizerSpec(
    raw_samples=24,
    restarts=2,
    maxiter=50,
    fit_maxiter=50,
)
robust_solver = bayopy.MultiStartSolver(
    method='multistart',
)

rembo_config = bayopy.REMBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
    Y_lower=None,
    Y_upper=None,
    kernel_type='low_dimensional',
    custom_kernel=None,
    acquisition='EI',
    initial_points=8,
    initial_points_sampling_method='sobol',
    batch_size=1,
    q_batch=None,
    lengthscale_lower=0.01,
    lengthscale_upper=50.0,
    initial_lengthscale=None,
    t_sigma=0.002,
    refit_interval=20,
    exploitation_count_limit=5,
    upper_bound_multiplier=0.9,
    seed=seed + 0,
)

hesbo_config = bayopy.HeSBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
    acquisition=ei,
    initial_points=8,
    initial_points_sampling_method='sobol',
    hash_family='paper',
    batch_size=1,
    q_batch=None,
    seed=seed + 1,
)

alebo_config = bayopy.ALEBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
    initial_points=8,
    initial_points_sampling_method='sobol',
    acquisition=ei,
    metric_inference='laplace',
    metric_posterior_samples=8,
    feasibility_tolerance=1e-09,
```

```
    acquisition_solver='slsqp',
    raw_feasible_samples=64,
    num_restarts=2,
    batch_size=1,
    q_batch=None,
    seed=seed + 2,
)

add_gp_ucb_config = bayopy.AddGPUCBConfig(
    groups=tuple((i,) for i in range(dimension)),
    maximum_group_size=1,
    learn_decomposition=False,
    decomposition_candidates=None,
    refresh_cycle=25,
    beta=3.0,
    initial_points=8,
    initial_points_sampling_method='sobol',
    component_optimizer=None,
    batch_size=1,
    q_batch=None,
    seed=seed + 3,
)

turbo_config = bayopy.TuRBOConfig(
    batch_size=1,
    q_batch=None,
    initial_points=8,
    initial_points_sampling_method='sobol',
    length_init=0.8,
    length_min=0.0078125,
    length_max=1.6,
    success_tolerance=3,
    failure_tolerance=None,
    candidate_count=128,
    perturbation_cap=20.0,
    acquisition=ei,
    success_relative_tolerance=0.001,
    noisy_center='posterior_mean',
    max_cholesky_size=2000,
    gp_training_steps=50,
    seed=seed + 4,
)

dropout_config = bayopy.DropoutConfig(
    active_dimension=reduced_dimension,
    fill_strategy='mix',
    random_fill_probability=0.1,
    beta=3.0,
    acquisition_optimizer=None,
    initial_points=8,
    initial_points_sampling_method='sobol',
    batch_size=1,
    q_batch=None,
    seed=seed + 5,
)

mcts_vs_config = bayopy.MCTSVSConfig(
    Nv=2,
    Ns=2,
    archive_size=20,
    initial_points=8,
    initial_points_sampling_method='sobol',
    bad_count_limit=5,
    split_threshold=3,
    exploration_constant=0.1,
    inner_optimizer='bo',
    candidate_pool_size=64,
    kernel_lengthscale=0.2,
```

```
    acquisition_beta=3.0,
    gp_maxiter=50,
    batch_size=1,
    q_batch=None,
    seed=seed + 6,
)

saasbo_config = bayopy.SAASBOConfig(
    shrinkage_scale=0.1,
    warmup_steps=64,
    posterior_samples=64,
    thinning=4,
    num_chains=1,
    initial_points=8,
    initial_points_sampling_method='sobol',
    acquisition='EI',
    max_tree_depth=4,
    target_accept_prob=0.8,
    num_restarts=2,
    raw_samples=32,
    mc_samples=32,
    ucb_beta=3.0,
    max_r_hat=1.1,
    min_effective_sample_size=10.0,
    allow_divergences=False,
    batch_size=1,
    q_batch=None,
    seed=seed + 7,
)

ag_vs_config = bayopy.AG_VSConfig(
    effective_dimension=reduced_dimension,
    sequence_length=2,
    split_groups=2,
    initial_points=8,
    initial_points_sampling_method='sobol',
    acquisition=ucb,
    stopping_threshold=None,
    stopping_delta=None,
    stagnation_patience=None,
    gp_fit_method='mle',
    gp_maxiter=50,
    acquisition_restarts=2,
    acquisition_raw_samples=64,
    batch_size=1,
    q_batch=None,
    seed=seed + 8,
)

la_mcts_config = bayopy.LAMCTSConfig(
    initial_points=8,
    initial_points_sampling_method='sobol',
    exploration_constant=1.0,
    split_threshold=12,
    min_child_size=3,
    max_depth=20,
    classifier='svm_rbf',
    local_sampler='turbo',
    candidate_pool_size=64,
    max_resampling_attempts=4,
    kernel_lengthscale=0.2,
    acquisition_beta=3.0,
    gp_maxiter=50,
    batch_size=1,
    q_batch=None,
    seed=seed + 9,
)
```

```
pg_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=8,
    initial_points_sampling_method='sobol',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 10,
)

eg_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=8,
    initial_points_sampling_method='sobol',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 11,
)

gs_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=8,
    initial_points_sampling_method='sobol',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 12,
)

satisficing_ts_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=8,
    initial_points_sampling_method='sobol',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
```

```
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 13,
)

cgp_ucb_config = bayopy.CGPUCBConfig(
    action_bounds=bounds,
    action_set=None,
    action_kernel=action_kernel,
    context_kernel=context_kernel,
    kernel_combination='product',
    beta=3.0,
    delta=0.1,
    finite_domain_cardinality=None,
    acquisition_optimizer=contextual_solver,
    initial_points=8,
    batch_size=1,
    q_batch=None,
    seed=seed + 14,
)

drbo_config = bayopy.DRBOConfig(
    context_kernel=context_kernel,
    action_kernel=action_kernel,
    beta=3.0,
    ambiguity_metric='mmd',
    inner_solver='slsqp',
    action_optimizer=distributional_solver,
    distribution_grid=None,
    enumeration_resolution=20,
    initial_points=8,
    feasibility_tolerance=1e-08,
    optimizer_tolerance=1e-06,
    batch_size=1,
    q_batch=None,
    seed=seed + 15,
)

robos_config = bayopy.RoBOSConfig(
    aspiration=-0.1,
    context_kernel=context_kernel,
    action_kernel=action_kernel,
    joint_kernel_combination='product',
    joint_kernel=None,
    beta=3.0,
    simplex_solver='slsqp',
    distribution_floor=0.0,
    distribution_grid=None,
    enumeration_resolution=20,
    action_raw_samples=24,
    action_restarts=2,
    action_maxiter=50,
    initial_points=8,
    fit_maxiter=50,
    feasibility_tolerance=1e-08,
    optimizer_tolerance=1e-06,
    batch_size=1,
    q_batch=None,
    seed=seed + 16,
)

stableopt_config = bayopy.StableOPTConfig(
    perturbation_radius=0.08,
    distance=bayopy.L2Distance(),
    beta=3.0,
```

```
    inner_solver=robust_solver,
    outer_solver=robust_solver,
    inner_raw_samples=16,
    outer_raw_samples=32,
    inner_restarts=2,
    outer_restarts=2,
    solver_tolerance=1e-06,
    initial_points=8,
    fit_maxiter=50,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 17,
)

advers_config = bayopy.AdveRSConfig(
    variant='RS1',
    aspiration=-0.1,
    p=1.0,
    beta=3.0,
    perturbation_domain=bayopy.FullDomainPerturbations(norm='l2', distance_floor=1e-8),
    candidate_solver=robust_solver,
    inner_solver=robust_solver,
    decision_model='ucb',
    outer_raw_samples=32,
    inner_raw_samples=16,
    radius_bisection_steps=8,
    solver_tolerance=1e-06,
    initial_points=8,
    fit_maxiter=50,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 18,
)

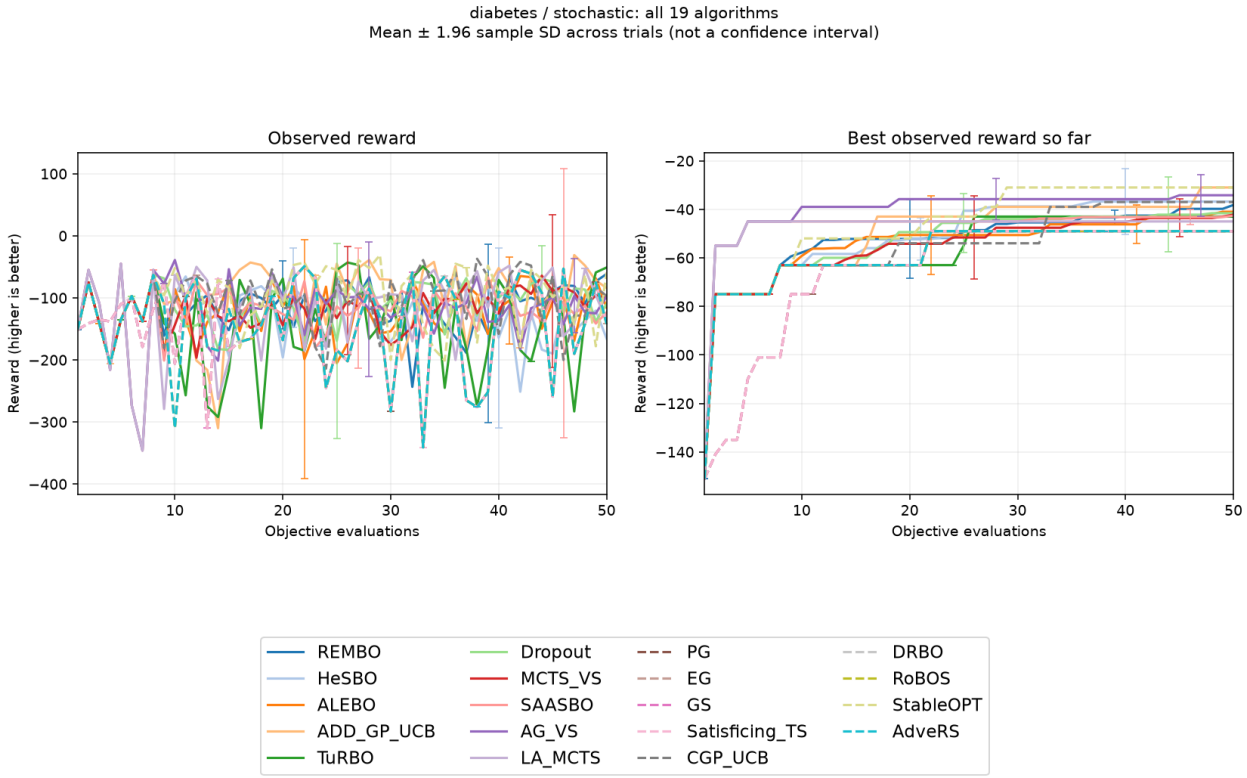
# SAASBO uses actual NUTS. Short chains can report poor ESS/divergences.
# Warnings retain finite draws; they do not certify convergence.
# For inference increase warmup/draws and use at least two chains.
algorithms = {
    'REMBO': bayopy.REMBO(
        bounds=bounds, config=rembo_config,
        **constructor_options,
    ),
    'HeSBO': bayopy.HeSBO(
        bounds=bounds, config=hesbo_config,
        **constructor_options,
    ),
    'ALEBO': bayopy.ALEBO(
        bounds=bounds, config=alebo_config,
        **constructor_options,
    ),
    'ADD_GP_UCB': bayopy.ADD_GP_UCB(
        bounds=bounds, config=add_gp_ucb_config,
        **constructor_options,
    ),
    'TuRBO': bayopy.TuRBO(
        bounds=bounds, config=turbo_config,
        **constructor_options,
    ),
    'Dropout': bayopy.Dropout(
        bounds=bounds, config=dropout_config,
        **constructor_options,
    ),
    'MCTS_VS': bayopy.MCTS_VS(
        bounds=bounds, config=mcts_vs_config,
        **constructor_options,
    ),
```

```
    'SAASBO': bayopy.SAASBO(
        bounds=bounds, config=saasbo_config,
        **constructor_options,
        diagnostic_policy='warn',
        jit_compile=True,
        mcmc_num_threads=1,
    ),
    'AG_VS': bayopy.AG_VS(
        bounds=bounds, config=ag_vs_config,
        **constructor_options,
    ),
    'LA_MCTS': bayopy.LA_MCTS(
        bounds=bounds, config=la_mcts_config,
        **constructor_options,
    ),
    'PG': bayopy.PG(
        bounds=bounds, config=pg_config,
        **constructor_options,
    ),
    'EG': bayopy.EG(
        bounds=bounds, config=eg_config,
        **constructor_options,
    ),
    'GS': bayopy.GS(
        bounds=bounds, config=gs_config,
        **constructor_options,
    ),
    'Satisficing_TS': bayopy.Satisficing_TS(
        bounds=bounds, config=satisficing_ts_config,
        **constructor_options,
    ),
    'CGP_UCB': bayopy.CGP_UCB(
        bounds=bounds, config=cgp_ucb_config,
        **constructor_options,
    ),
    'DRBO': bayopy.DRBO(
        bounds=bounds, config=drbo_config,
        **constructor_options,
    ),
    'RoBOS': bayopy.RoBOS(
        bounds=bounds, config=robos_config,
        **constructor_options,
    ),
    'StableOPT': bayopy.StableOPT(
        bounds=bounds, config=stableopt_config,
        **constructor_options,
    ),
    'AdveRS': bayopy.AdveRS(
        bounds=bounds, config=advers_config,
        **constructor_options,
    ),
}
print('Configured algorithms:', len(algorithms), list(algorithms))


environment = bayopy.StochasticEnvironment(
    bounds=bounds, dataset=loaded.dataset, function=None,
    previous_data=None, dtype="float64", device="cpu",
    noise_std=0.0, noise_fn=None, add_noise_to_data=False,
    allow_repeated_actions=False, data_row_selection="first", seed=seed,
)

assert set(algorithms) == set(bayopy.algorithm_names())
print("Algorithms submitted:", len(algorithms))
print("Names:", list(algorithms))
experiment = bayopy.Experiment(
    environment=environment, algorithm=algorithms,
    output_directory=run_directory, resume=False, save_every=10,
```

```
    stopping_rule=None,
)
result = experiment.run(
    trials=5, max_evaluations=50, plot=False, show=False, progress=True,
)
print(experiment.summary())
print(experiment.diagnostics())
figure = bayopy.plot(
    experiment, kind="comparison", title="diabetes / stochastic: all 19 algorithms",
    titles=("Observed reward", "Best observed reward so far"),
    xlabel="Objective evaluations", ylabel="Reward (higher is better)",
    figsize=(12.0, 7.5), xlim=(1, 50), grid=True,
    legend=True, legend_columns=4, legend_fontsize=12.0,
    save_path=bayopy.join_path(run_directory, "all_algorithms.png"),
    dpi=180, show=True,
)
print("Saved results:", run_directory)
```




Captured figure 5 / real stochastic

## Contextual: wine / all 19 algorithms

Run this entire block independently. Its CSV path, full algorithm configurations, and environment are declared again here.

### Editable Python / block 11

```
import bayopy

seed = bayopy.set_seed(211, threads=1)
data_path = bayopy.find_path("data/contextual/wine_bandit.csv")
loaded = bayopy.load_dataset(data_path, environment="contextual")
print(loaded.preview(rows=3))
bounds = loaded.bounds
run_directory = bayopy.new_run_directory("results/last_guide/real_contextual")

# Fresh, fully explicit configurations for contextual observations.
```

```
# These bounded computation settings are editable examples.
# No setting overrides are hidden behind a profile.
dimension = bounds.dimension
reduced_dimension = min(2, dimension)
constructor_options = {
    'maximize': True,
    'dtype': 'float64',
    'device': 'cpu',
    'plot': False,
    'verbose': False,
    'verbose_setting': False,
    'verbose_iter_by_iter': False,
}

# Isotropic kernels model every input; no active coordinates are given.
action_kernel = bayopy.KernelConfig(
    name='rbf',
    ard_num_dims=None,
    active_dims=None,
    use_outputscale=True,
    lengthscale_prior=None,
    outputscale_prior=None,
    kwargs={},
)
context_kernel = bayopy.KernelConfig(
    name='rbf',
    ard_num_dims=None,
    active_dims=None,
    use_outputscale=True,
    lengthscale_prior=None,
    outputscale_prior=None,
    kwargs={},
)
ei = bayopy.AcquisitionConfig(
    name='EI',
    beta=2.5,
    xi=0.0,
    mc_samples=64,
    seed=seed,
)
ucb = bayopy.AcquisitionConfig(
    name='UCB',
    beta=2.5,
    xi=0.0,
    mc_samples=64,
    seed=seed,
)
contextual_solver = bayopy.AcquisitionOptimizerSpec(
    raw_samples=32,
    restarts=2,
    maxiter=50,
    fit_maxiter=50,
    solver_tolerance=1e-06,
)
distributional_solver = bayopy.ActionOptimizerSpec(
    raw_samples=24,
    restarts=2,
    maxiter=50,
    fit_maxiter=50,
)
robust_solver = bayopy.MultiStartSolver(
    method='multistart',
)

rembo_config = bayopy.REMBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
    Y_lower=None,
```

```
    Y_upper=None,
    kernel_type='low_dimensional',
    custom_kernel=None,
    acquisition='EI',
    initial_points=9,
    initial_points_sampling_method='uniform',
    batch_size=1,
    q_batch=None,
    lengthscale_lower=0.01,
    lengthscale_upper=50.0,
    initial_lengthscale=None,
    t_sigma=0.002,
    refit_interval=20,
    exploitation_count_limit=5,
    upper_bound_multiplier=0.9,
    seed=seed + 0,
)

hesbo_config = bayopy.HeSBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
    acquisition=ei,
    initial_points=9,
    initial_points_sampling_method='uniform',
    hash_family='paper',
    batch_size=1,
    q_batch=None,
    seed=seed + 1,
)

alebo_config = bayopy.ALEBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
    initial_points=9,
    initial_points_sampling_method='uniform',
    acquisition=ei,
    metric_inference='laplace',
    metric_posterior_samples=8,
    feasibility_tolerance=1e-09,
    acquisition_solver='slsqp',
    raw_feasible_samples=64,
    num_restarts=2,
    batch_size=1,
    q_batch=None,
    seed=seed + 2,
)

add_gp_ucb_config = bayopy.AddGPUCBConfig(
    groups=tuple((i,) for i in range(dimension)),
    maximum_group_size=1,
    learn_decomposition=False,
    decomposition_candidates=None,
    refresh_cycle=25,
    beta=2.5,
    initial_points=9,
    initial_points_sampling_method='uniform',
    component_optimizer=None,
    batch_size=1,
    q_batch=None,
    seed=seed + 3,
)

turbo_config = bayopy.TuRBOConfig(
    batch_size=1,
    q_batch=None,
    initial_points=9,
    initial_points_sampling_method='uniform',
    length_init=0.8,
```

```
    length_min=0.0078125,
    length_max=1.6,
    success_tolerance=3,
    failure_tolerance=None,
    candidate_count=128,
    perturbation_cap=20.0,
    acquisition=ei,
    success_relative_tolerance=0.001,
    noisy_center='posterior_mean',
    max_cholesky_size=2000,
    gp_training_steps=50,
    seed=seed + 4,
)

dropout_config = bayopy.DropoutConfig(
    active_dimension=reduced_dimension,
    fill_strategy='mix',
    random_fill_probability=0.1,
    beta=2.5,
    acquisition_optimizer=None,
    initial_points=9,
    initial_points_sampling_method='uniform',
    batch_size=1,
    q_batch=None,
    seed=seed + 5,
)

mcts_vs_config = bayopy.MCTSVSConfig(
    Nv=2,
    Ns=2,
    archive_size=20,
    initial_points=9,
    initial_points_sampling_method='uniform',
    bad_count_limit=5,
    split_threshold=3,
    exploration_constant=0.1,
    inner_optimizer='bo',
    candidate_pool_size=64,
    kernel_lengthscale=0.2,
    acquisition_beta=2.5,
    gp_maxiter=50,
    batch_size=1,
    q_batch=None,
    seed=seed + 6,
)

saasbo_config = bayopy.SAASBOConfig(
    shrinkage_scale=0.1,
    warmup_steps=64,
    posterior_samples=64,
    thinning=4,
    num_chains=1,
    initial_points=9,
    initial_points_sampling_method='uniform',
    acquisition='EI',
    max_tree_depth=4,
    target_accept_prob=0.8,
    num_restarts=2,
    raw_samples=32,
    mc_samples=32,
    ucb_beta=2.5,
    max_r_hat=1.1,
    min_effective_sample_size=10.0,
    allow_divergences=False,
    batch_size=1,
    q_batch=None,
    seed=seed + 7,
)
```

```
ag_vs_config = bayopy.AG_VSConfig(
    effective_dimension=reduced_dimension,
    sequence_length=2,
    split_groups=2,
    initial_points=9,
    initial_points_sampling_method='uniform',
    acquisition=ucb,
    stopping_threshold=None,
    stopping_delta=None,
    stagnation_patience=None,
    gp_fit_method='mle',
    gp_maxiter=50,
    acquisition_restarts=2,
    acquisition_raw_samples=64,
    batch_size=1,
    q_batch=None,
    seed=seed + 8,
)

la_mcts_config = bayopy.LAMCTSConfig(
    initial_points=9,
    initial_points_sampling_method='uniform',
    exploration_constant=1.0,
    split_threshold=12,
    min_child_size=3,
    max_depth=20,
    classifier='svm_rbf',
    local_sampler='turbo',
    candidate_pool_size=64,
    max_resampling_attempts=4,
    kernel_lengthscale=0.2,
    acquisition_beta=2.5,
    gp_maxiter=50,
    batch_size=1,
    q_batch=None,
    seed=seed + 9,
)

pg_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=9,
    initial_points_sampling_method='uniform',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 10,
)

eg_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=9,
    initial_points_sampling_method='uniform',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
```

```
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 11,
)

gs_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=9,
    initial_points_sampling_method='uniform',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 12,
)

satisficing_ts_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=9,
    initial_points_sampling_method='uniform',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 13,
)

cgp_ucb_config = bayopy.CGPUCBConfig(
    action_bounds=bounds,
    action_set=None,
    action_kernel=action_kernel,
    context_kernel=context_kernel,
    kernel_combination='product',
    beta=2.5,
    delta=0.1,
    finite_domain_cardinality=None,
    acquisition_optimizer=contextual_solver,
    initial_points=9,
    batch_size=1,
    q_batch=None,
    seed=seed + 14,
)

drbo_config = bayopy.DRBOConfig(
    context_kernel=context_kernel,
    action_kernel=action_kernel,
    beta=2.5,
    ambiguity_metric='mmd',
```

```
    inner_solver='slsqp',
    action_optimizer=distributional_solver,
    distribution_grid=None,
    enumeration_resolution=20,
    initial_points=9,
    feasibility_tolerance=1e-08,
    optimizer_tolerance=1e-06,
    batch_size=1,
    q_batch=None,
    seed=seed + 15,
)

robos_config = bayopy.RoBOSConfig(
    aspiration=-0.1,
    context_kernel=context_kernel,
    action_kernel=action_kernel,
    joint_kernel_combination='product',
    joint_kernel=None,
    beta=2.5,
    simplex_solver='slsqp',
    distribution_floor=0.0,
    distribution_grid=None,
    enumeration_resolution=20,
    action_raw_samples=24,
    action_restarts=2,
    action_maxiter=50,
    initial_points=9,
    fit_maxiter=50,
    feasibility_tolerance=1e-08,
    optimizer_tolerance=1e-06,
    batch_size=1,
    q_batch=None,
    seed=seed + 16,
)

stableopt_config = bayopy.StableOPTConfig(
    perturbation_radius=0.08,
    distance=bayopy.L2Distance(),
    beta=2.5,
    inner_solver=robust_solver,
    outer_solver=robust_solver,
    inner_raw_samples=16,
    outer_raw_samples=32,
    inner_restarts=2,
    outer_restarts=2,
    solver_tolerance=1e-06,
    initial_points=9,
    fit_maxiter=50,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 17,
)

advers_config = bayopy.AdveRSConfig(
    variant='RS1',
    aspiration=-0.1,
    p=1.0,
    beta=2.5,
    perturbation_domain=bayopy.FullDomainPerturbations(norm='l2', distance_floor=1e-8),
    candidate_solver=robust_solver,
    inner_solver=robust_solver,
    decision_model='ucb',
    outer_raw_samples=32,
    inner_raw_samples=16,
    radius_bisection_steps=8,
    solver_tolerance=1e-06,
    initial_points=9,
```

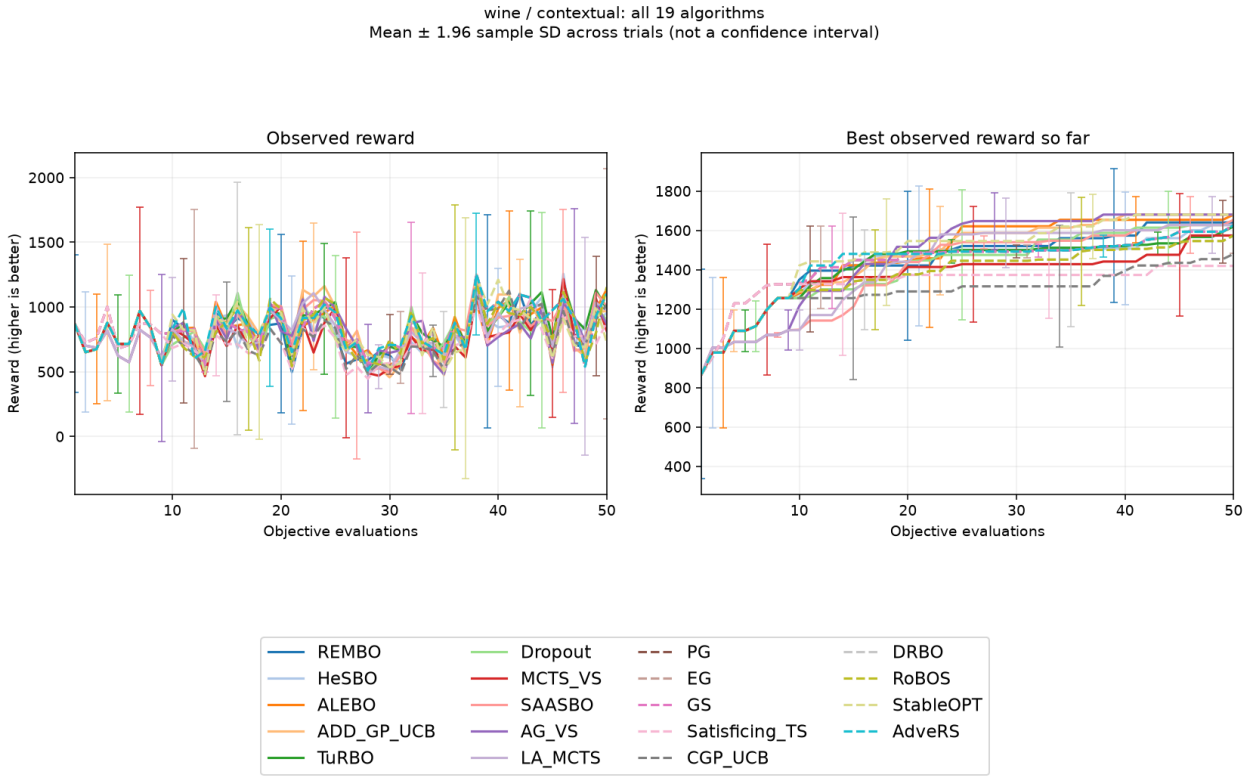
```
    fit_maxiter=50,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 18,
)

# SAASBO uses actual NUTS. Short chains can report poor ESS/divergences.
# Warnings retain finite draws; they do not certify convergence.
# For inference increase warmup/draws and use at least two chains.
algorithms = {
    'REMBO': bayopy.REMBO(
        bounds=bounds, config=rembo_config,
        **constructor_options,
    ),
    'HeSBO': bayopy.HeSBO(
        bounds=bounds, config=hesbo_config,
        **constructor_options,
    ),
    'ALEBO': bayopy.ALEBO(
        bounds=bounds, config=alebo_config,
        **constructor_options,
    ),
    'ADD_GP_UCB': bayopy.ADD_GP_UCB(
        bounds=bounds, config=add_gp_ucb_config,
        **constructor_options,
    ),
    'TuRBO': bayopy.TuRBO(
        bounds=bounds, config=turbo_config,
        **constructor_options,
    ),
    'Dropout': bayopy.Dropout(
        bounds=bounds, config=dropout_config,
        **constructor_options,
    ),
    'MCTS_VS': bayopy.MCTS_VS(
        bounds=bounds, config=mcts_vs_config,
        **constructor_options,
    ),
    'SAASBO': bayopy.SAASBO(
        bounds=bounds, config=saasbo_config,
        **constructor_options,
        diagnostic_policy='warn',
        jit_compile=True,
        mcmc_num_threads=1,
    ),
    'AG_VS': bayopy.AG_VS(
        bounds=bounds, config=ag_vs_config,
        **constructor_options,
    ),
    'LA_MCTS': bayopy.LA_MCTS(
        bounds=bounds, config=la_mcts_config,
        **constructor_options,
    ),
    'PG': bayopy.PG(
        bounds=bounds, config=pg_config,
        **constructor_options,
    ),
    'EG': bayopy.EG(
        bounds=bounds, config=eg_config,
        **constructor_options,
    ),
    'GS': bayopy.GS(
        bounds=bounds, config=gs_config,
        **constructor_options,
    ),
    'Satisficing_TS': bayopy.Satisficing_TS(
        bounds=bounds, config=satisficing_ts_config,
```

```
        **constructor_options,
    ),
    'CGP_UCB': bayopy.CGP_UCB(
        bounds=bounds, config=cgp_ucb_config,
        **constructor_options,
    ),
    'DRBO': bayopy.DRBO(
        bounds=bounds, config=drbo_config,
        **constructor_options,
    ),
    'RoBOS': bayopy.RoBOS(
        bounds=bounds, config=robos_config,
        **constructor_options,
    ),
    'StableOPT': bayopy.StableOPT(
        bounds=bounds, config=stableopt_config,
        **constructor_options,
    ),
    'AdveRS': bayopy.AdveRS(
        bounds=bounds, config=advers_config,
        **constructor_options,
    ),
}
print('Configured algorithms:', len(algorithms), list(algorithms))


environment = bayopy.ContextualEnvironment(
    bounds=bounds, dataset=loaded.dataset, function=None,
    context_provider=None,
    previous_data=None, dtype="float64", device="cpu",
    noise_std=0.0, noise_fn=None, add_noise_to_data=False,
    allow_repeated_actions=False, data_row_selection="first", seed=seed,
)

assert set(algorithms) == set(bayopy.algorithm_names())
print("Algorithms submitted:", len(algorithms))
print("Names:", list(algorithms))
experiment = bayopy.Experiment(
    environment=environment, algorithm=algorithms,
    output_directory=run_directory, resume=False, save_every=10,
    stopping_rule=None,
)
result = experiment.run(
    trials=5, max_evaluations=50, plot=False, show=False, progress=True,
)
print(experiment.summary())
print(experiment.diagnostics())
figure = bayopy.plot(
    experiment, kind="comparison", title="wine / contextual: all 19 algorithms",
    titles=("Observed reward", "Best observed reward so far"),
    xlabel="Objective evaluations", ylabel="Reward (higher is better)",
    figsize=(12.0, 7.5), xlim=(1, 50), grid=True,
    legend=True, legend_columns=4, legend_fontsize=12.0,
    save_path=bayopy.join_path(run_directory, "all_algorithms.png"),
    dpi=180, show=True,
)
print("Saved results:", run_directory)
```




Captured figure 6 / real contextual

## Distributional: wind_power / all 19 algorithms

Run this entire block independently. Its CSV path, full algorithm configurations, and environment are declared again here.

## Distributional wind-power commitment: dataset and replay

This replaces the breast-cancer example with hourly wind-generation data from Open Power System Data, version 2019-06-05, the data package cited in DRBO, Section 4. The retained source contains 98 consecutive German hourly observations from January 2013: generation, capacity, and capacity factor.

The action is promised energy as a fraction of capacity; context is realized wind generation. The paper's revenue is `0.1*max(c-x, 0) + min(x,c) - 5*max(x-c, 0)`. Here, a complete CSV table evaluates that formula on 101 action values and 11 context values. These revenues are calculated from an economic model; they are not recorded electricity payments.

The guide replays 50 consecutive hours after an initial 48-hour history, rounding recorded capacity factors to the nearest 0.1. Each round's public reference uses only its preceding 48 observations. The environment's private sampling distribution puts probability one on that hour's recorded, quantized realization; future context is not passed to the algorithm. The fixed ambiguity margin 0.1 is illustrative, not calibrated to guarantee coverage. Repeated commitments are allowed. All 19 methods receive the same chronological trace, with 5 independent algorithm seeds and at most 50 evaluations each.

This small replay is an adaptation of the paper's application, not a reproduction of its annual simulator experiment or robust-regret figures. The plotted observed reward and best-observed reward do not measure robust regret; a lucky high-wind hour can improve the latter. Because the physical action is one-dimensional, embedding dimension is also one; this example does not demonstrate an advantage from high-dimensional embedding.

RoBOS uses a Matern-3/2 context kernel to avoid the initially configured RBF kernel's near-singular Gram matrix on this support. Completed trials of the other 18 methods were resumed unchanged; RoBOS was rerun.

### Editable Python / block 12

```
import bayopy

seed = bayopy.set_seed(212, threads=1)
data_path = bayopy.find_path("data/distributional/wind_power_bandit.csv")
loaded = bayopy.load_dataset(data_path, environment="distributional")
print(loaded.preview(rows=3))
bounds = loaded.bounds
# This named directory resumes saved trials. Choose another name for a fresh run.
run_directory = bayopy.join_path("results/last_guide/real_distributional_wind", "run_matern_context")

# Fresh, fully explicit configurations for distributional observations.
# These bounded computation settings are editable examples.
# No setting overrides are hidden behind a profile.
dimension = bounds.dimension
reduced_dimension = min(2, dimension)
constructor_options = {
    'maximize': True,
    'dtype': 'float64',
    'device': 'cpu',
    'plot': False,
    'verbose': False,
    'verbose_setting': False,
    'verbose_iter_by_iter': False,
}

# Isotropic kernels model every input; no active coordinates are given.
action_kernel = bayopy.KernelConfig(
    name='matern32',
    ard_num_dims=None,
    active_dims=None,
    use_outputscale=True,
    lengthscale_prior=None,
    outputscale_prior=None,
    kwargs={},
)
context_kernel = bayopy.KernelConfig(
    name='rbf',
    ard_num_dims=None,
    active_dims=None,
    use_outputscale=True,
    lengthscale_prior=None,
    outputscale_prior=None,
    kwargs={},
)
ei = bayopy.AcquisitionConfig(
    name='EI',
    beta=3.5,
    xi=0.0,
    mc_samples=64,
    seed=seed,
)
ucb = bayopy.AcquisitionConfig(
    name='UCB',
    beta=3.5,
    xi=0.0,
    mc_samples=64,
    seed=seed,
)
contextual_solver = bayopy.AcquisitionOptimizerSpec(
    raw_samples=32,
    restarts=2,
    maxiter=50,
    fit_maxiter=50,
    solver_tolerance=1e-06,
)
distributional_solver = bayopy.ActionOptimizerSpec(
    raw_samples=24,
```

```
    restarts=2,
    maxiter=50,
    fit_maxiter=50,
)
robust_solver = bayopy.MultiStartSolver(
    method='multistart',
)

rembo_config = bayopy.REMBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
    Y_lower=None,
    Y_upper=None,
    kernel_type='low_dimensional',
    custom_kernel=None,
    acquisition='EI',
    initial_points=10,
    initial_points_sampling_method='sobol',
    batch_size=1,
    q_batch=None,
    lengthscale_lower=0.01,
    lengthscale_upper=50.0,
    initial_lengthscale=None,
    t_sigma=0.002,
    refit_interval=20,
    exploitation_count_limit=5,
    upper_bound_multiplier=0.9,
    seed=seed + 0,
)

hesbo_config = bayopy.HeSBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
    acquisition=ei,
    initial_points=10,
    initial_points_sampling_method='sobol',
    hash_family='paper',
    batch_size=1,
    q_batch=None,
    seed=seed + 1,
)

alebo_config = bayopy.ALEBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
    initial_points=10,
    initial_points_sampling_method='sobol',
    acquisition=ei,
    metric_inference='laplace',
    metric_posterior_samples=8,
    feasibility_tolerance=1e-09,
    acquisition_solver='slsqp',
    raw_feasible_samples=64,
    num_restarts=2,
    batch_size=1,
    q_batch=None,
    seed=seed + 2,
)

add_gp_ucb_config = bayopy.AddGPUCBConfig(
    groups=tuple((i,) for i in range(dimension)),
    maximum_group_size=1,
    learn_decomposition=False,
    decomposition_candidates=None,
    refresh_cycle=25,
    beta=3.5,
    initial_points=10,
    initial_points_sampling_method='sobol',
```

```
    component_optimizer=None,
    batch_size=1,
    q_batch=None,
    seed=seed + 3,
)

turbo_config = bayopy.TuRBOConfig(
    batch_size=1,
    q_batch=None,
    initial_points=10,
    initial_points_sampling_method='sobol',
    length_init=0.8,
    length_min=0.0078125,
    length_max=1.6,
    success_tolerance=3,
    failure_tolerance=None,
    candidate_count=128,
    perturbation_cap=20.0,
    acquisition=ei,
    success_relative_tolerance=0.001,
    noisy_center='posterior_mean',
    max_cholesky_size=2000,
    gp_training_steps=50,
    seed=seed + 4,
)

dropout_config = bayopy.DropoutConfig(
    active_dimension=reduced_dimension,
    fill_strategy='mix',
    random_fill_probability=0.1,
    beta=3.5,
    acquisition_optimizer=None,
    initial_points=10,
    initial_points_sampling_method='sobol',
    batch_size=1,
    q_batch=None,
    seed=seed + 5,
)

mcts_vs_config = bayopy.MCTSVSConfig(
    Nv=2,
    Ns=2,
    archive_size=20,
    initial_points=10,
    initial_points_sampling_method='sobol',
    bad_count_limit=5,
    split_threshold=3,
    exploration_constant=0.1,
    inner_optimizer='bo',
    candidate_pool_size=64,
    kernel_lengthscale=0.2,
    acquisition_beta=3.5,
    gp_maxiter=50,
    batch_size=1,
    q_batch=None,
    seed=seed + 6,
)

saasbo_config = bayopy.SAASBOConfig(
    shrinkage_scale=0.1,
    warmup_steps=64,
    posterior_samples=64,
    thinning=4,
    num_chains=1,
    initial_points=10,
    initial_points_sampling_method='sobol',
    acquisition='EI',
    max_tree_depth=4,
```

```
    target_accept_prob=0.8,
    num_restarts=2,
    raw_samples=32,
    mc_samples=32,
    ucb_beta=3.5,
    max_r_hat=1.1,
    min_effective_sample_size=10.0,
    allow_divergences=False,
    batch_size=1,
    q_batch=None,
    seed=seed + 7,
)

ag_vs_config = bayopy.AG_VSConfig(
    effective_dimension=reduced_dimension,
    sequence_length=2,
    split_groups=2,
    initial_points=10,
    initial_points_sampling_method='sobol',
    acquisition=ucb,
    stopping_threshold=None,
    stopping_delta=None,
    stagnation_patience=None,
    gp_fit_method='mle',
    gp_maxiter=50,
    acquisition_restarts=2,
    acquisition_raw_samples=64,
    batch_size=1,
    q_batch=None,
    seed=seed + 8,
)

la_mcts_config = bayopy.LAMCTSConfig(
    initial_points=10,
    initial_points_sampling_method='sobol',
    exploration_constant=1.0,
    split_threshold=12,
    min_child_size=3,
    max_depth=20,
    classifier='svm_rbf',
    local_sampler='turbo',
    candidate_pool_size=64,
    max_resampling_attempts=4,
    kernel_lengthscale=0.2,
    acquisition_beta=3.5,
    gp_maxiter=50,
    batch_size=1,
    q_batch=None,
    seed=seed + 9,
)

pg_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=10,
    initial_points_sampling_method='sobol',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 10,
```

```
)

eg_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=10,
    initial_points_sampling_method='sobol',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 11,
)

gs_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=10,
    initial_points_sampling_method='sobol',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 12,
)

satisficing_ts_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=10,
    initial_points_sampling_method='sobol',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 13,
)

cgp_ucb_config = bayopy.CGPUCBConfig(
    action_bounds=bounds,
    action_set=None,
    action_kernel=action_kernel,
    context_kernel=context_kernel,
    kernel_combination='product',
    beta=3.5,
    delta=0.1,
    finite_domain_cardinality=None,
```

```
    acquisition_optimizer=contextual_solver,
    initial_points=10,
    batch_size=1,
    q_batch=None,
    seed=seed + 14,
)

drbo_config = bayopy.DRBOConfig(
    context_kernel=context_kernel,
    action_kernel=action_kernel,
    beta=3.5,
    ambiguity_metric='mmd',
    inner_solver='slsqp',
    action_optimizer=distributional_solver,
    distribution_grid=None,
    enumeration_resolution=20,
    initial_points=10,
    feasibility_tolerance=1e-08,
    optimizer_tolerance=1e-06,
    batch_size=1,
    q_batch=None,
    seed=seed + 15,
)

robos_config = bayopy.RoBOSConfig(
    aspiration=-0.1,
    # Matern is numerically well-conditioned on this 11-point context grid.
    context_kernel=bayopy.KernelConfig(name='matern32'),
    action_kernel=action_kernel,
    joint_kernel_combination='product',
    joint_kernel=None,
    beta=3.5,
    simplex_solver='slsqp',
    distribution_floor=0.0,
    distribution_grid=None,
    enumeration_resolution=20,
    action_raw_samples=24,
    action_restarts=2,
    action_maxiter=50,
    initial_points=10,
    fit_maxiter=50,
    feasibility_tolerance=1e-08,
    optimizer_tolerance=1e-06,
    batch_size=1,
    q_batch=None,
    seed=seed + 16,
)

stableopt_config = bayopy.StableOPTConfig(
    perturbation_radius=0.08,
    distance=bayopy.L2Distance(),
    beta=3.5,
    inner_solver=robust_solver,
    outer_solver=robust_solver,
    inner_raw_samples=16,
    outer_raw_samples=32,
    inner_restarts=2,
    outer_restarts=2,
    solver_tolerance=1e-06,
    initial_points=10,
    fit_maxiter=50,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 17,
)

advers_config = bayopy.AdveRSConfig(
```

```
    variant='RS1',
    aspiration=-0.1,
    p=1.0,
    beta=3.5,
    perturbation_domain=bayopy.FullDomainPerturbations(norm='l2', distance_floor=1e-8),
    candidate_solver=robust_solver,
    inner_solver=robust_solver,
    decision_model='ucb',
    outer_raw_samples=32,
    inner_raw_samples=16,
    radius_bisection_steps=8,
    solver_tolerance=1e-06,
    initial_points=10,
    fit_maxiter=50,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 18,
)

# SAASBO uses actual NUTS. Short chains can report poor ESS/divergences.
# Warnings retain finite draws; they do not certify convergence.
# For inference increase warmup/draws and use at least two chains.
algorithms = {
    'REMBO': bayopy.REMBO(
        bounds=bounds, config=rembo_config,
        **constructor_options,
    ),
    'HeSBO': bayopy.HeSBO(
        bounds=bounds, config=hesbo_config,
        **constructor_options,
    ),
    'ALEBO': bayopy.ALEBO(
        bounds=bounds, config=alebo_config,
        **constructor_options,
    ),
    'ADD_GP_UCB': bayopy.ADD_GP_UCB(
        bounds=bounds, config=add_gp_ucb_config,
        **constructor_options,
    ),
    'TuRBO': bayopy.TuRBO(
        bounds=bounds, config=turbo_config,
        **constructor_options,
    ),
    'Dropout': bayopy.Dropout(
        bounds=bounds, config=dropout_config,
        **constructor_options,
    ),
    'MCTS_VS': bayopy.MCTS_VS(
        bounds=bounds, config=mcts_vs_config,
        **constructor_options,
    ),
    'SAASBO': bayopy.SAASBO(
        bounds=bounds, config=saasbo_config,
        **constructor_options,
        diagnostic_policy='warn',
        jit_compile=True,
        mcmc_num_threads=1,
    ),
    'AG_VS': bayopy.AG_VS(
        bounds=bounds, config=ag_vs_config,
        **constructor_options,
    ),
    'LA_MCTS': bayopy.LA_MCTS(
        bounds=bounds, config=la_mcts_config,
        **constructor_options,
    ),
    'PG': bayopy.PG(
```

```
        bounds=bounds, config=pg_config,
        **constructor_options,
    ),
    'EG': bayopy.EG(
        bounds=bounds, config=eg_config,
        **constructor_options,
    ),
    'GS': bayopy.GS(
        bounds=bounds, config=gs_config,
        **constructor_options,
    ),
    'Satisficing_TS': bayopy.Satisficing_TS(
        bounds=bounds, config=satisficing_ts_config,
        **constructor_options,
    ),
    'CGP_UCB': bayopy.CGP_UCB(
        bounds=bounds, config=cgp_ucb_config,
        **constructor_options,
    ),
    'DRBO': bayopy.DRBO(
        bounds=bounds, config=drbo_config,
        **constructor_options,
    ),
    'RoBOS': bayopy.RoBOS(
        bounds=bounds, config=robos_config,
        **constructor_options,
    ),
    'StableOPT': bayopy.StableOPT(
        bounds=bounds, config=stableopt_config,
        **constructor_options,
    ),
    'AdveRS': bayopy.AdveRS(
        bounds=bounds, config=advers_config,
        **constructor_options,
    ),
}
print('Configured algorithms:', len(algorithms), list(algorithms))


# Real hourly German wind generation, from the data package cited by DRBO.
# This CSV subset has no quoted fields; column 4 is recorded capacity factor.
with open(bayopy.find_path("data/distributional/opsd_wind_2013.csv")) as stream:
    source_lines = stream.read().splitlines()
wind_profiles = [float(line.split(",")[3]) for line in source_lines[1:99]]
# Fixed 11-point support: quantize observed capacity factors to nearest 0.1.
wind_indices = tuple(min(10, max(0, int(value * 10 + 0.5))) for value in wind_profiles)
assert len(wind_indices) == 98  # 48 history hours + 50 evaluation hours

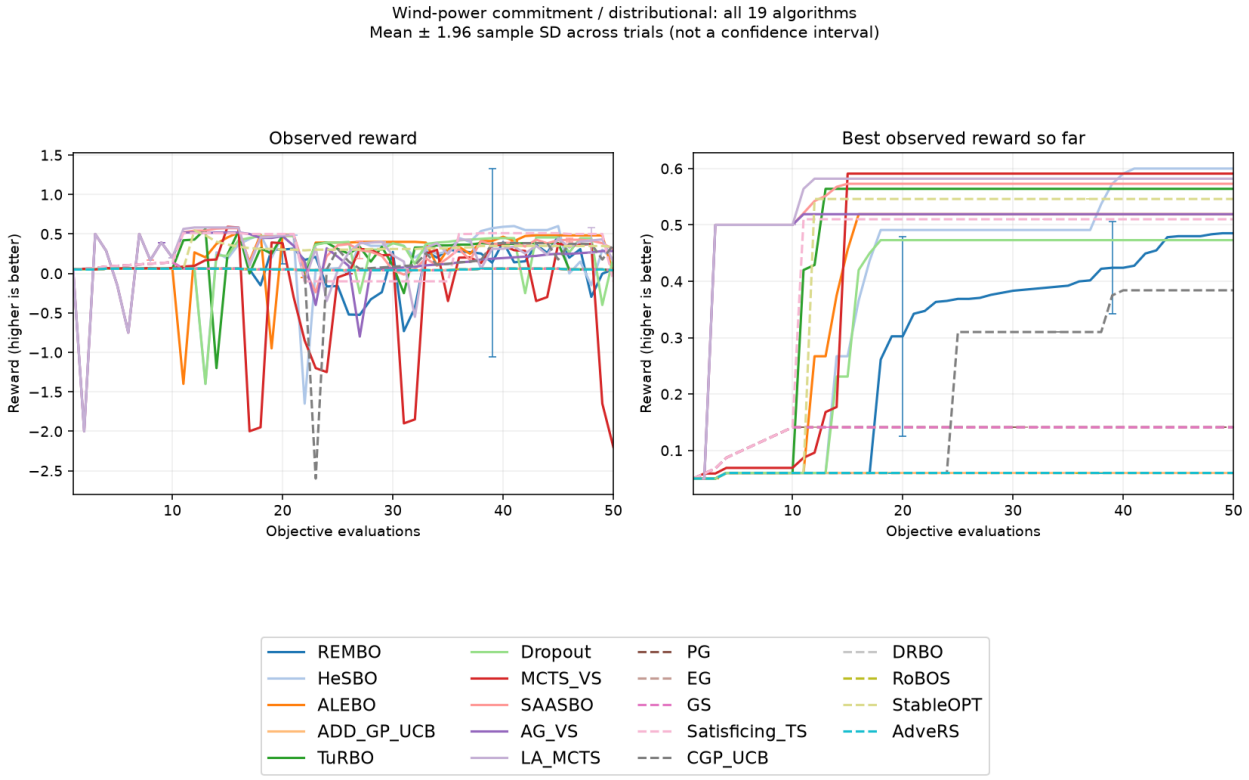

def reference_at_round(round_index):
    # Only past observations contribute to the public 48-hour reference.
    history = wind_indices[round_index:round_index + 48]
    return [history.count(j) / len(history) for j in range(11)]


def realized_wind_at_round(round_index):
    # The environment keeps this next observation private until evaluation.
    if not 0 <= round_index < 50:
        raise ValueError("This replay contains 50 evaluation hours.")
    actual_index = wind_indices[48 + round_index]
    return [float(j == actual_index) for j in range(11)]


# Every CSV reward is calculated from the paper's revenue formula:
# def objective(x, c):
#     return 0.1 * max(c - x, 0) + min(x, c) - 5 * max(x - c, 0)
# x and c are fractions of installed capacity. Revenue is modeled, not measured.
# The initial CSV probabilities are overridden by the providers above.
```

```
environment = bayopy.DistributionalEnvironment(
    bounds=bounds, dataset=loaded.dataset, function=None,
    context_support=loaded.context_support,
    reference_distribution=reference_at_round,
    sampling_distribution=realized_wind_at_round,
    margin=loaded.margin,
    previous_data=None, dtype="float64", device="cpu",
    noise_std=0.0, noise_fn=None, add_noise_to_data=False,
    allow_repeated_actions=True, data_row_selection="first", seed=seed,
)

assert set(algorithms) == set(bayopy.algorithm_names())
print("Algorithms submitted:", len(algorithms))
print("Names:", list(algorithms))
experiment = bayopy.Experiment(
    environment=environment, algorithm=algorithms,
    output_directory=run_directory, resume=True, save_every=10,
    stopping_rule=None,
)
result = experiment.run(
    trials=5, max_evaluations=50, plot=False, show=False, progress=True,
)
print(experiment.summary())
print(experiment.diagnostics())
figure = bayopy.plot(
    experiment, kind="comparison", title="Wind-power commitment / distributional: all 19 algorithms",
    titles=("Observed reward", "Best observed reward so far"),
    xlabel="Objective evaluations", ylabel="Reward (higher is better)",
    figsize=(12.0, 7.5), xlim=(1, 50), grid=True,
    legend=True, legend_columns=4, legend_fontsize=12.0,
    save_path=bayopy.join_path(run_directory, "all_algorithms.png"),
    dpi=180, show=True,
)
print("Saved results:", run_directory)
```




Captured figure 7 / real distributional

## Adversarial: iris / all 19 algorithms

Run this entire block independently. Its CSV path, full algorithm configurations, and environment are declared again here.

**Editable Python / block 13**

```
import bayopy

seed = bayopy.set_seed(213, threads=1)
data_path = bayopy.find_path("data/adversarial/iris_bandit.csv")
loaded = bayopy.load_dataset(data_path, environment="adversarial")
print(loaded.preview(rows=3))
bounds = loaded.bounds
run_directory = bayopy.new_run_directory("results/last_guide/real_adversarial")

# Fresh, fully explicit configurations for adversarial observations.
# These bounded computation settings are editable examples.
# No setting overrides are hidden behind a profile.
dimension = bounds.dimension
reduced_dimension = min(2, dimension)
constructor_options = {
    'maximize': True,
    'dtype': 'float64',
    'device': 'cpu',
    'plot': False,
    'verbose': False,
    'verbose_setting': False,
    'verbose_iter_by_iter': False,
}

# Isotropic kernels model every input; no active coordinates are given.
action_kernel = bayopy.KernelConfig(
    name='matern52',
    ard_num_dims=None,
    active_dims=None,
    use_outputscale=True,
    lengthscale_prior=None,
    outputscale_prior=None,
    kwargs={},
)
context_kernel = bayopy.KernelConfig(
    name='rbf',
    ard_num_dims=None,
    active_dims=None,
    use_outputscale=True,
    lengthscale_prior=None,
    outputscale_prior=None,
    kwargs={},
)
ei = bayopy.AcquisitionConfig(
    name='EI',
    beta=4.0,
    xi=0.0,
    mc_samples=64,
    seed=seed,
)
ucb = bayopy.AcquisitionConfig(
    name='UCB',
    beta=4.0,
    xi=0.0,
    mc_samples=64,
    seed=seed,
)
contextual_solver = bayopy.AcquisitionOptimizerSpec(
    raw_samples=32,
    restarts=2,
    maxiter=50,
```

```
    fit_maxiter=50,
    solver_tolerance=1e-06,
)
distributional_solver = bayopy.ActionOptimizerSpec(
    raw_samples=24,
    restarts=2,
    maxiter=50,
    fit_maxiter=50,
)
robust_solver = bayopy.MultiStartSolver(
    method='multistart',
)

rembo_config = bayopy.REMBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
    Y_lower=None,
    Y_upper=None,
    kernel_type='low_dimensional',
    custom_kernel=None,
    acquisition='EI',
    initial_points=11,
    initial_points_sampling_method='uniform',
    batch_size=1,
    q_batch=None,
    lengthscale_lower=0.01,
    lengthscale_upper=50.0,
    initial_lengthscale=None,
    t_sigma=0.002,
    refit_interval=20,
    exploitation_count_limit=5,
    upper_bound_multiplier=0.9,
    seed=seed + 0,
)

hesbo_config = bayopy.HeSBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
    acquisition=ei,
    initial_points=11,
    initial_points_sampling_method='uniform',
    hash_family='paper',
    batch_size=1,
    q_batch=None,
    seed=seed + 1,
)

alebo_config = bayopy.ALEBOConfig(
    embedding_dimension=reduced_dimension,
    finite_candidate_mode='project',
    initial_points=11,
    initial_points_sampling_method='uniform',
    acquisition=ei,
    metric_inference='laplace',
    metric_posterior_samples=8,
    feasibility_tolerance=1e-09,
    acquisition_solver='slsqp',
    raw_feasible_samples=64,
    num_restarts=2,
    batch_size=1,
    q_batch=None,
    seed=seed + 2,
)

add_gp_ucb_config = bayopy.AddGPUCBConfig(
    groups=tuple((i,) for i in range(dimension)),
    maximum_group_size=1,
    learn_decomposition=False,
```

```
    decomposition_candidates=None,
    refresh_cycle=25,
    beta=4.0,
    initial_points=11,
    initial_points_sampling_method='uniform',
    component_optimizer=None,
    batch_size=1,
    q_batch=None,
    seed=seed + 3,
)

turbo_config = bayopy.TuRBOConfig(
    batch_size=1,
    q_batch=None,
    initial_points=11,
    initial_points_sampling_method='uniform',
    length_init=0.8,
    length_min=0.0078125,
    length_max=1.6,
    success_tolerance=3,
    failure_tolerance=None,
    candidate_count=128,
    perturbation_cap=20.0,
    acquisition=ei,
    success_relative_tolerance=0.001,
    noisy_center='posterior_mean',
    max_cholesky_size=2000,
    gp_training_steps=50,
    seed=seed + 4,
)

dropout_config = bayopy.DropoutConfig(
    active_dimension=reduced_dimension,
    fill_strategy='mix',
    random_fill_probability=0.1,
    beta=4.0,
    acquisition_optimizer=None,
    initial_points=11,
    initial_points_sampling_method='uniform',
    batch_size=1,
    q_batch=None,
    seed=seed + 5,
)

mcts_vs_config = bayopy.MCTSVSConfig(
    Nv=2,
    Ns=2,
    archive_size=20,
    initial_points=11,
    initial_points_sampling_method='uniform',
    bad_count_limit=5,
    split_threshold=3,
    exploration_constant=0.1,
    inner_optimizer='bo',
    candidate_pool_size=64,
    kernel_lengthscale=0.2,
    acquisition_beta=4.0,
    gp_maxiter=50,
    batch_size=1,
    q_batch=None,
    seed=seed + 6,
)

saasbo_config = bayopy.SAASBOConfig(
    shrinkage_scale=0.1,
    warmup_steps=64,
    posterior_samples=64,
    thinning=4,
```

```
    num_chains=1,
    initial_points=11,
    initial_points_sampling_method='uniform',
    acquisition='EI',
    max_tree_depth=4,
    target_accept_prob=0.8,
    num_restarts=2,
    raw_samples=32,
    mc_samples=32,
    ucb_beta=4.0,
    max_r_hat=1.1,
    min_effective_sample_size=10.0,
    allow_divergences=False,
    batch_size=1,
    q_batch=None,
    seed=seed + 7,
)

ag_vs_config = bayopy.AG_VSConfig(
    effective_dimension=reduced_dimension,
    sequence_length=2,
    split_groups=2,
    initial_points=11,
    initial_points_sampling_method='uniform',
    acquisition=ucb,
    stopping_threshold=None,
    stopping_delta=None,
    stagnation_patience=None,
    gp_fit_method='mle',
    gp_maxiter=50,
    acquisition_restarts=2,
    acquisition_raw_samples=64,
    batch_size=1,
    q_batch=None,
    seed=seed + 8,
)

la_mcts_config = bayopy.LAMCTSConfig(
    initial_points=11,
    initial_points_sampling_method='uniform',
    exploration_constant=1.0,
    split_threshold=12,
    min_child_size=3,
    max_depth=20,
    classifier='svm_rbf',
    local_sampler='turbo',
    candidate_pool_size=64,
    max_resampling_attempts=4,
    kernel_lengthscale=0.2,
    acquisition_beta=4.0,
    gp_maxiter=50,
    batch_size=1,
    q_batch=None,
    seed=seed + 9,
)

pg_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=11,
    initial_points_sampling_method='uniform',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
```

```
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 10,
)

eg_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=11,
    initial_points_sampling_method='uniform',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 11,
)

gs_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=11,
    initial_points_sampling_method='uniform',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 12,
)

satisficing_ts_config = bayopy.GoodActionConfig(
    threshold=-0.1,
    candidate_domain=None,
    initial_points=11,
    initial_points_sampling_method='uniform',
    hyperparameter_refit_cycle=3,
    acquisition_restarts=2,
    reference_point=None,
    lookahead_samples=8,
    maximum_value_samples=8,
    candidate_samples=32,
    fit_maxiter=50,
    posterior_std_floor=1e-12,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 13,
)

cgp_ucb_config = bayopy.CGPUCBConfig(
    action_bounds=bounds,
    action_set=None,
    action_kernel=action_kernel,
```

```
    context_kernel=context_kernel,
    kernel_combination='product',
    beta=4.0,
    delta=0.1,
    finite_domain_cardinality=None,
    acquisition_optimizer=contextual_solver,
    initial_points=11,
    batch_size=1,
    q_batch=None,
    seed=seed + 14,
)

drbo_config = bayopy.DRBOConfig(
    context_kernel=context_kernel,
    action_kernel=action_kernel,
    beta=4.0,
    ambiguity_metric='mmd',
    inner_solver='slsqp',
    action_optimizer=distributional_solver,
    distribution_grid=None,
    enumeration_resolution=20,
    initial_points=11,
    feasibility_tolerance=1e-08,
    optimizer_tolerance=1e-06,
    batch_size=1,
    q_batch=None,
    seed=seed + 15,
)

robos_config = bayopy.RoBOSConfig(
    aspiration=-0.1,
    context_kernel=context_kernel,
    action_kernel=action_kernel,
    joint_kernel_combination='product',
    joint_kernel=None,
    beta=4.0,
    simplex_solver='slsqp',
    distribution_floor=0.0,
    distribution_grid=None,
    enumeration_resolution=20,
    action_raw_samples=24,
    action_restarts=2,
    action_maxiter=50,
    initial_points=11,
    fit_maxiter=50,
    feasibility_tolerance=1e-08,
    optimizer_tolerance=1e-06,
    batch_size=1,
    q_batch=None,
    seed=seed + 16,
)

stableopt_config = bayopy.StableOPTConfig(
    perturbation_radius=2.0,
    distance=bayopy.L2Distance(),
    beta=4.0,
    inner_solver=robust_solver,
    outer_solver=robust_solver,
    inner_raw_samples=16,
    outer_raw_samples=32,
    inner_restarts=2,
    outer_restarts=2,
    solver_tolerance=1e-06,
    initial_points=11,
    fit_maxiter=50,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
```

```
    seed=seed + 17,
)

advers_config = bayopy.AdveRSConfig(
    variant='RS1',
    aspiration=-0.1,
    p=1.0,
    beta=4.0,
    perturbation_domain=bayopy.FullDomainPerturbations(norm='l2', distance_floor=1e-8),
    candidate_solver=robust_solver,
    inner_solver=robust_solver,
    decision_model='ucb',
    outer_raw_samples=32,
    inner_raw_samples=16,
    radius_bisection_steps=8,
    solver_tolerance=1e-06,
    initial_points=11,
    fit_maxiter=50,
    kernel=action_kernel,
    batch_size=1,
    q_batch=None,
    seed=seed + 18,
)

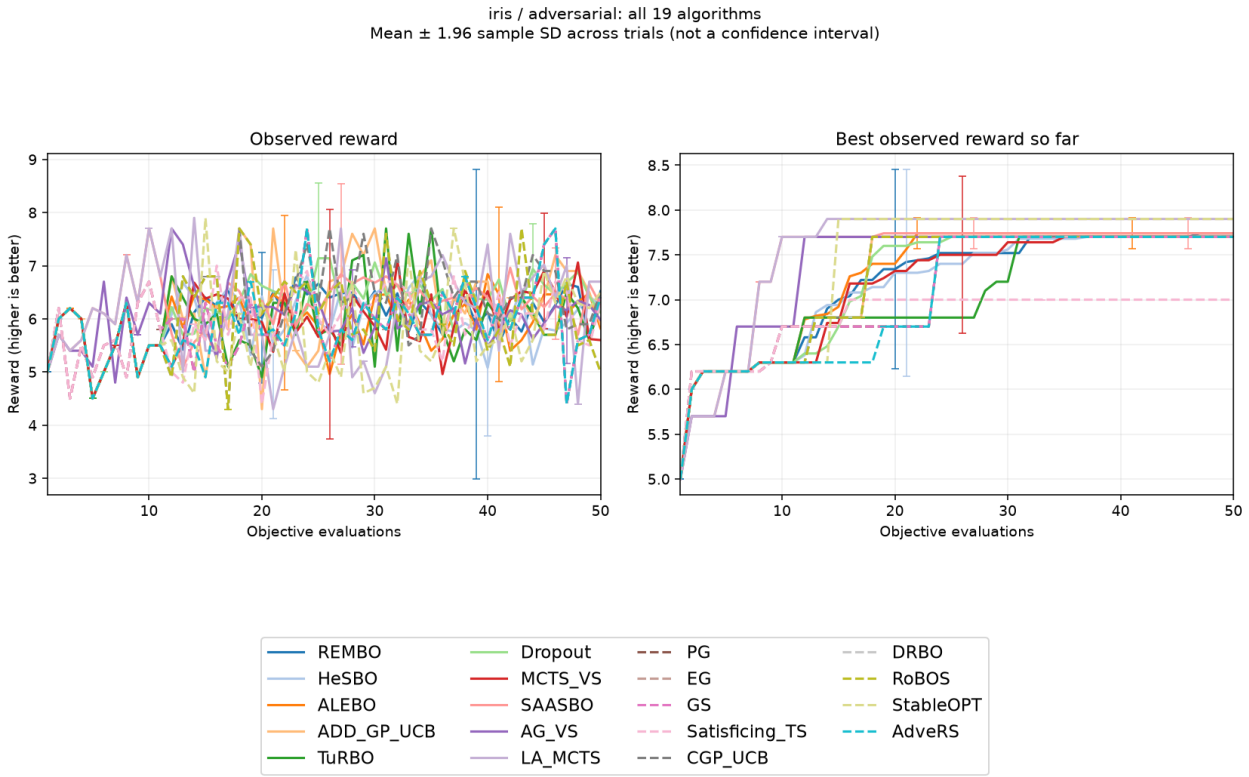
# SAASBO uses actual NUTS. Short chains can report poor ESS/divergences.
# Warnings retain finite draws; they do not certify convergence.
# For inference increase warmup/draws and use at least two chains.
algorithms = {
    'REMBO': bayopy.REMBO(
        bounds=bounds, config=rembo_config,
        **constructor_options,
    ),
    'HeSBO': bayopy.HeSBO(
        bounds=bounds, config=hesbo_config,
        **constructor_options,
    ),
    'ALEBO': bayopy.ALEBO(
        bounds=bounds, config=alebo_config,
        **constructor_options,
    ),
    'ADD_GP_UCB': bayopy.ADD_GP_UCB(
        bounds=bounds, config=add_gp_ucb_config,
        **constructor_options,
    ),
    'TuRBO': bayopy.TuRBO(
        bounds=bounds, config=turbo_config,
        **constructor_options,
    ),
    'Dropout': bayopy.Dropout(
        bounds=bounds, config=dropout_config,
        **constructor_options,
    ),
    'MCTS_VS': bayopy.MCTS_VS(
        bounds=bounds, config=mcts_vs_config,
        **constructor_options,
    ),
    'SAASBO': bayopy.SAASBO(
        bounds=bounds, config=saasbo_config,
        **constructor_options,
        diagnostic_policy='warn',
        jit_compile=True,
        mcmc_num_threads=1,
    ),
    'AG_VS': bayopy.AG_VS(
        bounds=bounds, config=ag_vs_config,
        **constructor_options,
    ),
    'LA_MCTS': bayopy.LA_MCTS(
```

```
        bounds=bounds, config=la_mcts_config,
        **constructor_options,
    ),
    'PG': bayopy.PG(
        bounds=bounds, config=pg_config,
        **constructor_options,
    ),
    'EG': bayopy.EG(
        bounds=bounds, config=eg_config,
        **constructor_options,
    ),
    'GS': bayopy.GS(
        bounds=bounds, config=gs_config,
        **constructor_options,
    ),
    'Satisficing_TS': bayopy.Satisficing_TS(
        bounds=bounds, config=satisficing_ts_config,
        **constructor_options,
    ),
    'CGP_UCB': bayopy.CGP_UCB(
        bounds=bounds, config=cgp_ucb_config,
        **constructor_options,
    ),
    'DRBO': bayopy.DRBO(
        bounds=bounds, config=drbo_config,
        **constructor_options,
    ),
    'RoBOS': bayopy.RoBOS(
        bounds=bounds, config=robos_config,
        **constructor_options,
    ),
    'StableOPT': bayopy.StableOPT(
        bounds=bounds, config=stableopt_config,
        **constructor_options,
    ),
    'AdveRS': bayopy.AdveRS(
        bounds=bounds, config=advers_config,
        **constructor_options,
    ),
}
print('Configured algorithms:', len(algorithms), list(algorithms))


environment = bayopy.AdversarialEnvironment(
    bounds=bounds, dataset=loaded.dataset, function=None,
    attack=loaded.attack,
    previous_data=None, dtype="float64", device="cpu",
    noise_std=0.0, noise_fn=None, add_noise_to_data=False,
    allow_repeated_actions=False, data_row_selection="first", seed=seed,
)

assert set(algorithms) == set(bayopy.algorithm_names())
print("Algorithms submitted:", len(algorithms))
print("Names:", list(algorithms))
experiment = bayopy.Experiment(
    environment=environment, algorithm=algorithms,
    output_directory=run_directory, resume=False, save_every=10,
    stopping_rule=None,
)
result = experiment.run(
    trials=5, max_evaluations=50, plot=False, show=False, progress=True,
)
print(experiment.summary())
print(experiment.diagnostics())
figure = bayopy.plot(
    experiment, kind="comparison", title="iris / adversarial: all 19 algorithms",
    titles=("Observed reward", "Best observed reward so far"),
    xlabel="Objective evaluations", ylabel="Reward (higher is better)",
```

```
    figsize=(12.0, 7.5), xlim=(1, 50), grid=True,
    legend=True, legend_columns=4, legend_fontsize=12.0,
    save_path=bayopy.join_path(run_directory, "all_algorithms.png"),
    dpi=180, show=True,
)
print("Saved results:", run_directory)
```




Captured figure 8 / real adversarial

# 3. Early Stopping

The eight preceding main comparisons used only a 50-evaluation cap. Here we explicitly enable other stopping mechanisms. `make_stopping()` returns fresh rule instances because patience tracks history. A multi-algorithm Experiment clones the supplied rule for each child; progress in one method cannot stop another. Each example below is independent.

| Rule | Meaning |
|---|---|
| MaximumEvaluationsStoppingRule | Stop after the allowed number of evaluated actions, counting initialization |
| MaximumRoundsStoppingRule | Limit ask/tell rounds; a batch can contain several evaluations |
| MaximumTimeStoppingRule | Check elapsed time between complete rounds; does not interrupt an optimizer |
| TargetValueStoppingRule | Stop when the best observed reward reaches a target |
| NoImprovementStoppingRule | Stop after patience rounds without improvement exceeding min_delta, after min_rounds |
| any_of / all_of | Compose conditions; any condition versus every condition |

`run(max_evaluations=50)` remains a separate hard safety cap. If a full batch would exceed the remaining budget, the runner stops before that batch rather than changing its acquisition semantics. For batch size 3, a cap of 50 can therefore end at 48 evaluations. Rule checks occur at completed-round boundaries. No-improvement and target rules concern observed rewards; noise can trigger them without finding the latent optimum. Method-specific posterior stopping is separate: AG_VS accepts `stopping_threshold` and `stopping_delta` together; leaving both as None disables it. Its `stagnation_patience` causes internal reinitialization, not experiment termination.

**Editable Python / block 14**

```
import bayopy

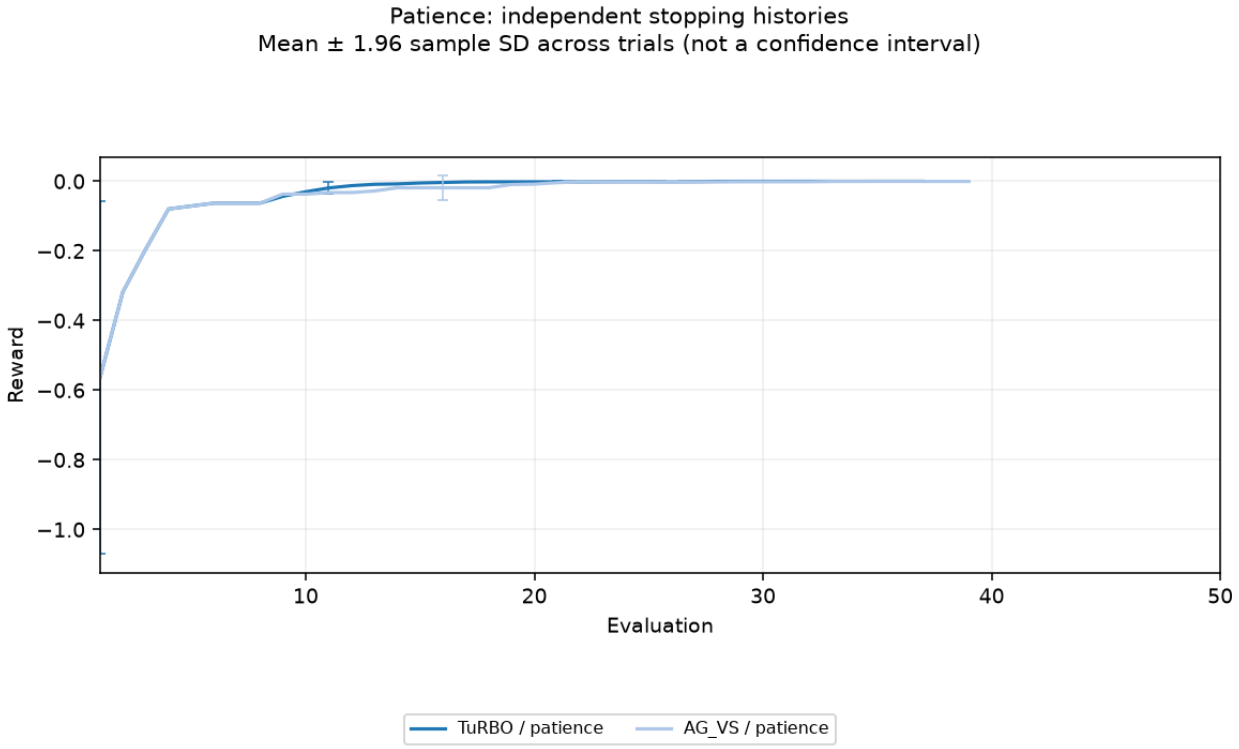
seed = bayopy.set_seed(310, threads=1)
run_directory = bayopy.new_run_directory("results/last_guide/early_patience")
bounds = bayopy.Bounds(lower=[0.0] * 3, upper=[1.0] * 3)

def objective(values):
    return -sum((x - 0.25) ** 2 for x in values)

def make_stopping():
    return bayopy.any_of(
        bayopy.MaximumEvaluationsStoppingRule(50),
        bayopy.NoImprovementStoppingRule(
            10, min_rounds=20, min_delta=0.001,
        ),
    )

algorithms = {
    "TuRBO / patience": bayopy.TuRBO(
        bounds=bounds, config=bayopy.TuRBOConfig(
            initial_points=8, candidate_count=128, seed=seed,
        ), dtype="float64",
    ),
    "AG_VS / patience": bayopy.AG_VS(
        bounds=bounds, config=bayopy.AG_VSConfig(
            effective_dimension=2, initial_points=8, sequence_length=2,
            acquisition_restarts=2, acquisition_raw_samples=32, seed=seed,
            stopping_threshold=None, stopping_delta=None,
        ), dtype="float64",
    ),
}
experiment = bayopy.Experiment(
```

```
    environment=bayopy.StochasticEnvironment(
        bounds=bounds, function=objective, dtype="float64", seed=seed,
    ), algorithm=algorithms, stopping_rule=make_stopping(),
    output_directory=run_directory,
)
result = experiment.run(trials=5, max_evaluations=50, plot=False, progress=True)
print(experiment.summary())
figure = bayopy.plot(
    experiment, kind="incumbent", title="Patience: independent stopping histories",
    xlim=(1, 50), save_path=bayopy.join_path(run_directory, "patience.png"),
)
```




Captured figure 9 / early patience

## Editable Python / block 15

```
import bayopy

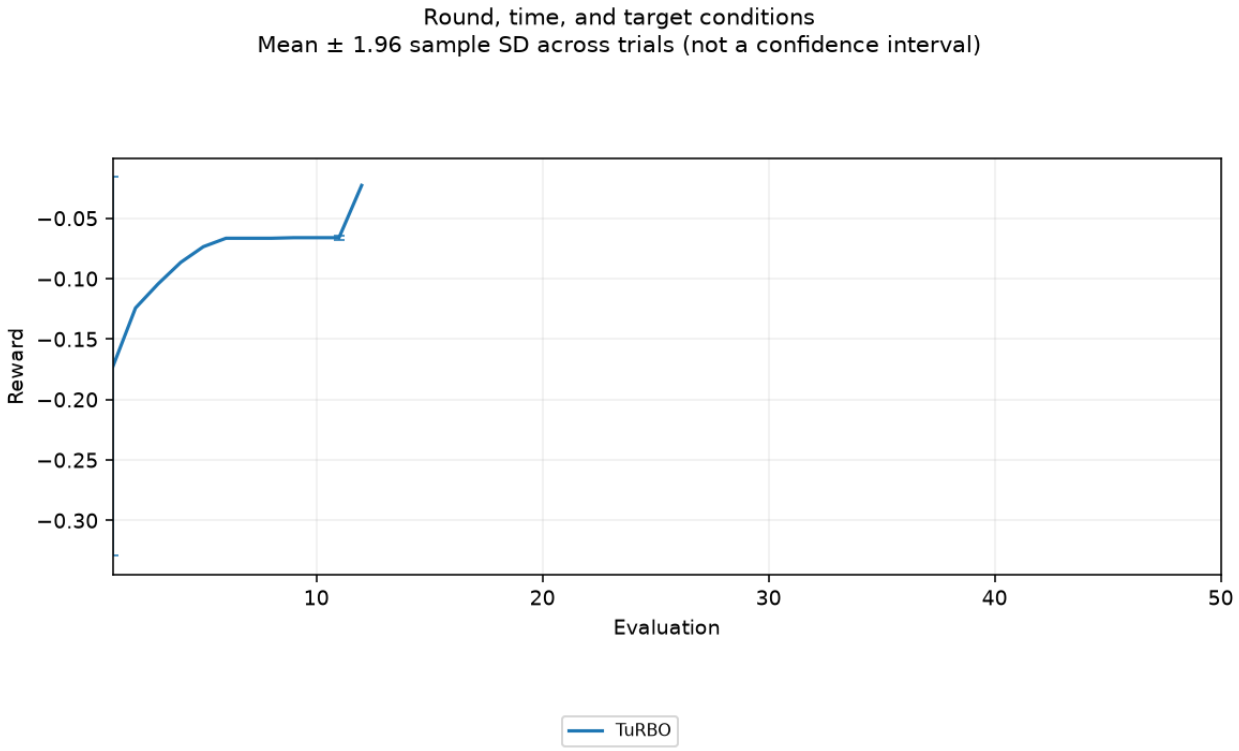
seed = bayopy.set_seed(311, threads=1)
run_directory = bayopy.new_run_directory("results/last_guide/early_composition")
bounds = bayopy.Bounds(lower=[0.0] * 3, upper=[1.0] * 3)

def objective(values):
    return -sum((x - 0.5) ** 2 for x in values)

def make_stopping():
    return bayopy.any_of(
        bayopy.MaximumEvaluationsStoppingRule(50),
        bayopy.MaximumRoundsStoppingRule(35),
        bayopy.MaximumTimeStoppingRule(20.0),
        bayopy.all_of(
            bayopy.MaximumRoundsStoppingRule(12),
            bayopy.TargetValueStoppingRule(-0.05, maximize=True),
        ),
    )

algorithm = bayopy.TuRBO(
    bounds=bounds,
    config=bayopy.TuRBOConfig(
        initial_points=8, candidate_count=128,
```

```
        acquisition=bayopy.AcquisitionConfig(name="EI", xi=0.0, seed=seed),
        seed=seed,
    ), dtype="float64",
)
experiment = bayopy.Experiment(
    environment=bayopy.StochasticEnvironment(
        bounds=bounds, function=objective, dtype="float64", seed=seed,
    ), algorithm=algorithm, stopping_rule=make_stopping(),
    output_directory=run_directory,
)
result = experiment.run(trials=5, max_evaluations=50, plot=False)
print(experiment.summary())
print("Triggered conditions by trial:", [
    trial.stopping_decision for trial in experiment.trial_experiments
])
figure = bayopy.plot(
    experiment, kind="incumbent", title="Round, time, and target conditions",
    xlim=(1, 50), save_path=bayopy.join_path(run_directory, "conditions.png"),
)
```




Captured figure 10 / early composition

## Accuracy and interpretation of the measured results

The notebook records warnings, failed contracts and actual evaluation counts. This edition presents explanations, code and figures; saved terminal outputs are available in the full guide. A common best-observed-reward trace is useful for inspecting execution; it is not the same as every method's mathematical objective. Contextual rewards depend on context. DRBO optimizes a worst-case distributional value; StableOPT uses robust confidence bounds. A high noisy observation alone does not establish robustness or global convergence.

Internal zero-context, point-mass-distribution and zero-perturbation defaults are interoperability extensions. The original papers do not claim that these choices recover native contextual, distributional or adversarial performance in every environment. The environment still owns action evaluation and reward feedback. Warnings identify the assumptions changed for execution; a curve obtained under an adaptation must be interpreted under that adaptation.

The numerical audit found and fixed a DRBO local-search defect: an accepted solver endpoint could be worse than the already-scored start, and flat surfaces could make Powell drift toward a boundary. After successful solver certification, the implementation now re-certifies and retains the start unless refinement improves it beyond tolerance. All-failed solver cases still raise. Focused tests cover worse endpoints, plateaus, and certification.

ALEBO's constrained acquisition optimizer could reject every local endpoint despite having usable feasible candidates. A reproduced failure after 43 evaluations had SLSQP reporting success with constraint residuals of 3.24e-9 and 2.18e-7, above the configured 1e-9 feasibility tolerance. It now screens the feasible candidate pool, refines its best starts with analytic gradients when available, and independently verifies each endpoint. It retains the best finite feasible candidate when refinement cannot improve it; refinement diagnostics remain visible as warnings. Feasible screening and constrained gradient refinement follow the official Ax ALEBO acquisition optimizer and the paper's constrained search domain. The correction preserves the polytope and metric-inference model; it does not relax feasibility or establish global acquisition optimality. Targeted tests cover infeasible and worse local endpoints. The failing checkpoint subsequently continued to 50 evaluations, and a fresh five-trial check completed all five 50-evaluation runs with feasible actions.

This ALEBO correction concerns acquisition search only. The existing option named `metric_inference="laplace"` records fixed-scale Gaussian perturbations of fitted lower-factor parameters; it does not derive a Laplace covariance from the posterior Hessian. Acquisition decisions still use the fitted GP, rather than averaging over those recorded metric samples. The displayed results therefore do not establish equivalence to the paper's integrated metric inference. That separate modeling limitation remains explicit.

Checkpoint checks compare resumed actions and rewards with uninterrupted runs, including independent trial random states and an interruption after an environment measurement was committed but before algorithm feedback. Restoring the GoodAction family's GP warm start and fitting cadence prevents continuation from silently following a different numerical path. Immutable setting mismatches and corrupt checkpoints are rejected before replacing saved aggregate results. These checks concern BayOPy's state; external simulators and side-effecting objectives need their own durable state.

A separate bounded resume audit covered all 19 methods in their native environments with two seeds each. It checkpointed after each method's initial design (8-11 evaluations) and continued through two model-based choices: resumed actions and rewards matched uninterrupted runs exactly in this audit. Its records are retained in `results/resume_catalog/validation.json`. This small continuation check complements the five-trial guide experiments; it is not a claim of exact reproducibility across every device, dependency version or external system.

The frozen implementation passed 694 automated tests, Ruff checks and Mypy checks. The built wheel was checked against all 72 library source files and contains no datasets. This evidence establishes the checked behavior and packaging scope, rather than an exhaustive proof of every possible setting.

The audit checked DRBO's max-min expected UCB, StableOPT's UCB/LCB ordering, and CGP_UCB's joint action-context predictions against their primary sources: DRBO, StableOPT, and CGP_UCB. SAASBO uses genuine NUTS inference here, but the short displayed chains do not establish convergence.

A warning such as "NUTS reported 4 divergent transitions" concerns four numerical integration trajectories in that GP fit, not four failed objective evaluations. A later warning reporting one divergence belongs to a later fit. Finite posterior draws can still be biased. `diagnostic_policy="warn"` retains usable draws and continues with source guarantees invalidated; `diagnostic_policy="raise", allow_divergences=False` rejects the fit instead. The line in `experiment.py` identifies where the warning is surfaced.

The main SAASBO configurations expose `target_accept_prob=0.8`. To investigate divergences, edit it to `0.95` and increase `warmup_steps` (for example 512). Larger `posterior_samples` and multiple `num_chains` can improve assessment of ESS and R-hat, but extra samples alone do not cure divergences. These changes cost time and do not guarantee reliable inference. Do not suppress warnings to claim convergence. See the Stan NUTS diagnostic guidance.

For `AcquisitionConfig`, CGP_UCB and DRBO, UCB is mean + sqrt(beta) * standard deviation; a paper using coefficient b * standard deviation needs beta=b*b for a numeric match. MCTS_VS uses its separate `acquisition_beta` as a direct multiplier.

AG_VS refers to a local manuscript whose complete source was unavailable for this audit. Its tests exercise the implemented equations and invariants; an independent paper-faithfulness certification cannot be claimed. These finite runs and targeted checks do not prove every configuration of all 19 algorithms. Numerical

negative-variance warnings can arise from cancellation or ill-conditioned covariance matrices. Finite precision, variance floors, jitter and explicit fit-failure diagnostics remain visible; a warning is not hidden by the guide.

# 4. Different Settings Examples

These shorter examples show selected changes without repeating all optional fields. Omitted values use the public configuration class defaults. Each block still defines its own input, algorithms, environment, and output directory. Each run goes through `Experiment.run` and stays within 50 evaluations.

## Defaults and a list of algorithms

This example passes a Python list of algorithm instances. `TuRBOConfig` uses its defaults except for the displayed seed: initialization and candidate count are resolved from dimension, and its default acquisition is Thompson sampling. The printed config shows None for dimension-resolved defaults. Required AG_VS effective dimension remains an explicit scientific choice. Passing just `algorithms[0]` instead of the list uses the same runner for one algorithm. There is no second comparison interface and no profile variable.

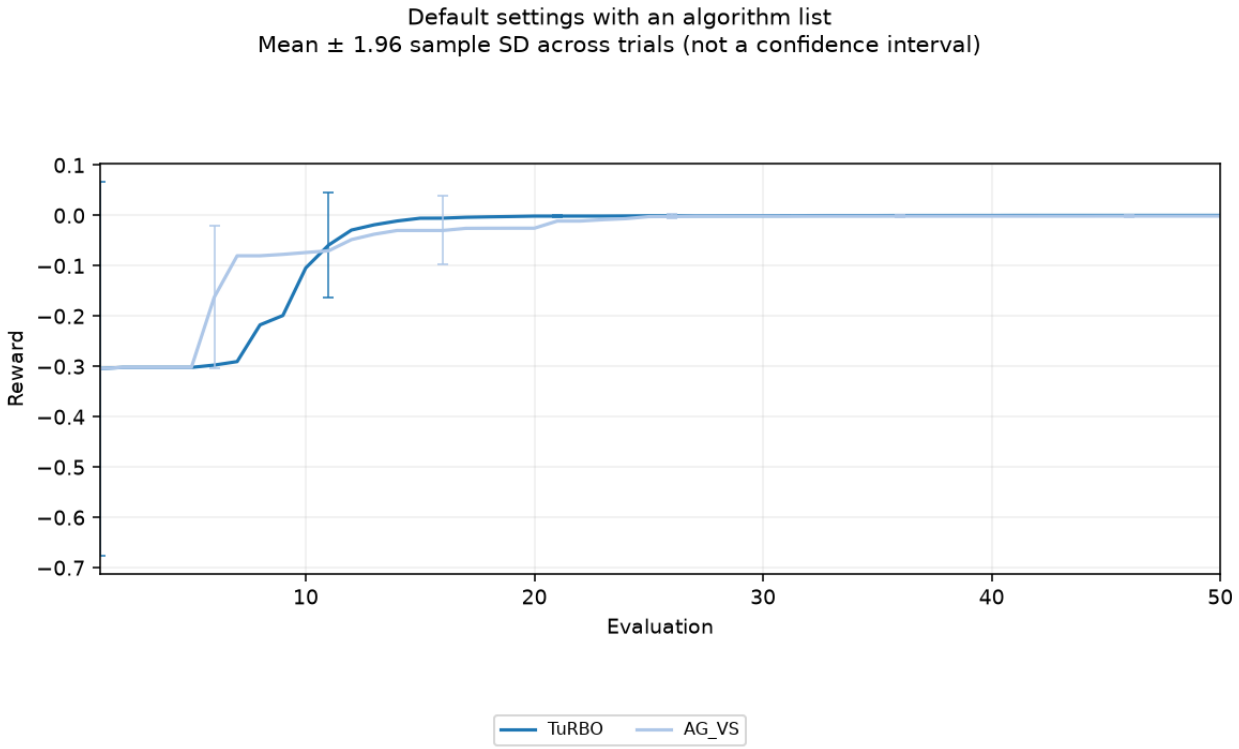
**Editable Python / block 16**

```
import bayopy

seed = bayopy.set_seed(410, threads=1)
run_directory = bayopy.new_run_directory("results/last_guide/defaults")
bounds = bayopy.Bounds(lower=[0.0] * 3, upper=[1.0] * 3)

def objective(values):
    return -sum((x - 0.3) ** 2 for x in values)

algorithms = [
    bayopy.TuRBO(
        bounds=bounds, config=bayopy.TuRBOConfig(seed=seed), dtype="float64",
    ),
    bayopy.AG_VS(
        bounds=bounds,
        config=bayopy.AG_VSConfig(effective_dimension=2, seed=seed),
        dtype="float64",
    ),
]
print("TuRBO defaults:", algorithms[0].config)
experiment = bayopy.Experiment(
    environment=bayopy.StochasticEnvironment(
        bounds=bounds, function=objective, dtype="float64", seed=seed,
    ), algorithm=algorithms, output_directory=run_directory,
)
result = experiment.run(trials=5, max_evaluations=50, plot=False, progress=True)
print(experiment.summary())
figure = bayopy.plot(
    experiment, kind="incumbent", title="Default settings with an algorithm list",
    xlim=(1, 50), save_path=bayopy.join_path(run_directory, "defaults.png"),
)
```




Captured figure 11 / settings defaults

## Acquisition settings: UCB beta and EI offset

The three named configurations below all use TuRBO, with explicit EI/UCB extensions. UCB beta=9 has uncertainty multiplier 3; beta=3 has multiplier sqrt(3). `xi=0.01` changes EI's improvement threshold. These are comparisons of acquisition settings within the displayed trust-region implementation, not three separate algorithm families.

### Editable Python / block 17

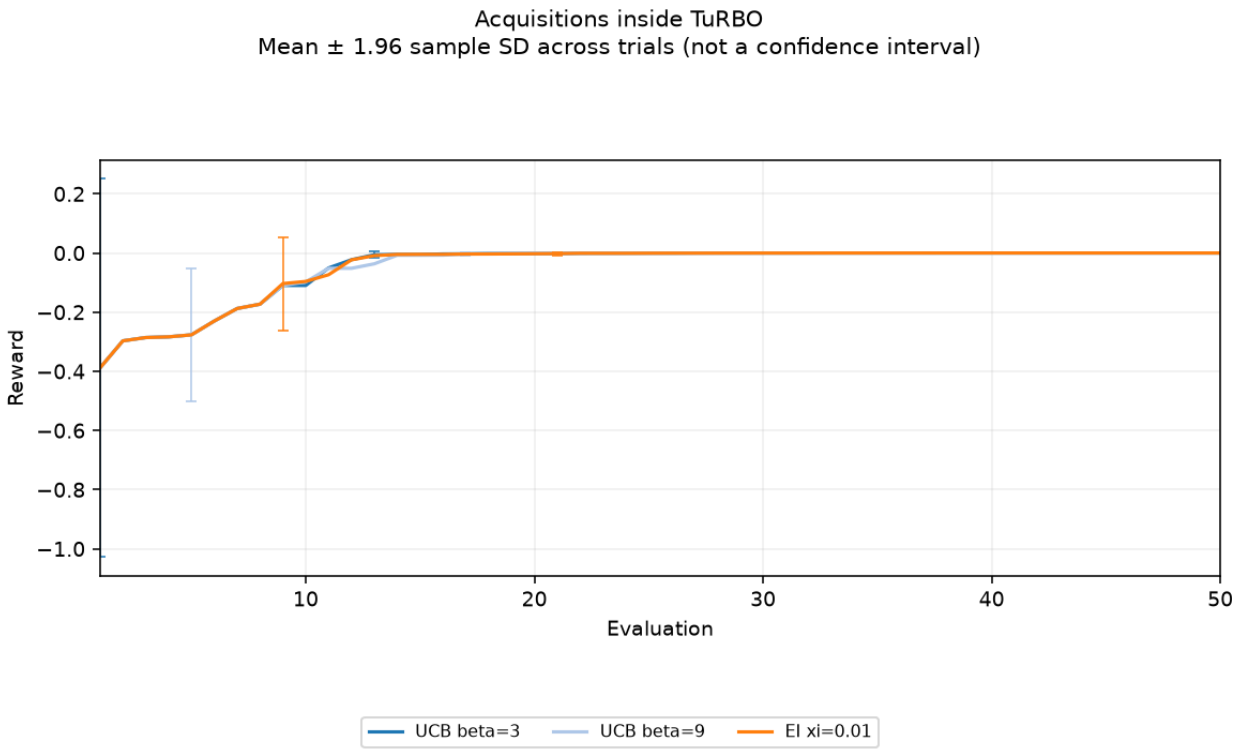
```
import bayopy

seed = bayopy.set_seed(411, threads=1)
run_directory = bayopy.new_run_directory("results/last_guide/acquisitions")
bounds = bayopy.Bounds(lower=[0.0] * 3, upper=[1.0] * 3)

def objective(values):
    return -sum((x - 0.4) ** 2 for x in values)

algorithms = {
    "UCB beta=3": bayopy.TuRBO(
        bounds=bounds, config=bayopy.TuRBOConfig(
            acquisition=bayopy.AcquisitionConfig(name="UCB", beta=3.0, seed=seed),
            initial_points=8, candidate_count=128, seed=seed,
        ), dtype="float64",
    ),
    "UCB beta=9": bayopy.TuRBO(
        bounds=bounds, config=bayopy.TuRBOConfig(
            acquisition=bayopy.AcquisitionConfig(name="UCB", beta=9.0, seed=seed),
            initial_points=8, candidate_count=128, seed=seed,
        ), dtype="float64",
    ),
    "EI xi=0.01": bayopy.TuRBO(
        bounds=bounds, config=bayopy.TuRBOConfig(
            acquisition=bayopy.AcquisitionConfig(name="EI", xi=0.01, seed=seed),
            initial_points=8, candidate_count=128, seed=seed,
```

```
        ), dtype="float64",
    ),
}
experiment = bayopy.Experiment(
    environment=bayopy.StochasticEnvironment(
        bounds=bounds, function=objective, dtype="float64", seed=seed,
    ), algorithm=algorithms, output_directory=run_directory,
)
result = experiment.run(trials=5, max_evaluations=50, plot=False, progress=True)
print(experiment.summary())
figure = bayopy.plot(
    experiment, kind="incumbent", title="Acquisitions inside TuRBO",
    xlim=(1, 50), save_path=bayopy.join_path(run_directory, "acquisitions.png"),
)
```




Captured figure 12 / settings acquisitions

## GP kernels inside a contextual real-data experiment

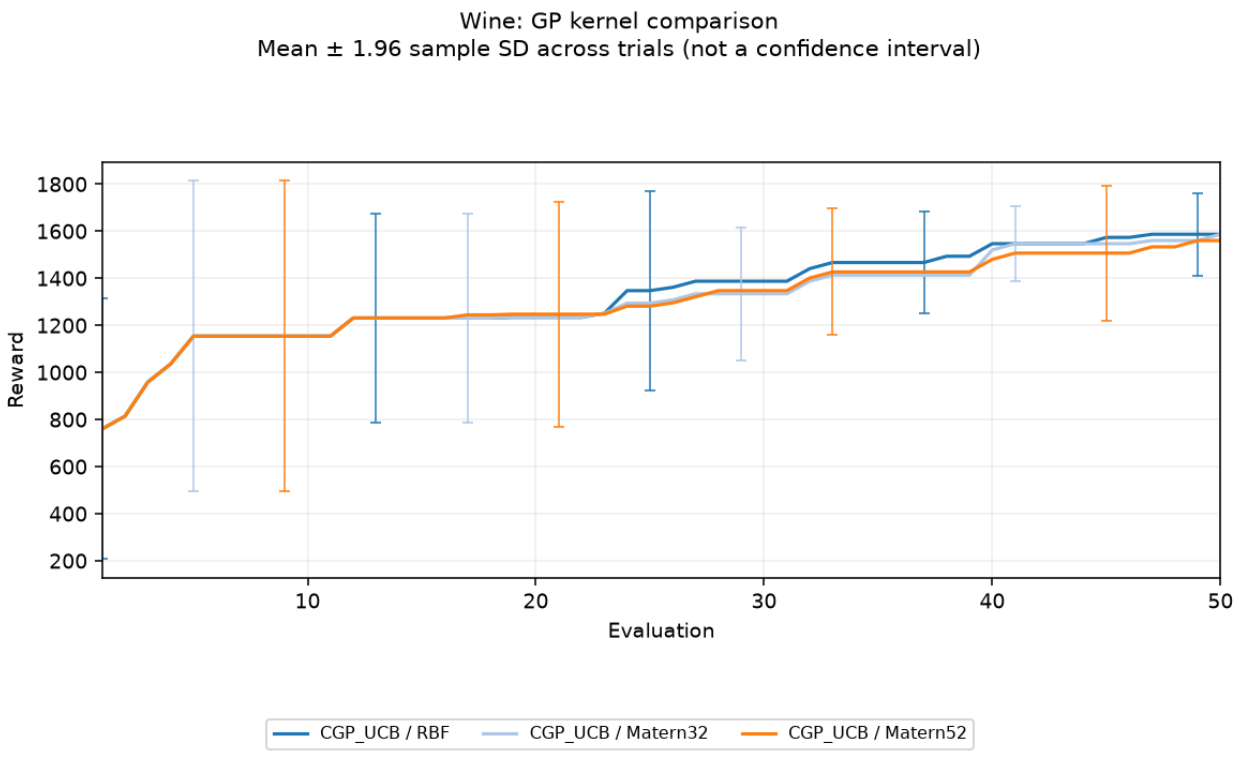
Here the kernel changes the surrogate used to choose measured Wine rows. We compare three CGP_UCB configurations on the same dataset and seed, with the same fixed beta and optimizer budgets. The context kernel operates on one-hot cultivar, while the action kernel operates on the three scaled descriptors. These are optimization runs, not drawings of kernel functions.

### Editable Python / block 18

```
import bayopy

seed = bayopy.set_seed(412, threads=1)
run_directory = bayopy.new_run_directory("results/last_guide/real_kernels")
loaded = bayopy.load_dataset(
    bayopy.find_path("data/contextual/wine_bandit.csv"), environment="contextual",
)
bounds = loaded.bounds
optimizer = bayopy.AcquisitionOptimizerSpec(
    raw_samples=32, restarts=2, maxiter=50, fit_maxiter=50,
```

```
)
algorithms = {
    "CGP_UCB / RBF": bayopy.CGP_UCB(
        bounds=bounds, config=bayopy.CGPUCBConfig(
            action_bounds=bounds, action_kernel=bayopy.KernelConfig(name="rbf"),
            context_kernel=bayopy.KernelConfig(name="rbf"), beta=3.0,
            initial_points=8, acquisition_optimizer=optimizer, seed=seed,
        ), dtype="float64",
    ),
    "CGP_UCB / Matern32": bayopy.CGP_UCB(
        bounds=bounds, config=bayopy.CGPUCBConfig(
            action_bounds=bounds, action_kernel=bayopy.KernelConfig(name="matern32"),
            context_kernel=bayopy.KernelConfig(name="rbf"), beta=3.0,
            initial_points=8, acquisition_optimizer=optimizer, seed=seed,
        ), dtype="float64",
    ),
    "CGP_UCB / Matern52": bayopy.CGP_UCB(
        bounds=bounds, config=bayopy.CGPUCBConfig(
            action_bounds=bounds, action_kernel=bayopy.KernelConfig(name="matern52"),
            context_kernel=bayopy.KernelConfig(name="rbf"), beta=3.0,
            initial_points=8, acquisition_optimizer=optimizer, seed=seed,
        ), dtype="float64",
    ),
}
experiment = bayopy.Experiment(
    environment=bayopy.ContextualEnvironment(
        bounds=bounds, dataset=loaded.dataset, dtype="float64", seed=seed,
    ), algorithm=algorithms, output_directory=run_directory,
)
result = experiment.run(trials=5, max_evaluations=50, plot=False, progress=True)
print(experiment.summary())
print(experiment.diagnostics())
figure = bayopy.plot(
    experiment, kind="incumbent", title="Wine: GP kernel comparison",
    xlim=(1, 50), save_path=bayopy.join_path(run_directory, "kernels.png"),
)
```




Captured figure 13 / settings real kernels

# Sampling and batches

Sampling overrides are attached to the experiment by algorithm label. The two TuRBO instances below compare Sobol and uniform sampling. Both request batches of 2, so 25 completed rounds consume 50 evaluations. `batch_size=2` uses the method's independent batch behavior. For a supported joint Monte Carlo acquisition, use `batch_size=None, q_batch=2` instead; the third configuration demonstrates joint EI. The budget applies to actions, not rounds. Change a sampler only if that method supports the phase.

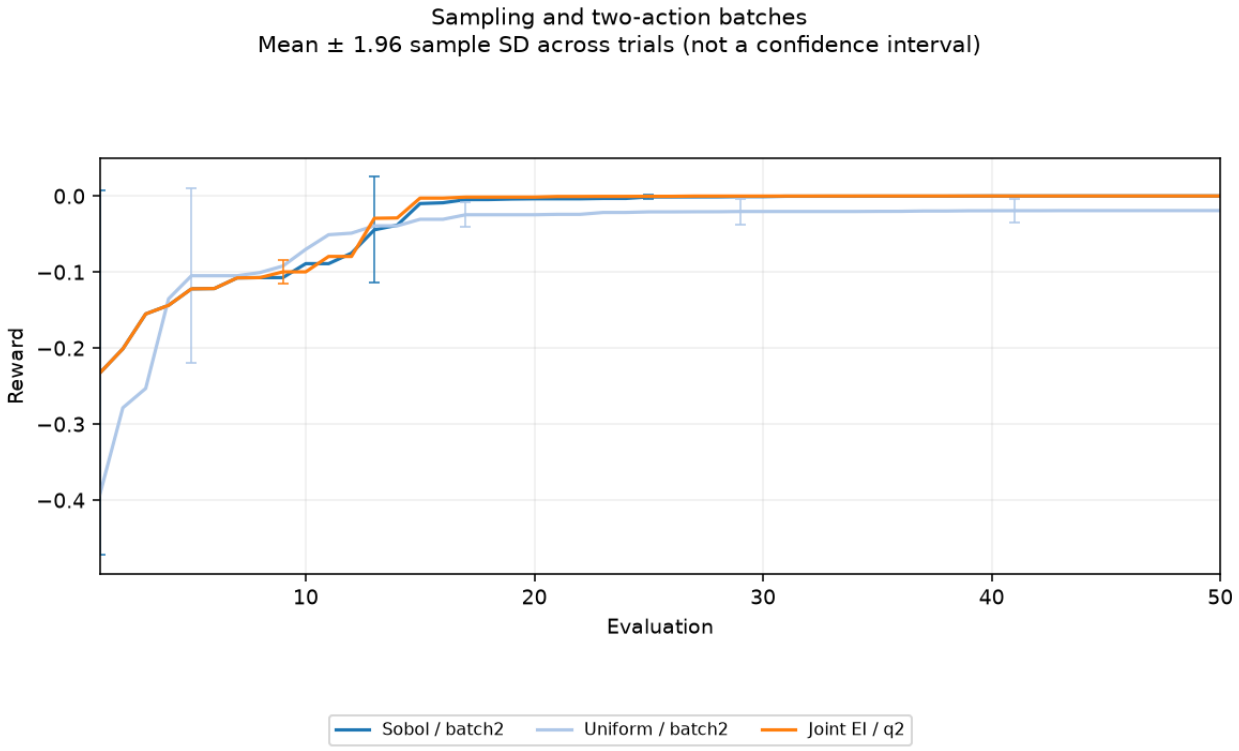
**Editable Python / block 19**

```
import bayopy

seed = bayopy.set_seed(413, threads=1)
run_directory = bayopy.new_run_directory("results/last_guide/sampling_batches")
bounds = bayopy.Bounds(lower=[0.0] * 3, upper=[1.0] * 3)

def objective(values):
    return -sum((x - 0.6) ** 2 for x in values)

algorithms = {
    "Sobol / batch2": bayopy.TuRBO(
        bounds=bounds, config=bayopy.TuRBOConfig(
            initial_points=8, candidate_count=128, batch_size=2, seed=seed,
        ), dtype="float64",
    ),
    "Uniform / batch2": bayopy.TuRBO(
        bounds=bounds, config=bayopy.TuRBOConfig(
            initial_points=8, candidate_count=128, batch_size=2, seed=seed,
        ), dtype="float64",
    ),
    "Joint EI / q2": bayopy.TuRBO(
        bounds=bounds, config=bayopy.TuRBOConfig(
            initial_points=8, candidate_count=128, batch_size=None, q_batch=2,
            acquisition=bayopy.AcquisitionConfig(
                name="EI", mc_samples=64, seed=seed,
            ), seed=seed,
        ), dtype="float64",
    ),
}
experiment = bayopy.Experiment(
    environment=bayopy.StochasticEnvironment(
        bounds=bounds, function=objective, dtype="float64", seed=seed,
    ), algorithm=algorithms, output_directory=run_directory,
    initial_points_sampling_method={
        "Sobol / batch2": "sobol", "Uniform / batch2": "uniform",
        "Joint EI / q2": "sobol",
    },
    acquisition_sampling_method={
        "Sobol / batch2": "sobol", "Uniform / batch2": "uniform",
        "Joint EI / q2": "sobol",
    },
)
result = experiment.run(trials=5, max_evaluations=50, plot=False, progress=True)
print(experiment.summary())
figure = bayopy.plot(
    experiment, kind="incumbent", title="Sampling and two-action batches",
    xlim=(1, 50), save_path=bayopy.join_path(run_directory, "batches.png"),
)
```




Captured figure 14 / settings sampling batches

## Save, resume, and explicit display

A single run writes into its chosen directory. A collection adds readable `algorithms/NN-label/` subdirectories and a combined `results.json`. Each method's settings, observations, diagnostics, and checkpoints remain separate. The runner saves final records even if a periodic `save_every` checkpoint is not due. Repeated experiments add `trials/trial-001/` through `trials/trial-005/`; each trial contains its own method directories. Resuming uses total ceilings: resuming 10 evaluations with a cap of 50 adds at most 40 per trial. A fresh object must match the saved configuration.

`plot=False` disables the automatic progress plot. `plot=True, show=False` creates it without displaying it. `show=True` displays it on a capable backend. None of these settings change rewards or warning output. `bayopy.plot` controls title, axes, limits, legends, file saving, and display; assign its result to avoid notebook final-expression display when show=False. `bayopy.show` can display the retained plot explicitly.

### Editable Python / block 20

```
import bayopy

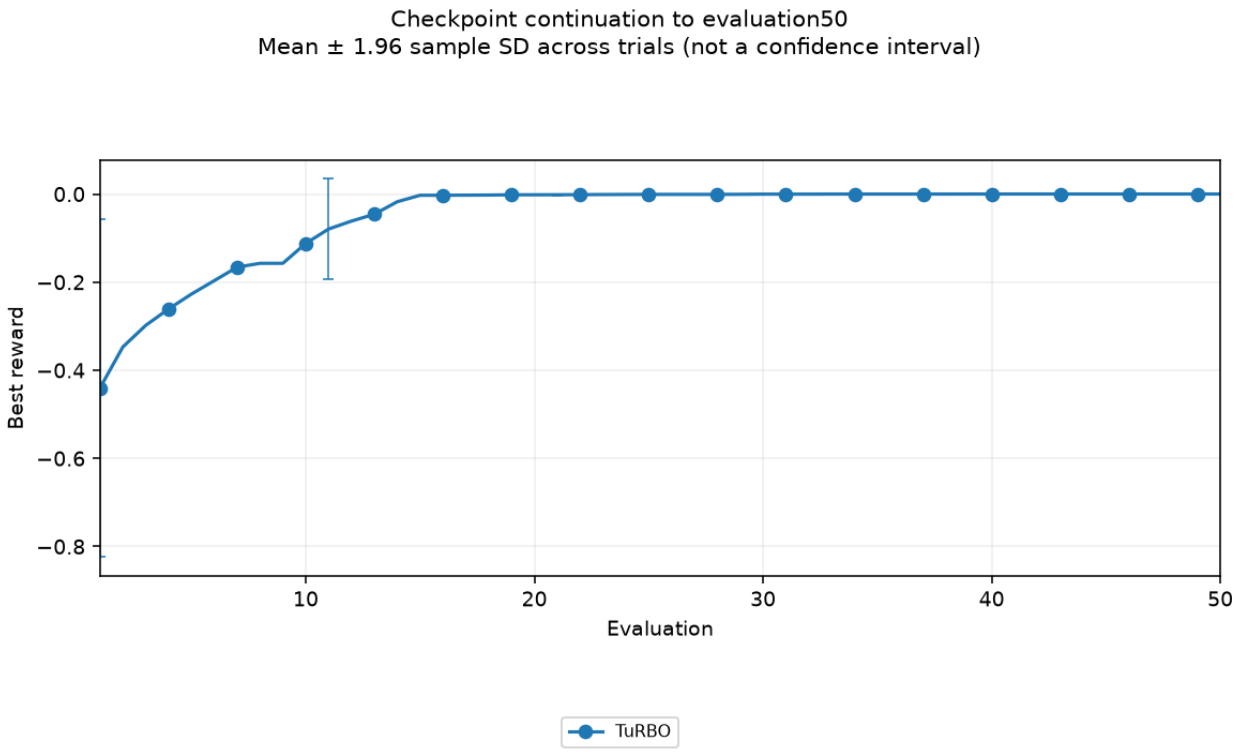
seed = bayopy.set_seed(414, threads=1)
run_directory = bayopy.new_run_directory("results/last_guide/resume")
bounds = bayopy.Bounds(lower=[0.0] * 3, upper=[1.0] * 3)

def objective(values):
    return -sum((x - 0.45) ** 2 for x in values)

def make_experiment(resume):
    return bayopy.Experiment(
        environment=bayopy.StochasticEnvironment(
            bounds=bounds, function=objective, dtype="float64", seed=seed,
        ),
        algorithm=bayopy.TuRBO(
            bounds=bounds, config=bayopy.TuRBOConfig(
                initial_points=8, candidate_count=128, seed=seed,
            ), dtype="float64",
        ), output_directory=run_directory, resume=resume, save_every=5,
```

```
    )

first = make_experiment(False)
first_result = first.run(trials=5, max_evaluations=10, plot=True)
resumed = make_experiment(True)
resumed_result = resumed.run(trials=5, max_evaluations=50, plot=True, show=False)
print("Before:", first_result.evaluations, "After:", resumed_result.evaluations)
print(resumed.summary())
figure = bayopy.plot(
    resumed, kind="incumbent", title="Checkpoint continuation to evaluation50",
    xlabel="Evaluation", ylabel="Best reward", xlim=(1, 50),
    grid=True, legend=True, legend_columns=1, marker="o",
    save_path=bayopy.join_path(run_directory, "resumed.png"), show=False,
)
print("show=False retained the plot. Display it explicitly next:")
bayopy.show(figure)
```




Captured figure 15 / settings resume

# Appendix A. Algorithm adaptations across environments

This appendix describes the current BayOPy implementation. An adaptation preserves the original environment, its evaluator, candidate restrictions, noise, and actual rewards. It supplies missing learner inputs or ignores unused metadata; it does not replace a distributional, contextual, or adversarial environment with a stochastic environment.

An algorithm outputs an action or evaluation query. The environment returns the scalar reward and feedback. Here, x denotes the selected action, x_eval the actual evaluated location, c a context, C finite context support, and p reference probabilities over C. Information must be available at the correct time: a future sampled context must never be revealed before selection.

## A.1. Stochastic algorithms

The 14 stochastic-family algorithms are REMBO, HeSBO, ALEBO, ADD_GP_UCB, TuRBO, Dropout, MCTS_VS, SAASBO, AG_VS, LA_MCTS, PG, EG, GS, and Satisficing_TS. Their shared environmental inputs are the action domain or available candidates and previous actions and rewards. Their output is the next action, or a batch where supported. They do not require contexts or reference distributions to choose an action.

Their modeling requirements differ: REMBO, HeSBO, and ALEBO use low-dimensional embeddings; ADD_GP_UCB uses additive groups; TuRBO uses local trust regions; Dropout, MCTS_VS, and AG_VS select variables; LA_MCTS partitions the domain; SAASBO uses a sparse-axis GP prior and posterior inference. PG, EG, GS, and Satisficing_TS use their respective goodness, improvement, or posterior-sampling criteria. Embedding dimension, kernels, confidence settings, and thresholds remain algorithm choices, not environment observations.

### A.1.1. Native stochastic environment

The algorithm selects x, the environment evaluates the stochastic objective, and the observed reward updates the learner. Normally x_eval equals x. Compatibility still depends on each method's assumptions, such as low effective dimensionality for embedding algorithms; successful execution does not establish those assumptions.

### A.1.2. Stochastic algorithms in an adversarial environment

The learner selects an action with its ordinary policy. The real adversarial environment applies its perturbation or response and returns the actual reward. Unneeded attack metadata is ignored; the learner does not acquire an adversarial acquisition rule.

REMBO, HeSBO, and ALEBO specifically retain the nominal action's embedded coordinates and associate the resulting reward with those coordinates. They therefore learn the response obtained by submitting that action, including the attack's effect. A changing attack can make that response nonstationary. This is a warned extension, not a claim of adversarial robustness.

### A.1.3. Stochastic algorithms in a distributional environment

The learner chooses x without conditioning on support C or probabilities p. The environment then samples its real context and evaluates the objective under that context. The learner consumes the actual reward. Public reference probabilities do not replace the environment's actual sampling mechanism.

With stationary sampling, the observations can support learning average performance over encountered contexts. These algorithms do not explicitly optimize a worst-case distributional objective. Changing distributions can make their observed response nonstationary.

### A.1.4. Stochastic algorithms in a contextual environment

The environment reveals its current context, but the learner ignores it as a model input. It still honors the currently available actions, including context-dependent CSV candidate restrictions. The reward comes from the real contextual evaluator.

The same action can produce different rewards in different contexts. Ignoring context does not provide a context-dependent decision policy, nor does it permit selecting unavailable rows.

### A.1.5. Finite real-data adaptation

All finite-data policies select existing available rows; missing rewards are not invented or interpolated. REMBO uses least-squares row compression, HeSBO uses signed bucket averages, and ALEBO uses its B projection. With the default finite_candidate_mode="project", each scores real candidate rows in compressed coordinates and returns the original row. This is a separately warned finite-candidate extension. The "exact" mode retains stricter embedding-geometry checks.

ADD_GP_UCB scores complete rows instead of combining unrelated group optima. TuRBO restricts rows to its trust region. Dropout, MCTS_VS, and AG_VS score complete candidates through selected coordinates. LA_MCTS enforces its classifier path. SAASBO and the satisficing methods evaluate their criteria on the available candidates. The environment retains control of context availability, distributional reward coverage, and actual evaluations.

## A.2. Adversarial and robust algorithms

BayOPy groups AdveRS and StableOPT in this family. AdveRS needs a feasible action domain and rewards at actual evaluated locations, with perturbation information when available. It outputs an action using its robust-satisficing criterion. Its hidden actual attack budget is not required as a learner input.

StableOPT needs a domain and a chosen robustness neighborhood. It selects a robust center using a worst-neighborhood upper-confidence criterion, then an evaluation query using a lower-confidence criterion within the neighborhood. Feedback supplies the actual evaluated location and reward. Robust optimization of a stochastic function is a native StableOPT use case: an external attacker is not required. Its robustness radius and metric define its optimization criterion and remain legitimate user settings.

### A.2.1. Adversarial environment

Environment-provided evaluated coordinates and perturbations take precedence. Both implementations train their GP at the location where the reward was actually observed. For StableOPT, an external attack is an additional extension because it can move the evaluation away from the query chosen by its robust procedure.

### A.2.2. Adversarial/robust algorithms in a stochastic environment

For AdveRS, missing x_eval defaults to x. Missing perturbation defaults to x_eval minus x, which is zero when no attack or displacement is reported. Supplied displacement information is preserved. The source evaluates its original stochastic objective, while AdveRS retains its robust-satisficing criterion.

StableOPT can operate directly on the stochastic objective. Its chosen robustness neighborhood remains active even when the source applies no external attack. No attack does not mean a zero robustness radius.

### A.2.3. Adversarial/robust algorithms in a distributional environment

The learner selects an action or query, and the distributional source samples its context and returns the corresponding reward. These policies ignore reference probabilities and context as conditioning inputs. Actual evaluated coordinates are preserved; missing external-attack metadata uses the defaults above.

Their action-space robustness criterion is not a distributional robustness criterion. Variation caused by context remains part of their observed response. The run proceeds with a cross-environment diagnostic.

### A.2.4. Adversarial/robust algorithms in a contextual environment

The source retains its actual context and any context-dependent candidate restrictions. The algorithms select without conditioning their GP on that context and receive the true context-dependent reward. AdveRS defaults to zero external displacement when none is reported; StableOPT retains its selected robustness neighborhood.

Neither adaptation implements CGP_UCB's contextual policy. In finite data, required neighborhood queries must exist as available rows. A fallback cannot manufacture missing evaluations.

## A.3. Distributional algorithms

The native algorithms are DRBO and RoBOS. Both need public finite support C and reference probabilities p before selection, and the realized context and reward afterward. They output an action before the future context realization is known. DRBO additionally needs an ambiguity margin epsilon. RoBOS uses its aspiration-based formulation and does not require DRBO's ambiguity margin.

### A.3.1. Native distributional environment

The environment supplies support and reference information before selection and reveals the actual realization with the reward. The private sampling distribution remains private. Native support and its ordering stay fixed; references and DRBO's margin may vary between rounds. Algorithm configuration retains tunable kernels, confidence, divergence/solver choices, and aspiration where applicable.

### A.3.2. Distributional algorithms in a stochastic environment

When all context and distribution inputs are absent, context defaults to a zero vector, support to the singleton containing that vector, probabilities to [1.0], and DRBO's ambiguity margin to 0. The source evaluates its original objective; the observation is recorded with the constant context.

The distributional calculation reduces to a singleton-context problem. It is executable but does not create meaningful uncertainty across multiple contexts.

### A.3.3. Distributional algorithms in an adversarial environment

The same singleton defaults apply if no context or distribution information is provided. The actual adversarial source still applies its attack and returns the resulting reward and evaluated location.

A context-distribution fallback does not model the attacker. In particular, DRBO's ambiguity margin is not an action-perturbation budget. Execution continues as a warned extension without an adversarial robustness claim.

### A.3.4. Distributional algorithms in a contextual environment

If the source reveals c_t but no distribution, the round's support becomes the singleton {c_t}, its reference probabilities are [1.0], and DRBO's margin defaults to 0. Selection uses this revealed context. The joint GP stores actual historical contexts and rewards.

The singleton integration support changes when the revealed context changes. Historical contexts are not overwritten. This is an explicitly warned extension of the native fixed-support distributional formulation.

For every adaptation, defaults apply independently: supplied support is retained, and supplied reference

probabilities override fallback probabilities. When no reference is supplied, use a point mass on a provided current context when represented in support, otherwise uniform probabilities over known support when no current context is available. Malformed or incompatible supplied information is not silently repaired.

## A.4. Contextual algorithms

CGP_UCB is the native contextual algorithm. It needs the current context before selecting an action, the action domain or candidates, and previous observations. It models action-context pairs and outputs an action chosen for the decision context.

### A.4.1. Native contextual environment

The source reveals c_t before selection. CGP_UCB chooses an action using its joint GP and trains on the corresponding action-context observation after receiving the reward. Native feedback context must agree with the decision context.

### A.4.2. CGP_UCB in a stochastic environment

Missing context becomes an internal zero vector, used for both selection and observation. Its dimension follows public support or the context-kernel dimension when known, otherwise one. The source still evaluates its actual objective. A constant context supplies no changing contextual information for the learner to exploit.

### A.4.3. CGP_UCB in an adversarial environment

Missing context uses the same zero-vector default. The adversarial source still applies its attack, and its actual evaluated location and reward are retained. The learner uses the default context for the observation. This does not add an adversarially robust decision criterion; warnings identify the family mismatch and missing context.

### A.4.4. CGP_UCB in a distributional environment

The actual future realization is unavailable before selection. The current automatic adaptation therefore selects using the zero-context default. After the source samples and evaluates, the learner trains using the actual context revealed with the reward. Decision context and training context can consequently differ in this extension.

The automatic adaptation does not use the distribution mean as the decision context. A separate explicit distribution_mean_context adapter is another modeling choice. Neither choice reproduces native contextual information timing; performance can suffer when the default decision context is unrepresentative.

## A.5. Shared precedence, warnings, and limits

Environment-provided values override internal defaults independently for each field. Extra metadata is ignored by algorithms that do not need it, but can still influence the source's actual reward. Missing inputs receive defaults only when those defaults make execution well-defined. Standalone historical initialization likewise retains supplied contexts and warns when substituting missing historical values.

Context observations, support, reference distributions, and actual external perturbations belong to the environment. They are not ordinary AlgorithmConfig parameters. Scientific choices such as kernels, confidence levels, aspiration, embedding dimensions, and StableOPT's chosen robustness criterion remain tunable.

Cross-family diagnostics identify the native-family mismatch and any expected information that was missing, together with the fallback actually used. Successful execution does not transfer published guarantees. Invalid

supplied values, incompatible model dimensions, and impossible candidate constraints remain errors or explicit stopping conditions. Finite exhaustion stops cleanly; an insufficient atomic batch is not completed with fabricated rows.

This appendix documents the implementation and its limitations; it adds no new experimental measurements. The earlier five-trial results and their recorded numerical diagnostics remain unchanged.